

<< CIVADO >>



Universidad de Castilla-La Mancha

Escuela Superior de Ingeniería Informática de Albacete

Escuela Superior de Informática de Ciudad Real

Trabajo Fin de Máster

Máster Universitario en Ingeniería Informática

Simulador web de escenarios de movilidad urbana mediante técnicas de Inteligencia Artificial

Iván Vicente Hernández García de Mora

Noviembre, 2025

TRABAJO FIN DE MÁSTER

Máster Universitario en Ingeniería Informática

Simulador web de escenarios de movilidad urbana mediante técnicas de Inteligencia Artificial

Autor: Iván Vicente Hernández García de Mora

Director: Nombre del director

Codirector: Nombre del codirector

*Aquí va la dedicatoria que cada cual
quiera escribir. El ancho se controla
manualmente*

Declaración de autoría

Yo, ... , con DNI ... , declaro que soy el único autor del trabajo fin de grado titulado “ ... ”, que el citado trabajo no infringe las leyes en vigor sobre propiedad intelectual, y que todo el material no original contenido en dicho trabajo está apropiadamente atribuido a sus legítimos autores.

Albacete, a ... de ... de 20 ...

Fdo.: Iván Vicente Hernández García de Mora

Resumen

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Agradecimientos

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Índice general

1	Introducción	1
1.1	Contexto y motivación	1
1.2	Objetivos del TFM	1
1.2.1	<i>Objetivo general</i>	1
1.2.2	<i>Objetivos específicos</i>	1
1.2.3	<i>Competencias trabajadas</i>	2
1.3	Alcance y limitaciones	2
2	Estado del arte y marco tecnológico	3
2.1	Fundamentos y enfoques	3
2.2	Datos abiertos para movilidad	3
2.2.1	<i>OpenStreetMap</i>	4
2.2.2	<i>GTFS</i>	4
2.3	Tecnologías abiertas de enrutado	5
2.3.1	<i>OSRM</i>	6
2.3.2	<i>OpenTripPlanner</i>	6
2.4	Plataformas propietarias y servicios gestionados	7
2.4.1	<i>Google Maps Platform</i>	7
2.4.2	<i>Otras plataformas de referencia</i>	8
2.5	Modelado de elección modal	9
2.5.1	<i>Modelos de utilidad aleatoria</i>	9
2.5.2	<i>Modelo Logit Multinomial</i>	10
2.5.3	<i>Limitaciones del enfoque clásico</i>	12
2.5.4	<i>Aprendizaje automático para elección modal</i>	13
2.5.5	<i>Random Forests</i>	13

2.5.6	<i>Gradient Boosting Decision Trees</i>	14
2.5.7	<i>Redes Neuronales Profundas</i>	15
3	Metodología	17
3.1	SCRUM adaptado a un desarrollo unipersonal	17
3.2	Product Backlog	20
3.2.1	<i>Backlog inicial</i>	20
3.2.2	<i>Priorización de items</i>	21
3.3	Histórico de sprints ejecutados	21
3.3.1	<i>Sprint 1: Validación inicial de OSRM mediante servicios remotos</i>	22
3.3.2	<i>Sprint 2: Despliegue del servicio OSRM local multi-perfil</i>	23
3.3.3	<i>Sprint 3: Incorporación del GTFS urbano de Toledo</i>	23
3.3.4	<i>Sprint 4: Integración de OpenTripPlanner para rutas multimodales</i>	24
3.3.5	<i>Sprint 5: Diseño de la interfaz web y codificación visual por modo</i>	24
3.3.6	<i>Sprint 6: Diseño de un modelo de clasificación de modo de viaje y procesado de datos</i>	24
3.3.7	<i>Sprint 7: Entrenamiento y validación de los modelos de clasificación</i>	25
3.3.8	<i>Sprint 8: Escritura de la memoria y consolidación metodológica</i>	25
3.3.9	<i>Sprint 9: Integración del módulo de inferencia en el simulador</i>	26
3.3.10	<i>Sprint 10: Reestructuración del repositorio y documentación de la arquitectura</i>	26
3.3.11	<i>Sprint 11: Entrenamiento de modelos adicionales y comparativa multi-modelo</i>	27
3.3.12	<i>Sprint 12: Redacción del capítulo de implementación y primer rediseño de la interfaz</i>	27
3.3.13	<i>Sprint 13: Consolidación de la interfaz y preparación del repositorio para entrega</i>	28
3.3.14	<i>Sprint 14: Pulido final de la aplicación y revisión integral de la memoria</i>	28
3.4	Retrospectiva del ciclo de desarrollo	29
3.4.1	<i>Aprendizajes por iteración</i>	29
3.4.2	<i>Acciones de mejora aplicadas</i>	30
3.5	Cierre del ciclo de desarrollo	31
4	Arquitectura del sistema	33
4.1	Visión general.	33
4.2	Infraestructura, orquestación y portabilidad.	34
4.3	Backend: API y routers	38
4.4	Frontend: interfaz cartográfica	39
4.5	Pipeline de inferencia modal	40

5	Implementación y resultados	43
5.1	Infraestructura de enrutado viario: OSRM	43
5.1.1	Origen y características del extracto OSM	43
5.1.2	Evolución del despliegue	44
5.1.3	Pipeline de preprocesado por perfil	45
5.1.4	Despliegue y verificación	46
5.2	Transporte público: OpenTripPlanner y feed GTFS	48
5.2.1	Fuentes de datos GTFS y proceso de selección	48
5.2.2	Construcción del grafo multimodal	49
5.2.3	Estructura del feed GTFS y particularidades del feed de Toledo	50
5.2.4	Periodo de validez del feed y selección de fecha	51
5.2.5	Integración de itinerarios en el backend	52
5.3	Implementación del backend FastAPI	53
5.3.1	Estructura modular	53
5.3.2	Router OSRM: consultas multiperfil	53
5.3.3	Router OTP: itinerarios multimodales	54
5.3.4	Router GTFS: capa estática	54
5.3.5	Router LPMC: inferencia modal y descubrimiento de modelos	55
5.4	Implementación del frontend	56
5.4.1	Tecnologías y estructura	56
5.4.2	Interfaz cartográfica y controles del mapa	57
5.4.3	Panel Inicio	58
5.4.4	Panel Rutas	59
5.4.5	Panel Red GTFS	61
5.4.6	Panel Predicción IA	62
5.4.7	Panel Capas	64
5.4.8	Panel Ajustes	65
5.4.9	Gestión del estado con TanStack Query	65
5.5	Modelos de elección modal	66
5.5.1	Dataset LPMC y variables de entrada	66
5.5.2	Preprocesado de datos	67
5.5.3	Ajuste de hiperparámetros	69
5.5.4	Entrenamiento y evaluación	72
5.5.5	Ajustes y conocimiento experto	74
6	Conclusiones y trabajo futuro	77
6.1	Conclusiones técnicas	77
6.2	Cumplimiento de objetivos	77

6.3 Trabajo futuro	77
A Anexos.....	79
A.1 Manual de despliegue y ejecución	79
A.1.1 <i>Requisitos del entorno.</i>	79
A.1.2 <i>Despliegue automático en un comando</i>	79
A.1.3 <i>Generación manual de los artefactos de enrutado.</i>	80
A.1.4 <i>Verificación.</i>	81
A.2 Endpoints y contratos de la API	81
A.2.1 <i>Rutas viarias multiperfil (POST /api/osrm/routes).</i>	81
A.2.2 <i>Itinerario de transporte público (POST /api/otp/routes)</i>	82
A.2.3 <i>Inferencia de elección modal (POST /api/lpmc/predict)</i>	82
A.3 Configuración experimental y reproducibilidad	83
A.4 Evidencias de sprints	85
Referencia bibliografica.....	91

Índice de figuras

2.1	Forma sigmoideal ilustrativa de la probabilidad de elección en el modelo MNL. Elaboración propia.	11
3.1	Esquema del ciclo de trabajo SCRUM adaptado al desarrollo unipersonal del proyecto. Elaboración propia.	19
3.2	Cronograma aproximado de los sprints ejecutados durante el desarrollo del TFM, representado como diagrama de Gantt simplificado. Elaboración propia.	22
4.1	Arquitectura general del simulador de movilidad urbana.	34
4.2	Servicios Docker Compose del simulador: dependencias y puertos expuestos.	37
4.3	Pipeline de inferencia de elección modal.	40
5.1	Pipeline de preprocesado OSRM por perfil de transporte.	46
5.2	Rutas OSRM para los tres perfiles de transporte sobre Toledo.	48
5.3	Cruces entre los ficheros del feed GTFS para resolver las consultas de la capa estática.	50
5.4	Documentación OpenAPI generada automáticamente por FastAPI.	53
5.5	Vista general de la interfaz del simulador de movilidad urbana.	58
5.6	Itinerario multimodal OTP con desglose de tramos en la interfaz.	60
5.7	Panel Rutas con resultados OSRM e itinerario OTP.	61
5.8	Panel Red GTFS con trazado, paradas y horarios de una línea.	62
5.9	Panel Predicción IA con el formulario y el resultado de la inferencia.	63
5.10	Panel de comparación de probabilidades de los tres modelos de elección modal.	64
5.11	Ventana de depuración del vector de características que recibe el modelo. .	64
5.12	Panel de ajustes con selector de modelo activo y documentación de extensión.	66
5.13	Arquitectura de la red neuronal profunda de elección modal.	72

Índice de tablas

2.1	Comparación resumida de las soluciones de enrutado consideradas en este trabajo. Elaboración propia a partir de [Project OSRM, 2026a, OpenTripPlanner Team, 2026b, Google, 2026b, Google, 2026a].	8
4.1	Artefactos versionados mediante Git LFS.	35
4.2	Servicios Docker Compose del simulador.	36
4.3	Endpoints principales de la API REST del backend.	38
5.1	Comparativa de extractos OSM disponibles en Geofabrik.	44
5.2	Características generales del dataset LPMC.	67
5.3	Hiperparámetros del Random Forest hallados por la búsqueda propia (HyperOpt/TPE, 40 evaluaciones).	69
5.4	Hiperparámetros de XGBoost adoptados de la búsqueda de referencia (HyperOpt/TPE, 1.000 evaluaciones).	70
5.5	Configuración de la red neuronal profunda hallada por la búsqueda propia.	71
5.6	Resultados en entrenamiento con validación cruzada de 5-folds agrupada por hogar (dataset LPMC).	74
5.7	Resultados en el conjunto de prueba independiente (dataset LPMC).	74
A.1	Variables de entrada al modelo de elección modal. Las variables de ruta se derivan automáticamente de OSRM y OTP; las sociodemográficas las aporta el usuario a través de la interfaz.	84

Índice de algoritmos

Índice de listados de código

- 5.1 Verificación de las tres instancias OSRM desde el terminal del anfitrión. El parámetro `overview=full` fuerza la devolución de la geometría completa de la ruta; sin él, OSRM devuelve una versión simplificada con menos puntos. 47

1. Introducción

1.1. Contexto y motivación

1.2. Objetivos del TFM

1.2.1. Objetivo general

Desarrollar un prototipo de simulador web de movilidad urbana que integre modelos de predicción basados en Machine Learning y servicios de enrutado, con el fin de analizar el impacto de distintas políticas de transporte en el reparto modal de los viajes y generar métricas de apoyo para la planificación y la toma de decisiones.

1.2.2. Objetivos específicos

- Entrenar y validar modelos de Machine Learning con datos de movilidad para predecir la elección modal de los usuarios en función de variables sociodemográficas, de tiempo y de coste.
- Integrar servicios de enrutado (OSRM y OpenTripPlanner) para obtener tiempos y costes realistas entre pares origen-destino en el entorno urbano.
- Desarrollar una aplicación web interactiva que permita definir viajes sobre un mapa, simular el comportamiento de los usuarios y visualizar resultados en un cuadro de mando.
- Implementar un módulo de análisis de escenarios *what-if* para modificar parámetros de política (coste del coche, tarifas del transporte público, frecuencia de autobuses, etc.) y observar su efecto en el reparto modal.

-
- Generar métricas de impacto (reparto modal, tiempo medio de viaje y estimación de emisiones de CO₂) como apoyo a la toma de decisiones en movilidad urbana.

1.2.3. Competencias trabajadas

El desarrollo del TFM contribuye a las siguientes competencias del Máster:

- **CP01:** Integrar tecnologías, aplicaciones, servicios y sistemas de la Ingeniería Informática en un contexto multidisciplinar.
- **CP05:** Aplicar métodos matemáticos, estadísticos y de inteligencia artificial para modelar y desarrollar sistemas inteligentes.
- **CP07:** Realizar un proyecto integral original de carácter profesional que sintetice las competencias adquiridas en el plan de estudios.

1.3. Alcance y limitaciones

2. Estado del arte y marco tecnológico

2.1. Fundamentos y enfoques

La movilidad urbana se ha consolidado como un ámbito de estudio en el que confluyen la ingeniería del transporte, la analítica de datos y la inteligencia artificial. En este contexto, los simuladores actuales no se limitan a representar mapas o a calcular rutas aisladas, sino que actúan como herramientas para evaluar políticas y escenarios de movilidad, permitiendo analizar de forma anticipada cómo cambios en los costes, en la oferta de transporte o en las condiciones operativas pueden modificar el comportamiento de la población.

Este tipo de sistemas exige combinar varias capas de conocimiento. Por un lado, la capa de datos, que determina la calidad y la trazabilidad de la información sobre la red de transporte y los servicios de transporte público asociados. Por otro, la capa de enrutado, que permite transformar un par origen-destino en variables de servicio observables (tiempo, distancia, segmentos de viaje, transbordos o coste aproximado). Finalmente, la capa de modelado, en la que dichas variables se integran con atributos sociodemográficos para estimar la elección modal y construir escenarios *what-if*. Esta arquitectura en capas es coherente con la evolución reciente de herramientas de simulación en investigación y planificación [Horni et al., 2016, Lopez et al., 2018].

2.2. Datos abiertos para movilidad

El desarrollo de los sistemas descritos en el apartado anterior requiere la disponibilidad de grandes volúmenes de datos fiables y reutilizables. Por este motivo, en los últimos años han adquirido gran protagonismo los conjuntos de datos abiertos en movilidad, ya que

permiten construir, validar y reproducir modelos de análisis sin depender de fuentes propietarias cerradas. En este contexto, OpenStreetMap y GTFS se han consolidado como los dos pilares más utilizados para representar, respectivamente, la red urbana y la oferta de transporte público.

2.2.1. OpenStreetMap

OpenStreetMap (OSM) es una base de datos geográfica abierta y colaborativa ampliamente utilizada en proyectos de movilidad, tanto en investigación como en aplicaciones operativas. La propia fundación del proyecto define OSM como una fuente de datos abiertos reutilizable por terceros, con cobertura global y actualización continua por parte de la comunidad [OSM Foundation, 2026]. Esta naturaleza abierta resulta especialmente relevante en un trabajo experimental, ya que permite reproducir resultados a partir de un conjunto de datos público, versionable e independiente de condiciones comerciales impuestas por terceros.

En términos de calidad cartográfica, diversos estudios han mostrado que OSM puede ofrecer niveles de precisión adecuados para el análisis del transporte, aunque con cierta heterogeneidad espacial entre zonas y entre tipos de elemento, por ejemplo, entre la red viaria principal y los detalles de la infraestructura peatonal [Haklay, 2010]. Por ello, en este trabajo OSM se considera una fuente adecuada para construir la red de enrutado, teniendo en cuenta las características específicas del área de estudio al interpretar los resultados.

2.2.2. GTFS

GTFS (*General Transit Feed Specification*) es el estándar abierto dominante para publicar oferta de transporte público programada. La documentación oficial describe de forma precisa el contrato de datos entre productor y consumidor: estructura de ficheros, relaciones entre entidades y restricciones semánticas necesarias para que un planificador pueda reconstruir correctamente servicios, horarios y calendarios [MobilityData, 2026c]. En otras palabras, GTFS no es solo un formato de intercambio, sino también un marco común de interoperabilidad entre operadores, agregadores y motores de planificación.

La estructura del estándar se basa en un conjunto de ficheros fundamentales, entre los que destacan `stops.txt`, `routes.txt`, `trips.txt`, `stop_times.txt`, `calendar.txt` y `calendar_dates.txt`. Estos archivos permiten representar simultáneamente la topología de la red y la dimensión temporal del servicio [MobilityData, 2026c]. Esta separación resulta esencial, ya que el componente espacial define dónde se presta el servicio y el componen-

te temporal determina cuándo se presta. En un entorno de simulación multimodal, ambos planos deben mantenerse coherentes para evitar itinerarios inviables o resultados inconsistentes.

En el contexto español, el canal institucional más relevante para localizar y descargar feeds GTFS es el *Punto de Acceso Nacional de Transporte Multimodal* (NAP), gestionado por el Ministerio de Transportes y Movilidad Sostenible. La propia descripción oficial del portal lo define como el punto único nacional para concentrar información de oferta de transporte de viajeros, en línea con el marco europeo de datos de movilidad [NAP Transporte Multimodal, 2026a, NAP Transporte Multimodal, 2026b].

En este TFM, la descarga de datos para transporte urbano se realizó precisamente desde NAP. Como ejemplo de referencia nacional se consultó el conjunto de EMT Valencia y, para el caso de estudio del proyecto, el conjunto de autobuses urbanos de Toledo [EMT Valencia, 2026, Unauto, 2026]. En ambos casos, la página de detalle expone metadatos operativos (por ejemplo, número de paradas y formatos de descarga), así como los atributos incluidos en el conjunto de datos, como la accesibilidad, las tarifas o la geometría de las rutas. Esta información permite verificar de forma rápida el tamaño y la estructura del feed antes de integrarlo en el flujo de procesamiento del simulador.

La propia comunidad GTFS publica recomendaciones de calidad de publicación (persistencia de identificadores, estabilidad de URL de feed, cobertura temporal mínima y buenas prácticas de consistencia), que son relevantes para garantizar explotación automatizada en entornos de producción o experimentación [MobilityData, 2026b]. Además, el validador canónico mantenido por MobilityData se ha convertido en herramienta estándar para control de calidad de feeds estáticos antes de su uso analítico [MobilityData, 2026d]. En este trabajo, GTFS cumple una doble función: por un lado, alimentar la capa de consulta de la red de autobús urbano y, por otro, proporcionar los datos necesarios para construir el grafo multimodal empleado posteriormente por OpenTripPlanner.

2.3. Tecnologías abiertas de enrutado

Una vez descritas las fuentes de datos que permiten representar la red urbana y la oferta de transporte público, la segunda pieza fundamental de un simulador de movilidad es el algoritmo de enrutado. Estos motores permiten transformar la información geoespacial y temporal en itinerarios concretos, así como obtener variables de servicio relevantes, como el tiempo de viaje, la distancia recorrida, los transbordos o las fases de acceso y egreso. En este trabajo se emplean dos tecnologías abiertas con funciones complementarias: OSRM,

orientado al cálculo eficiente de rutas sobre red viaria, y OpenTripPlanner, especializado en planificación multimodal con transporte público.

2.3.1. OSRM

OSRM (*Open Source Routing Machine*) es uno de los motores abiertos más utilizados para cálculo de rutas sobre red viaria OSM, especialmente cuando se necesitan tiempos de respuesta bajos y gran volumen de consultas [Project OSRM, 2026a, Project OSRM, 2026b]. Su interés en un simulador de movilidad no está solo en obtener una ruta entre dos puntos, sino en poder generar de forma sistemática atributos de viaje (tiempo, distancia o matrices origen-destino) que después se integran en análisis y modelos predictivos.

La principal fortaleza técnica de OSRM es su enfoque de preprocesado: antes de servir consultas, el sistema transforma el extracto `.osm.pbf`, que es el formato binario comprimido habitualmente utilizado para distribuir datos de OpenStreetMap, en estructuras optimizadas de búsqueda. Este diseño desplaza coste al inicio, pero permite consultas rápidas y estables durante la fase de explotación. Además, OSRM trabaja con perfiles de movilidad diferenciados (por ejemplo, coche, bicicleta y a pie), lo que permite modelar de forma explícita costes de viaje distintos por modo. A nivel interno, OSRM se apoya en algoritmos de aceleración de rutas: MLD (*Multi-Level Dijkstra*), que divide la red en niveles para reducir el espacio de búsqueda en cada consulta, y CH (*Contraction Hierarchies*), que simplifica el grafo con atajos para acelerar rutas largas [Delling et al., 2017, Geisberger et al., 2008].

2.3.2. OpenTripPlanner

OpenTripPlanner (OTP) es un planificador multimodal diseñado para combinar red de calle (OSM) y red de transporte público programado (GTFS) en un único problema de viaje puerta a puerta [OpenTripPlanner Team, 2026b, OpenTripPlanner Team, 2026c]. Su función difiere de la de un motor de enrutado viario convencional, ya que no se limita a calcular el tramo sobre la red de calles, sino que también representa de manera integrada las fases de acceso peatonal, los tiempos de espera, el recorrido en transporte público, los posibles transbordos y el tramo final hasta el destino.

La documentación oficial de OTP describe una arquitectura basada en construcción previa de grafo y posterior explotación por API de planificación. Ese grafo integra simultáneamente geometría de red y programación temporal del servicio, por lo que la consulta de rutas depende de fecha y hora, no solo de coordenadas [OpenTripPlanner Team, 2026a, OpenTripPlanner Team, 2026d]. Esta dimensión temporal es la base para representar trans-

porte público de forma operativa.

En la salida, OTP devuelve itinerarios descompuestos en tramos o segmentos de viaje, denominados *legs* en su propia documentación, con información sobre el modo utilizado, los tiempos parciales, el número de transbordos y los metadatos de cada línea. Esta estructura resulta especialmente útil para el análisis multimodal, ya que permite distinguir con precisión las distintas fases del desplazamiento. Por contraste, motores viarios como OSRM están orientados al cálculo rápido de rutas sobre red de calle y no incorporan de forma nativa la lógica propia del transporte público, con horarios, transbordos y secuencias diferenciadas dentro de un mismo itinerario.

2.4. Plataformas propietarias y servicios gestionados

Junto a las tecnologías abiertas de enrutado, el ecosistema actual de movilidad incluye también plataformas propietarias que ofrecen estos mismos servicios mediante API gestionadas. La relevancia industrial de este problema ha favorecido la aparición de soluciones comerciales muy maduras, orientadas tanto al desarrollo de producto como a la integración rápida en aplicaciones web y móviles. Aunque estas plataformas simplifican el despliegue técnico, también introducen condicionantes específicos en términos de coste, dependencia del proveedor y reproducibilidad.

2.4.1. Google Maps Platform

Google Maps Platform se ha convertido en un estándar de facto en los servicios comerciales de localización y navegación orientados al desarrollo de productos digitales. En particular, *Routes API* ofrece cálculo de rutas multimodo, metadatos de viaje y un ecosistema de integración especialmente maduro para aplicaciones web, móviles y, de forma destacada, entornos basados en Android [Google, 2026b].

Para proyectos de desarrollo de producto, este modelo reduce de forma notable la complejidad operativa, ya que no exige desplegar ni mantener infraestructura propia de enrutado y permite acceder a funcionalidades avanzadas mediante API. Esta característica ha favorecido su adopción en aplicaciones comerciales, especialmente en entornos móviles y ecosistemas integrados con servicios de Google. No obstante, en contextos de investigación aplicada o en escenarios con un volumen elevado de simulaciones, aparecen limitaciones relevantes: por un lado, el coste económico crece con el número de peticiones y con el tipo de servicio consumido; por otro, la ejecución queda condicionada por cuotas de uso, políticas

del proveedor y disponibilidad externa de la plataforma [Google, 2026a]. Estas restricciones reducen su idoneidad cuando se busca trazabilidad completa, control experimental y escalabilidad económica predecible.

2.4.2. Otras plataformas de referencia

Además de Google Maps Platform, el ecosistema actual incluye otras alternativas consolidadas de consumo por API, entre ellas Mapbox [Mapbox, 2026] o TomTom [TomTom, 2026]. Estas plataformas ofrecen catálogos amplios (rutas, matrices, navegación, tráfico y servicios avanzados según proveedor), y comparten una lógica común de uso gestionado mediante credenciales, límites de consumo y planes de servicio.

También existen servicios intermedios como openrouteservice, que proporcionan API pública y, al mismo tiempo, opciones de despliegue propio [HeiGIT gGmbH, 2026, openrouteservice, 2026]. Este tipo de solución puede ser útil como puente entre experimentación rápida y control de infraestructura, aunque con sus propias restricciones de uso en modalidad pública.

La Tabla 2.1 resume esta comparación y la decisión tecnológica adoptada en el proyecto. Para un simulador académico orientado a experimentación, trazabilidad y ejecución repetida de escenarios, la combinación OSRM + OTP en infraestructura propia ofrece un equilibrio más adecuado que una API comercial gestionada, aunque implique una mayor complejidad de despliegue.

Tabla 2.1: Comparación resumida de las soluciones de enrutado consideradas en este trabajo. Elaboración propia a partir de [Project OSRM, 2026a, OpenTripPlanner Team, 2026b, Google, 2026b, Google, 2026a].

Solución	Tipo de red	Coste de uso	Adecuación al TFM
OSRM	Red viaria OSM, perfiles por modo	Infraestructura propia	Muy adecuado para consultas masivas y control experimental
OTP	Red multimodal OSM + GTFS	Infraestructura propia	Muy adecuado para transporte público y análisis puerta a puerta
Google Maps Platform	API comercial multimodo gestionada	Pago por petición y cuotas	Útil como referencia industrial, menos adecuado para reproducibilidad

2.5. Modelado de elección modal

Una vez descrita la infraestructura tecnológica que permite representar la red y calcular itinerarios, es necesario incorporar una capa de modelado que permita analizar el comportamiento de los viajeros. En un simulador de movilidad, no basta con conocer qué alternativas existen para un desplazamiento determinado, sino que también es necesario estimar cuál de ellas es más probable que sea elegida en función de sus atributos de servicio y de las características del usuario. Esta cuestión se aborda habitualmente mediante modelos de elección discreta y, más recientemente, mediante técnicas de aprendizaje automático aplicadas a la predicción de la elección modal [Train, 2009, Martín-Baos, 2023].

En la literatura reciente, una de las referencias más utilizadas para este tipo de problemas es el dataset London Passenger Mode Choice (LPMC), construido a partir de viajes observados en Londres y enriquecido con variables de nivel de servicio para distintos modos de transporte [Hillel et al., 2018, Hillel, 2019]. Su interés en este trabajo no está en el caso de estudio concreto, sino en que proporciona una base metodológica y experimental muy próxima al problema abordado en este TFM.

2.5.1. Modelos de utilidad aleatoria

Este problema se ha abordado tradicionalmente mediante los denominados modelos de utilidad aleatoria (*Random Utility Models*, RUM), ampliamente utilizados en transporte para analizar decisiones de elección discreta, como la elección del modo de transporte. En este enfoque, el decisor elige una alternativa dentro de un conjunto de opciones mutuamente excluyentes, por ejemplo caminar, usar la bicicleta, utilizar transporte público o desplazarse en coche [Train, 2009, McFadden, 1974].

La idea central de estos modelos es que cada viajero asocia una utilidad a cada alternativa disponible dentro de su conjunto de elección, y se asume que elegirá aquella que le proporcione una mayor utilidad. Sin embargo, dicha utilidad no puede observarse de forma completa, ya que parte de los factores que influyen en la decisión sí pueden ser medidos por el analista, mientras que otros permanecen ocultos o no observados. Por ello, la utilidad de una alternativa suele descomponerse en un término determinista y un término aleatorio:

$$U_{in} = V_{in} + \varepsilon_{in} \quad (2.1)$$

donde U_{in} representa la utilidad total que el individuo n asocia a la alternativa i , V_{in} es la componente sistemática o determinista de dicha utilidad, y ε_{in} recoge los factores no

observados que también influyen en la decisión.

Bajo este planteamiento, el viajero se considera racional en el sentido de que selecciona la alternativa que maximiza su utilidad. En consecuencia, la elección observada puede expresarse en términos probabilísticos como la probabilidad de que la utilidad de una alternativa sea mayor o igual que la del resto de alternativas disponibles:

$$P_{in} = P(U_{in} \geq U_{jn} \forall j \in C) \quad (2.2)$$

donde C representa el conjunto de alternativas consideradas. De este modo, los RUM permiten traducir un problema de comportamiento individual en un modelo probabilístico de elección, capaz de relacionar atributos del viaje, características del individuo y probabilidad de seleccionar cada modo de transporte.

2.5.2. Modelo Logit Multinomial

Entre los modelos derivados del enfoque RUM, el más extendido en el análisis de elección modal es el Modelo Logit Multinomial (*Multinomial Logit*, MNL). Su popularidad se debe a que ofrece una formulación matemática sencilla, una interpretación clara de los parámetros y una estimación relativamente eficiente desde el punto de vista computacional. En el contexto del transporte, el MNL ha sido utilizado de forma recurrente para estudiar cómo variables como el tiempo de viaje, el coste, los transbordos o determinadas características sociodemográficas influyen en la probabilidad de elegir un modo de transporte frente a otro [McFadden, 1974].

El modelo MNL se obtiene al asumir que el término aleatorio de la utilidad sigue una distribución Gumbel independiente e idénticamente distribuida para todas las alternativas. Bajo esta hipótesis, la probabilidad de que el individuo n elija la alternativa i puede expresarse de la siguiente forma:

$$P_{in} = \frac{\exp(V_{in})}{\sum_{j=1}^I \exp(V_{jn})} \quad (2.3)$$

donde I es el número total de alternativas disponibles y V_{in} representa la utilidad sistemática de la alternativa i para el individuo n . Esta expresión da lugar a una función de probabilidad de tipo sigmoideal, de manera que pequeñas variaciones en la utilidad representativa tienen mayor efecto cuando la probabilidad de elección se encuentra en valores intermedios que cuando una alternativa ya es claramente dominante o claramente poco atractiva. Esta propiedad resulta especialmente útil para interpretar cambios marginales en los atributos del servicio, por ejemplo ante mejoras en tiempo o coste de una determinada alternativa.

La Figura 2.1 ilustra de forma esquemática esta forma sigmoïdal de la probabilidad de elecci3n en el modelo MNL.

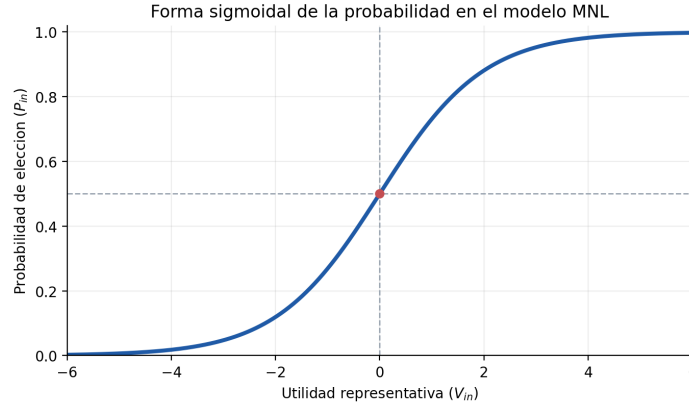


Figura 2.1: Forma sigmoïdal ilustrativa de la probabilidad de elecci3n en el modelo MNL. Elaboraci3n propia.

En la pr3ctica, la componente determinista de la utilidad suele especificarse como una combinaci3n lineal de atributos observables:

$$V_{in} = \beta_{i0} + \sum_k \beta_{ik} x_{ink} \quad (2.4)$$

donde x_{ink} representa el valor del atributo k asociado a la alternativa i para el individuo n , β_{ik} es el par3metro que mide la influencia de dicho atributo y β_{i0} es una constante específica de la alternativa. Esta formulaci3n permite cuantificar el efecto relativo de variables como el tiempo de viaje, el coste monetario, el n3mero de transbordos o el acceso peatonal, entre otras. En t3rminos interpretativos, los par3metros estimados indican c3mo varía la utilidad de cada modo cuando cambian sus atributos observables.

La estimaci3n de los par3metros del modelo se realiza habitualmente mediante máxima verosimilitud. Para ello, se define una funci3n de verosimilitud que mide la probabilidad de observar las elecciones registradas en la muestra dadas unas determinadas estimaciones de los par3metros. El objetivo del proceso de ajuste consiste en encontrar los valores de β que maximizan dicha verosimilitud, o de forma equivalente, que maximizan la log-verosimilitud del modelo. Este procedimiento constituye la base clásica de estimaci3n de modelos de elecci3n discreta y permite obtener un modelo interpretable y coherente con la teoría del comportamiento del viajero.

En este TFM, el modelo MNL resulta relevante como referencia metodológica, ya que proporciona una base teórica sólida para representar la elección modal a partir de atributos del desplazamiento. Además, sirve como punto de comparación frente a enfoques más flexibles basados en aprendizaje automático, que se describen en los apartados siguientes.

Una limitación conocida del modelo MNL es la hipótesis de independencia de alternativas irrelevantes (*Independence of Irrelevant Alternatives*, IIA), derivada de la suposición de errores independientes e idénticamente distribuidos. Esta propiedad implica que la relación de sustitución entre dos alternativas no depende de la presencia o de las características del resto de modos, lo cual puede resultar restrictivo en problemas de movilidad donde algunas opciones están más correlacionadas entre sí que otras [McFadden, 1974, Train, 2009]. Aun así, el MNL sigue siendo el punto de partida más habitual en la literatura por su equilibrio entre simplicidad, interpretabilidad y capacidad descriptiva.

2.5.3. Limitaciones del enfoque clásico

A pesar de su amplia utilización en el análisis de elección modal, los modelos basados en utilidad aleatoria presentan ciertas limitaciones cuando se aplican a problemas complejos de movilidad. En primer lugar, requieren especificar de forma explícita la forma funcional de la utilidad sistemática, lo que implica que el analista debe decidir previamente qué variables incluir y cómo se relacionan con la utilidad de cada alternativa. En muchos casos, esta especificación se realiza mediante combinaciones lineales de atributos del viaje, lo que puede resultar restrictivo cuando las relaciones reales entre variables son no lineales o presentan interacciones complejas [Martín-Baos, 2023, p. 20].

Otra limitación relevante es que los modelos clásicos suelen asumir estructuras relativamente simples en la distribución de los términos de error. En el caso del modelo Logit Multinomial, por ejemplo, la hipótesis de independencia de alternativas irrelevantes puede no reflejar adecuadamente situaciones en las que algunas alternativas presentan correlaciones más fuertes entre sí, como ocurre frecuentemente entre distintos modos de transporte público o entre modos motorizados.

Además, desde un punto de vista práctico, la especificación de modelos econométricos puede requerir un proceso iterativo de selección de variables, transformación de atributos y validación de supuestos teóricos. Aunque este enfoque proporciona modelos interpretables y coherentes con la teoría del comportamiento del viajero, puede resultar menos flexible cuando se trabaja con conjuntos de datos de alta dimensionalidad o con relaciones complejas entre variables.

2.5.4. Aprendizaje automático para elección modal

Con el aumento de la disponibilidad de datos y de la capacidad computacional, en los últimos años han ganado relevancia los enfoques basados en aprendizaje automático para el análisis de elección modal. Desde esta perspectiva, la elección de modo puede formularse también como un problema de clasificación supervisada, en el que el objetivo consiste en predecir la alternativa elegida a partir de un conjunto de variables explicativas relacionadas con el viaje y con las características del individuo.

A diferencia de los modelos clásicos de elección discreta, muchos algoritmos de aprendizaje automático no requieren especificar de forma explícita la forma funcional de la relación entre variables. En su lugar, aprenden patrones directamente a partir de los datos de entrenamiento, lo que les permite capturar relaciones no lineales, interacciones complejas entre atributos y estructuras de decisión difíciles de representar mediante modelos lineales tradicionales.

Entre las técnicas más utilizadas en este contexto destacan los métodos basados en árboles de decisión y los modelos de conjunto (*ensemble models*) derivados de ellos, como Random Forest o Gradient Boosting Decision Trees, así como los modelos basados en redes neuronales profundas. Estos enfoques han mostrado un rendimiento predictivo competitivo en numerosos problemas de clasificación tabular, incluyendo aplicaciones en transporte y elección modal.

En este trabajo se adopta esta perspectiva complementaria: el modelo Logit Multinomial se considera como referencia metodológica clásica, mientras que distintos modelos de aprendizaje automático se utilizan para explorar alternativas con mayor capacidad predictiva. En particular, la arquitectura del simulador se ha diseñado para permitir la integración de diferentes modelos de clasificación, entre ellos Random Forest, Gradient Boosting Decision Trees y redes neuronales profundas, de forma que puedan compararse sus resultados en el contexto del problema de elección modal analizado. Esta comparación resulta coherente con la evidencia reciente disponible para datasets reales de elección modal, entre ellos LPMC [Hillel et al., 2018].

2.5.5. Random Forests

Random Forest es un método de aprendizaje automático basado en la combinación de múltiples árboles de decisión entrenados sobre subconjuntos distintos de los datos. El modelo fue propuesto por Breiman [Breiman, 2001] como una extensión de los árboles de clasificación tradicionales, con el objetivo de mejorar su estabilidad y capacidad predictiva. En

diversas revisiones de la literatura, este tipo de modelos aparece además como una de las familias más utilizadas en comparativas recientes de elección modal.

La idea fundamental consiste en entrenar un gran número de árboles de decisión sobre diferentes subconjuntos de los datos de entrenamiento. Cada árbol se construye utilizando una muestra aleatoria del conjunto de datos original (técnica conocida como *bootstrap sampling*) y, además, en cada división del árbol se considera únicamente un subconjunto aleatorio de variables explicativas. Este doble proceso de aleatorización permite reducir la correlación entre árboles individuales y, en consecuencia, mejorar el rendimiento del modelo agregado.

En problemas de clasificación, como el de elección modal, cada árbol produce una predicción independiente y la decisión final se obtiene mediante votación mayoritaria entre todos los árboles del bosque. Este mecanismo permite capturar relaciones no lineales entre variables y manejar de forma robusta conjuntos de datos con múltiples atributos.

En el ámbito del transporte, los modelos Random Forest se han utilizado en distintos trabajos para predecir la elección de modo de transporte, ya que ofrecen un buen equilibrio entre capacidad predictiva, robustez frente al sobreajuste y relativa facilidad de interpretación mediante medidas de importancia de variables. No obstante, la evidencia comparativa reciente sugiere que su rendimiento puede quedar por debajo del de otros métodos más potentes de boosting en algunos conjuntos de datos reales [José Ángel Martín-Baos, 2023].

2.5.6. Gradient Boosting Decision Trees

Los modelos basados en *Gradient Boosting Decision Trees* (GBDT) constituyen otra familia de métodos de aprendizaje automático ampliamente utilizados en problemas de clasificación con datos tabulares. A diferencia de los métodos basados en bagging, como Random Forest, el enfoque de boosting construye los árboles de decisión de forma secuencial, de modo que cada nuevo árbol intenta corregir los errores cometidos por el conjunto de árboles anteriores [Friedman, 2001].

El proceso comienza entrenando un primer árbol de decisión sencillo sobre los datos de entrenamiento. A continuación, se construyen árboles adicionales que se ajustan sobre los residuos o errores del modelo previo. De este modo, el modelo final se obtiene como la suma ponderada de múltiples árboles débiles, cada uno de los cuales contribuye a mejorar gradualmente la predicción global.

Entre las implementaciones más conocidas de este enfoque se encuentra XGBoost (*Extreme Gradient Boosting*), una biblioteca optimizada que introduce mejoras en eficiencia computacional, regularización del modelo y manejo de grandes conjuntos de datos

[Chen and Guestrin, 2016]. Gracias a estas características, XGBoost se ha convertido en una de las técnicas más utilizadas en problemas de aprendizaje supervisado con datos estructurados y ha mostrado un comportamiento especialmente competitivo en comparativas recientes de elección modal.

En el contexto de este TFM, los modelos basados en Gradient Boosting resultan especialmente relevantes, ya que permiten capturar relaciones no lineales entre los atributos del viaje y la elección modal. Además, su buen rendimiento predictivo y su flexibilidad para manejar distintos tipos de variables los convierten en una herramienta adecuada para integrarse en el simulador de movilidad desarrollado en este trabajo.

2.5.7. Redes Neuronales Profundas

Las redes neuronales profundas (*Deep Neural Networks*, DNN) constituyen otra familia de modelos de aprendizaje automático que ha experimentado un crecimiento significativo en los últimos años. Estas técnicas se basan en arquitecturas formadas por múltiples capas de neuronas artificiales que transforman progresivamente las variables de entrada mediante combinaciones lineales y funciones de activación no lineales [Goodfellow et al., 2016].

A diferencia de los modelos basados en árboles, las redes neuronales permiten aprender representaciones internas de los datos a través de varias capas ocultas, lo que facilita capturar relaciones complejas entre variables. Esta capacidad de modelar interacciones no lineales ha motivado su aplicación en numerosos problemas de predicción y clasificación, incluyendo el análisis del comportamiento de los viajeros.

En problemas de elección modal, las redes neuronales profundas pueden utilizarse para estimar la probabilidad de seleccionar cada modo de transporte a partir de atributos del desplazamiento y características sociodemográficas del usuario. Aunque estos modelos suelen requerir mayor volumen de datos y un proceso de ajuste más cuidadoso que los métodos basados en árboles, también ofrecen una elevada flexibilidad para capturar patrones complejos presentes en los datos.

Los tres enfoques descritos en esta sección (Random Forest, Gradient Boosting y redes neuronales profundas) se evalúan de forma comparativa sobre el dataset LPMC. Su entrenamiento, ajuste e integración en el simulador de movilidad se detallan en el capítulo 5.

3. Metodología

Este capítulo recoge la gestión del proyecto con un enfoque SCRUM adaptado a un TFM individual. Se busca dejar trazabilidad clara de qué se planificó, qué se ejecutó, qué bloqueos aparecieron y qué decisiones técnicas se tomaron en cada iteración.

3.1. SCRUM adaptado a un desarrollo unipersonal

SCRUM es una metodología ágil de carácter iterativo e incremental, orientada a organizar el trabajo en ciclos cortos, priorizar entregas de valor y revisar de forma continua el avance del producto [Schwaber and Sutherland, 2020]. Aunque su formulación original está pensada para equipos, sus principios también pueden adaptarse a proyectos individuales cuando se busca mantener planificación, trazabilidad y capacidad de reacción ante cambios.

En este trabajo se ha utilizado una adaptación práctica de SCRUM, no como aplicación estricta del marco en todos sus elementos formales, sino como guía para estructurar el desarrollo del simulador, registrar decisiones y ordenar la evolución del proyecto. Esta adaptación ha resultado útil por tres motivos. En primer lugar, porque se han combinado tareas heterogéneas, incluyendo desarrollo backend, frontend, integración de motores de enrutado, tratamiento de datos, entrenamiento de modelos y redacción de memoria. En segundo lugar, porque el alcance real del proyecto ha ido evolucionando a medida que se validaban soluciones técnicas y se detectaban bloqueos. Por último, porque el seguimiento periódico con el tutor ha funcionado como mecanismo natural de revisión y reorientación.

Desde el punto de vista organizativo, la Guía Scrum 2020 define un *Scrum Team* compuesto por tres responsabilidades principales: *Product Owner*, *Scrum Master* y *Developers* [Schwaber and Sutherland, 2020]. En un trabajo unipersonal esta separación no puede reproducirse de forma literal, por lo que se ha reinterpretado del siguiente modo:

-
- **Product Owner:** el tutor académico ha asumido de forma aproximada esta función, al validar prioridades, orientar el alcance funcional del prototipo y revisar la utilidad de los entregables desde el punto de vista del objetivo del TFM.
 - **Scrum Master:** esta responsabilidad ha sido asumida por el propio autor del trabajo, en la medida en que ha organizado iteraciones, registrado bloqueos, mantenido la trazabilidad del progreso y ajustado la planificación cuando han surgido incidencias técnicas o cambios de enfoque.
 - **Developers:** el desarrollo técnico también ha recaído íntegramente en el autor, responsable de la implementación del simulador, la integración de servicios, el entrenamiento de modelos y la documentación de resultados.

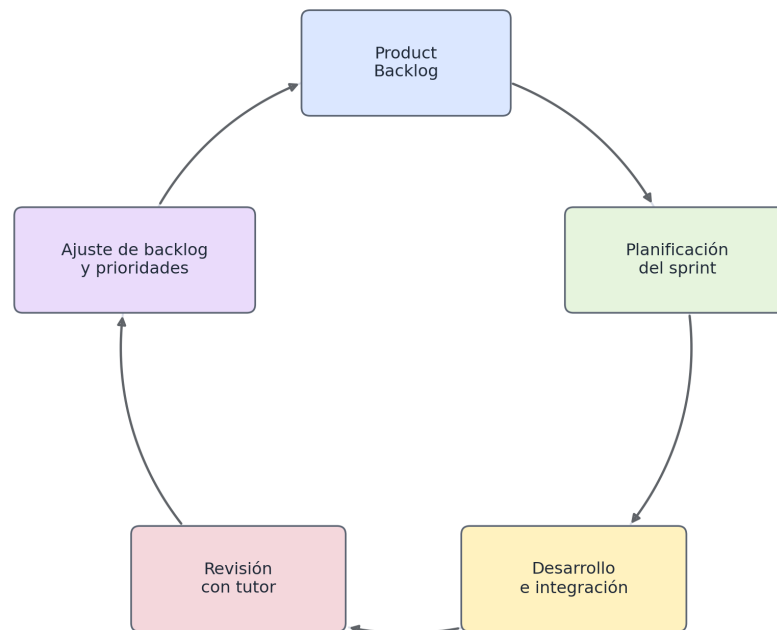
En consecuencia, varias responsabilidades propias de un equipo Scrum han quedado concentradas en una única persona, mientras que el tutor ha actuado como agente externo de supervisión, validación y priorización. Esta circunstancia introduce diferencias claras respecto a un SCRUM de equipo, pero no invalida su uso como marco ligero de gestión. En este contexto, lo más relevante no ha sido reproducir todas las ceremonias de forma estricta, sino conservar los elementos que aportan valor real al proyecto: backlog priorizado, trabajo por iteraciones, revisión periódica y registro explícito de decisiones.

La metodología seguida en este trabajo se ha apoyado en los siguientes principios:

- dividir el trabajo en sprints con objetivos acotados y entregables identificables;
- mantener un *Product Backlog* vivo, revisado conforme avanzaba el proyecto;
- registrar en cada iteración qué se planificó, qué se completó y qué problemas aparecieron;
- utilizar las reuniones con el tutor como mecanismo de revisión y re-priorización;
- orientar cada sprint a la obtención de un incremento funcional o documental del proyecto.

En cuanto al seguimiento, las reuniones con el tutor comenzaron a finales de 2025, aproximadamente entre octubre y noviembre, con una periodicidad quincenal, normalmente los martes. Una vez finalizado el primer cuatrimestre y superada la carga principal de exámenes, la cadencia pasó a ser semanal a partir de febrero y marzo de 2026, con el objetivo de acelerar el ritmo de desarrollo y acotar con mayor precisión el trabajo asociado a cada sprint. Este cambio de frecuencia reforzó el carácter iterativo del proyecto y facilitó una validación más cercana del backlog y de los entregables.

La Figura 3.1 resume visualmente este ciclo de trabajo adaptado a un desarrollo unipersonal, mostrando la relación entre backlog, planificación, ejecución y revisión periódica con el tutor.



SCRUM adaptado a desarrollo unipersonal

Figura 3.1: Esquema del ciclo de trabajo SCRUM adaptado al desarrollo unipersonal del proyecto. Elaboración propia.

Esta metodología ha encajado bien con la naturaleza del proyecto. Por una parte, ha permitido avanzar de manera incremental desde un prototipo inicial de rutas hasta un MVP funcional con integración de OSRM, OTP, GTFS y un modelo de elección modal. Por otra, ha facilitado documentar de forma ordenada tanto el trabajo ya completado como las líneas de evolución previstas, incluyendo mejoras visuales, ampliación de modelos y posibles análisis adicionales de simulación. A partir de este marco general, en los apartados siguientes se presenta el backlog del proyecto y el histórico de sprints ejecutados.

3.2. Product Backlog

3.2.1. Backlog inicial

El *Product Backlog* se definió como una lista viva de trabajo, organizada en grandes bloques funcionales y técnicos. En lugar de partir de un conjunto cerrado de tareas detalladas desde el inicio, se optó por un backlog de alto nivel que pudiera refinarse a medida que se validaban decisiones de diseño y se consolidaba el MVP del simulador.

En una primera aproximación, los elementos principales del backlog fueron los siguientes:

1. Definir una arquitectura base cliente-servidor para desacoplar frontend, backend y servicios externos.
2. Construir un prototipo web funcional con mapa interactivo, selección de origen-destino y visualización de rutas.
3. Integrar OSRM en local con perfiles diferenciados para coche, bicicleta y desplazamiento a pie.
4. Incorporar transporte público urbano mediante GTFS de Toledo y OpenTripPlanner.
5. Diseñar una API propia que unifique consultas de rutas, exploración GTFS e inferencia modal.
6. Preparar una pipeline reproducible de datos para el dataset LPMC.
7. Entrenar y validar un primer modelo de elección modal utilizable dentro del simulador.
8. Integrar el modelo de inferencia en backend y frontend de forma modular.
9. Mejorar la interfaz, la experiencia de usuario y la capacidad de análisis visual del sistema.
10. Documentar la solución, registrar la evolución del proyecto y redactar la memoria final.

Conforme avanzó el trabajo, este backlog se fue refinando en tareas más concretas. Algunos elementos inicialmente planteados como trabajo futuro pasaron a formar parte del desarrollo principal, mientras que otros quedaron aplazados para iteraciones posteriores. A fecha de redacción de esta memoria, la parte nuclear del simulador puede considerarse ya implementada, por lo que el backlog actual se concentra principalmente en consolidación documental, mejoras visuales y ampliaciones funcionales sobre una base ya operativa.

Para facilitar su seguimiento, los elementos del backlog se agruparon en cuatro bloques:

- **Bloque funcional:** interfaz web, selección de viajes, cálculo de rutas, visualización de resultados y cuadro de información.
- **Bloque de integración de servicios:** despliegue y conexión de OSRM, OTP y GTFS dentro de una arquitectura reproducible.
- **Bloque de inteligencia artificial:** preprocesado del dataset LPMC, entrenamiento de modelos, validación e integración de inferencia.
- **Bloque documental y de cierre:** trazabilidad del proyecto, redacción de memoria, mejora de la presentación y preparación de resultados.

3.2.2. Priorización de items

La priorización del backlog no se realizó únicamente por orden técnico de implementación, sino por valor aportado al objetivo general del proyecto. En consecuencia, se siguió el siguiente criterio:

- **Prioridad alta:** elementos necesarios para disponer de un simulador funcional extremo a extremo, incluyendo mapa interactivo, rutas por modo, integración de transporte público y una primera versión operativa de la inferencia modal.
- **Prioridad media:** mejoras que aumentan la usabilidad, la claridad visual o la trazabilidad del sistema, pero que no bloquean el funcionamiento básico del MVP.
- **Prioridad evolutiva:** ampliaciones previstas una vez estabilizado el núcleo del sistema, como el entrenamiento de modelos adicionales, la mejora de análisis de escenarios o la incorporación de nuevas visualizaciones y métricas.

Este criterio explica el orden seguido durante el desarrollo. Primero se priorizó la infraestructura mínima para obtener rutas y visualizar resultados reales sobre el mapa; después se incorporó el bloque de modelado e inferencia; y finalmente se abordaron tareas de consolidación, mejora y documentación. De esta forma, cada sprint pudo orientarse a producir incrementos útiles sobre una base ya validada, en lugar de avanzar en paralelo sobre demasiados frentes abiertos.

3.3. Histórico de sprints ejecutados

En este apartado se resume la secuencia principal de trabajo seguida durante el desarrollo del proyecto. Dado que el TFM no arrancó con una planificación completamente cerrada

y que parte de la trazabilidad se ha reconstruido a partir de versiones funcionales, reuniones de seguimiento y artefactos generados, los periodos y esfuerzos indicados deben interpretarse como estimaciones razonables. Aun así, este histórico permite mostrar con claridad cómo evolucionó el proyecto desde la validación inicial de tecnologías hasta la integración final del simulador.

La Figura 3.2 muestra un diagrama de Gantt simplificado de los sprints ejecutados durante el desarrollo del proyecto. El diagrama refleja tanto la secuencia principal del desarrollo como la existencia de trabajo en paralelo, especialmente entre las fases de modelado y diseño web y en la redacción de la memoria.

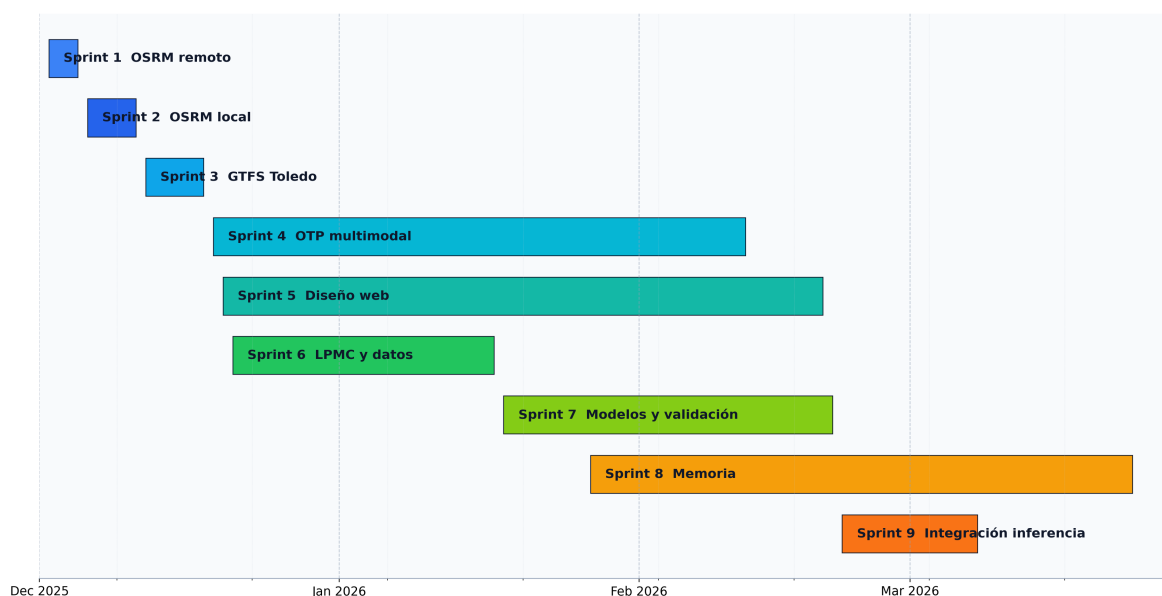


Figura 3.2: Cronograma aproximado de los sprints ejecutados durante el desarrollo del TFM, representado como diagrama de Gantt simplificado. Elaboración propia.

3.3.1. Sprint 1: Validación inicial de OSRM mediante servicios remotos

Fechas: del 2 al 5 de diciembre de 2025.

Esfuerzo estimado: 8–12 horas.

Este sprint tuvo como objetivo investigar distintas herramientas de enrutado y, en particular, validar la viabilidad de OSRM dentro del proyecto mediante llamadas remotas desde una primera arquitectura cliente-servidor. El propósito no era todavía disponer de una so-

lución definitiva, sino comprobar que el flujo completo entre mapa, backend y cálculo de rutas resultaba técnicamente factible.

El principal resultado de esta iteración fue la construcción de un primer prototipo funcional y la identificación temprana de las limitaciones de los servicios remotos utilizados para las pruebas iniciales. Esta validación permitió confirmar el interés de OSRM como base del simulador, pero también puso de manifiesto la necesidad de avanzar hacia una infraestructura propia para garantizar comparabilidad real entre modos y mayor control experimental.

3.3.2. Sprint 2: Despliegue del servicio OSRM local multi-perfil

Fechas: del 6 al 11 de diciembre de 2025.

Esfuerzo estimado: 12–16 horas.

Una vez validado el uso de OSRM para este trabajo, este sprint se orientó al despliegue de una instancia local que permitiera obtener rutas diferenciadas para distintos modos de desplazamiento y, al mismo tiempo, mantener un control total sobre la infraestructura. Este cambio era necesario para asegurar reproducibilidad, trazabilidad y estabilidad en las comparaciones realizadas por el simulador.

Como resultado, el proyecto dejó de depender de servicios públicos externos y pasó a apoyarse en una base de cálculo local sobre la que posteriormente se construirían tanto la comparación modal como la integración con el resto de componentes del sistema.

3.3.3. Sprint 3: Incorporación del GTFS urbano de Toledo

Fechas: del 12 al 18 de diciembre de 2025.

Esfuerzo estimado: 10–14 horas.

El objetivo de esta iteración fue incorporar la red de transporte público urbano de Toledo al simulador a partir del feed GTFS descargado desde el Punto de Acceso Nacional (NAP) del Ministerio de Transportes. Para ello se desarrolló el módulo de lectura `gtfs_loader.py`, que parsea y mantiene en memoria la información de paradas, rutas, viajes y horarios, y se implementaron cuatro endpoints en el router `/api/gtfs`: listado de paradas con filtro opcional por bounding box, listado de líneas de la red, detalle de ruta con paradas ordenadas y trazado geométrico, y resumen de horarios por fecha.

En la capa de visualización se añadieron las paradas como marcadores circulares en el mapa con popup interactivo, desde el que el usuario puede consultar qué líneas dan servicio en cada parada. Este sprint no incluyó planificación de itinerarios, que se abordó en el sprint siguiente. Su resultado principal fue disponer de una representación operativa completa de

la red de bus urbano de Toledo, desacoplada del grafo OTP, lo que permite consultar la red aunque el planificador no esté disponible.

3.3.4. Sprint 4: Integración de OpenTripPlanner para rutas multimodales

Fechas: del 19 de diciembre de 2025 al 12 de febrero de 2026.

Esfuerzo estimado: 16–24 horas.

Tras la incorporación del GTFS, este sprint se centró en habilitar el cálculo de rutas de transporte público puerta a puerta mediante OpenTripPlanner. El objetivo era pasar de una visualización estática de red a una capacidad real de planificación multimodal, incorporando la combinación entre red peatonal y servicio de autobús urbano.

Esta fase supuso un salto cualitativo dentro del proyecto, ya que permitió integrar en una misma aplicación los modos viarios y el modo tránsito, haciendo posible una comparación más completa entre alternativas de viaje en el contexto urbano de Toledo.

3.3.5. Sprint 5: Diseño de la interfaz web y codificación visual por modo

Fechas: del 20 de diciembre de 2025 al 20 de febrero de 2026.

Esfuerzo estimado: 8–12 horas.

Este sprint se orientó a consolidar la experiencia de usuario y la legibilidad comparada de resultados. Las principales decisiones de diseño adoptadas fueron: codificación visual de rutas por modo (azul sólido para conducción, verde sólido para bicicleta y gris discontinuo para desplazamiento a pie); visualización de segmentos OTP diferenciada entre tramos en vehículo, representados en naranja, y tramos a pie, con la misma trama discontinua que el modo peatonal; basemaps seleccionables por el usuario; y marcadores SVG personalizados para origen y destino con iconografía diferenciada.

Se consolidó también el uso de TanStack Query para las peticiones al backend, que proporciona caché de respuestas, gestión declarativa de estados de carga y reintento automático en caso de error. Los segmentos de transporte público se restringieron a modo *transit*, evitando solapamientos visuales con las rutas viarias. El resultado fue una interfaz directamente utilizable como herramienta de análisis, no solo como demostración técnica del sistema de enrutado.

3.3.6. Sprint 6: Diseño de un modelo de clasificación de modo de viaje y procesamiento de datos

Fechas: del 21 de diciembre de 2025 al 17 de enero de 2026.

Esfuerzo estimado: 12–18 horas.

Este sprint tuvo como finalidad preparar la base metodológica del módulo de inteligencia artificial del proyecto. Para ello se abordó el estudio del dataset LPMC, la revisión de trabajos de referencia y el diseño del proceso de preparación de datos necesario para entrenar modelos de clasificación de modo de viaje con un criterio reproducible.

El trabajo realizado en esta fase no se orientó todavía a la integración en el simulador, sino a establecer una base sólida de modelado sobre la que poder entrenar, comparar y validar distintos enfoques en iteraciones posteriores.

3.3.7. Sprint 7: Entrenamiento y validación de los modelos de clasificación

Fechas: del 18 de enero al 21 de febrero de 2026.

Esfuerzo estimado: 14–20 horas.

Una vez definida la base de datos y el proceso de preparación, este sprint se centró en entrenar y validar los primeros modelos de clasificación de modo de viaje. El objetivo fue disponer de una referencia cuantitativa útil, suficientemente estable como para servir de punto de partida a la integración del módulo de inferencia dentro del simulador.

Como resultado, el proyecto avanzó desde una fase exploratoria sobre datos hacia una fase claramente aplicada, en la que ya era posible valorar el rendimiento de los modelos y decidir cuáles podían incorporarse al sistema de forma operativa.

3.3.8. Sprint 8: Escritura de la memoria y consolidación metodológica

Fechas: del 27 de enero al 24 de marzo de 2026, en paralelo a varios sprints técnicos.

Esfuerzo estimado: 10–14 horas en su arranque, ampliado de forma continua durante la redacción final.

La escritura de la memoria no se desarrolló como una actividad aislada al final del proyecto, sino como una línea de trabajo paralela que fue ganando peso a medida que el simulador y el bloque de modelado alcanzaban mayor madurez. El objetivo de este sprint fue consolidar la estructura documental del TFM, fijar el enfoque metodológico y dejar trazabilidad suficiente de las decisiones adoptadas a lo largo del desarrollo.

Su carácter transversal hace que este sprint deba interpretarse como una actividad de acompañamiento al resto del proyecto. En la práctica, sirvió para ordenar el material generado, depurar la narrativa técnica y conectar la evolución funcional del sistema con la justificación académica del trabajo.

3.3.9. Sprint 9: Integración del módulo de inferencia en el simulador

Fechas: del 22 de febrero al 8 de marzo de 2026.

Esfuerzo estimado: 14–20 horas.

El objetivo de esta iteración fue integrar el módulo de inferencia modal dentro del simulador ya construido, conectando el bloque de modelado con la aplicación web y con la lógica general del backend. Con ello se buscaba que el sistema dejara de ser únicamente un comparador de rutas para incorporar también una capa predictiva sobre la elección de modo de viaje.

Esta integración representó la convergencia de las dos grandes líneas del TFM: por un lado, la construcción del simulador de movilidad urbana; por otro, el diseño y validación de modelos de clasificación. A partir de este punto, el proyecto quedó configurado como una aplicación capaz de combinar cálculo de rutas, visualización cartográfica e inferencia modal en un mismo entorno de trabajo.

3.3.10. Sprint 10: Reestructuración del repositorio y documentación de la arquitectura

Fechas: del 17 de marzo al 15 de abril de 2026.

Esfuerzo estimado: 14–20 horas.

El proyecto presentaba una deuda técnica acumulada: los distintos componentes, incluyendo backend, frontend, scripts de entrenamiento y configuración Docker, se habían desarrollado de forma incremental y necesitaban unificarse en un repositorio coherente para la entrega final. Este sprint abordó esa consolidación integrando todos los módulos en un monorepo con estructura clara de directorios y preparando la documentación técnica para la fase de escritura del capítulo de arquitectura.

La tarea documental más relevante fue la redacción del capítulo 4, que formalizó las decisiones de diseño acumuladas durante los sprints anteriores: la tabla de servicios Docker, los cuatro grupos de endpoints de la API, el diagrama del pipeline de inferencia y la descripción de la capa de orquestación. Documentar la arquitectura antes de entrar en el detalle del capítulo 5 permitió detectar algunas inconsistencias menores en los contratos de la API y corregirlas antes de que quedaran fijadas en la memoria.

3.3.11. Sprint 11: Entrenamiento de modelos adicionales y comparativa multi-modelo

Fechas: del 28 de abril al 6 de mayo de 2026.

Esfuerzo estimado: 20–28 horas.

El bloque de inteligencia artificial contaba en ese momento únicamente con el modelo XGBoost entrenado en el sprint 7. Este sprint amplió el repertorio con un segundo modelo de tipo bosque aleatorio (Random Forest) y un tercero de red neuronal profunda implementado en PyTorch mediante un módulo `TorchModalWrapper`, lo que permitió comparar tres enfoques de aprendizaje automático sobre el mismo dataset LPMC y el mismo conjunto de variables de entrada. En los tres modelos se aplicó validación cruzada con `GroupKFold`, agrupando por identificador de hogar para evitar que viajes de un mismo hogar aparecieran simultáneamente en los conjuntos de entrenamiento y de test.

Durante esta fase se detectó y corrigió un error en el pipeline de inferencia: las duraciones de ruta devueltas por OSRM y OTP estaban en segundos, mientras que el dataset LPMC las registra en horas. Sin esa conversión, las duraciones llegaban al modelo con un factor de inflación de 3.600, lo que distorsionaba completamente las predicciones. La corrección se aplicó tanto en el módulo de entrenamiento como en el de inferencia. Se añadió también el endpoint `POST /api/lpmc/compare`, que ejecuta los tres modelos en paralelo y devuelve sus probabilidades de forma comparada, junto con el panel de comparación correspondiente en el frontend.

3.3.12. Sprint 12: Redacción del capítulo de implementación y primer rediseño de la interfaz

Fechas: del 19 de mayo al 10 de junio de 2026.

Esfuerzo estimado: 28–36 horas.

Este sprint combinó dos líneas de trabajo en paralelo. Por un lado, la redacción de las seis secciones del capítulo 5, que recorren la implementación de OSRM, OTP, el módulo GTFS, el backend FastAPI, el frontend React y el bloque de modelado. La escritura en detalle de cada sección exigió revisar el estado real del código y, en varios casos, corregir implementaciones que habían quedado incompletas o cuya documentación no reflejaba su funcionamiento real.

Por otro lado, la interfaz recibió su primer rediseño estructural: se sustituyó el diseño anterior por un sidebar-rail fijo de 64 píxeles con cuatro paneles funcionales (Rutas, Red de transporte, Predicción IA y Capas) que flotan sobre el mapa sin desplazarlo. Se incorporó un

menú contextual al hacer clic derecho sobre el mapa para establecer el origen o el destino del viaje. Se activó además el Plan A de manejo de itinerarios sin transporte público real: cuando OTP no devuelve ningún tramo de bus, las variables de transporte público se sustituyen por valores de penalización extremos antes de la inferencia, de forma que el modelo descarte esa alternativa por sus propias reglas sin necesidad de intervención posterior.

3.3.13. Sprint 13: Consolidación de la interfaz y preparación del repositorio para entrega

Fechas: del 11 al 19 de junio de 2026.

Esfuerzo estimado: 22–30 horas.

Este sprint agrupó un conjunto de mejoras de interfaz y de infraestructura orientadas a dejar el sistema en el estado que se presenta en la defensa. En la parte visual, el panel de la red GTFS fue rediseñado por completo: las líneas se agrupan por nombre corto en un acordeón desplegable con un diagrama de paradas al estilo de un cartel de línea, y se garantizan colores únicos asignando una paleta de 26 tonos a las 25 líneas del feed de Toledo por índice alfabético. Se añadió un panel de Ajustes con un selector de modelo activo que carga las opciones dinámicamente desde la API sin necesidad de reiniciar el contenedor, resolviendo una dependencia que hasta entonces requería edición manual de la variable de entorno. Se sustituyeron también los controles de zoom nativos de Leaflet por componentes React con lógica de limpieza de rutas, se instaló la librería Lucide React para la iconografía de la interfaz y se añadió un panel de Inicio con la descripción del proyecto y los créditos.

En cuanto a la infraestructura, el repositorio se reorganizó para la entrega pública: se configuró Git LFS para los artefactos pesados (grafos OSRM, grafo OTP, fichero GTFS ZIP y modelos ML), se añadieron servicios de inicialización idempotentes en Docker Compose para que el sistema arranque correctamente en cualquier máquina sin pasos manuales, y se redactaron los ficheros README en inglés y español. El repositorio fue renombrado a `movilidad-urbana` en GitHub.

3.3.14. Sprint 14: Pulido final de la aplicación y revisión integral de la memoria

Fechas: del 24 al 25 de junio de 2026.

Esfuerzo estimado: 20–26 horas.

Los dos últimos días de desarrollo activo sirvieron para cerrar los flecos técnicos pendientes y dejar la memoria en estado de depósito. En la aplicación se resolvieron varios

problemas acumulados: el prefijo del feedId de OTP impedía identificar la línea de bus en el catálogo GTFS, por lo que las insignias de color del panel Rutas mostraban un tono incorrecto; la fecha y hora del viaje estaban repartidas en tres controles distintos de la interfaz que no siempre estaban sincronizados; y el grafo de OTP había sido construido con el feed de diciembre de 2025, mientras que el panel de la red usaba el feed de 2026, lo que generaba una inconsistencia estructural: consultar una fecha de 2026 en el planificador devolvía únicamente itinerarios a pie mientras el panel de red mostraba horarios correctamente. Se reconstruyó el grafo con el feed de 2026 (22 de febrero a 22 de mayo), se unificó el control de fecha y hora en un único elemento permanente sobre el mapa y se corrigió un error de zona horaria en los horarios mostrados. Se añadieron también la edición directa de coordenadas de origen y destino, la copia de coordenadas al portapapeles y el recálculo automático de rutas al modificar un extremo del viaje.

En la memoria se revisaron exhaustivamente los capítulos 4 y 5 contrastándolos con el estado final de la aplicación, se crearon los cuatro anexos (manual de despliegue, contratos de la API, configuración experimental y reproducibilidad, y evidencias de sprints) y se corrigieron cuatro referencias al anexo de despliegue que estaban rotas por la ausencia de su etiqueta `\label`. Los puertos de los servicios OSRM en el host se cambiaron de 5000/5001/5002 a 5001/5002/5003 para evitar el conflicto con el servicio AirPlay/Control Center en macOS.

3.4. Retrospectiva del ciclo de desarrollo

3.4.1. Aprendizajes por iteración

El desarrollo del proyecto dejó cinco aprendizajes técnicos que merecen recogerse explícitamente, tanto por su impacto real sobre el trabajo como por su aplicabilidad en proyectos similares:

- **Validar los servicios externos en una iteración temprana y aislada evita retrabajo costoso.** El descubrimiento en el sprint 1 de que el servidor de demostración de OSRM solo ofrece el perfil de conducción motivó la transición inmediata a una infraestructura local, que fue la base de todo el desarrollo posterior. Posponer esa validación al sprint 5 o posterior habría obligado a replantear la arquitectura cuando ya existía código construido sobre ella.
- **La coherencia entre las unidades de los datos de entrada y las del dataset de entrenamiento es un invariante crítico en cualquier pipeline de inferencia.** El factor de

conversión de segundos a horas entre las salidas de OSRM/OTP y el dataset LPMC pasó desapercibido durante varios sprints y produjo predicciones completamente incorrectas hasta su detección y corrección en el sprint 11. Documentar las unidades esperadas por el modelo junto a la definición de cada variable habría evitado el error desde el principio.

- **Cuando dos componentes dependen del mismo conjunto de datos con mecanismos de carga distintos, es necesario mantener un único punto de referencia temporal.** La coexistencia del feed GTFS de diciembre de 2025 en el grafo de OTP con el feed de 2026 en el módulo de consulta estática creó una incoherencia que solo se detectó al intentar visualizar horarios para las mismas fechas en ambos componentes. La lección es verificar la consistencia temporal de los datos entre servicios antes de integrarlos en la interfaz de usuario.
- **Las técnicas de evaluación de modelos de clasificación pueden introducir data leakage si no se tiene en cuenta la estructura jerárquica de los datos.** En el dataset LPMC varios miembros de un mismo hogar realizaron viajes distintos; sin el uso de GroupK-Fold agrupado por identificador de hogar, viajes del mismo hogar podían aparecer simultáneamente en los conjuntos de entrenamiento y de test, inflando artificialmente las métricas de evaluación. El split temporal por survey year era necesario pero insuficiente por sí solo.
- **Registrar las decisiones de diseño en la memoria en paralelo al desarrollo acelera la redacción final y reduce la pérdida de contexto técnico.** Las secciones del capítulo 5 que más tiempo requirieron fueron precisamente aquellas para las que no se había dejado trazabilidad escrita durante los sprints correspondientes. Las secciones redactadas inmediatamente después de completar su implementación resultaron más precisas y requirieron menos revisión.

3.4.2. Acciones de mejora aplicadas

En respuesta a los aprendizajes anteriores, se tomaron las siguientes acciones correctoras:

- Se migró a OSRM local en el sprint 2, inmediatamente después de identificar las limitaciones del servidor público, sin esperar a que el problema bloqueara fases más avanzadas del desarrollo.
- Se añadió la conversión explícita de segundos a horas en el módulo de inferencia, se

documentó el invariante en el código y se verificó de forma explícita que los scripts de entrenamiento y el servicio de predicción aplicaran el mismo factor de escala.

- El grafo de OTP se reconstruyó con el feed de 2026 y se eliminó el feed antiguo del repositorio, garantizando que ambos componentes operaran sobre la misma ventana temporal.
- Se incorporó GroupKFold con el identificador de hogar como criterio de agrupación en el entrenamiento de los tres modelos, y se mantuvo el split temporal por survey year como criterio de partición del conjunto de evaluación.
- Se estableció la práctica de actualizar la memoria al cierre de cada sprint con las decisiones técnicas más relevantes, reduciendo la carga documental en la fase final y mejorando la precisión de las secciones de implementación.

3.5. Cierre del ciclo de desarrollo

La secuencia de catorce sprints descrita en este capítulo llevó el proyecto desde una primera validación de tecnologías de enrutado hasta un simulador funcional completo, con integración de rutas viarias, transporte público multimodal, red GTFS estática, inferencia de elección modal mediante tres modelos de aprendizaje automático y una interfaz de usuario lista para su presentación.

El desarrollo puede dividirse en tres fases diferenciadas. Una primera fase de infraestructura y validación tecnológica (sprints 1 a 5) orientada a demostrar la viabilidad del sistema y establecer la arquitectura de integración de servicios. Una segunda fase de desarrollo del bloque de inteligencia artificial (sprints 6, 7, 9 y 11), centrada en el procesamiento del dataset LPMC, el entrenamiento de tres modelos de clasificación y su integración en la API del sistema. Una tercera fase de consolidación, pulido visual y documentación (sprints 8, 10, 12, 13 y 14), en la que el simulador alcanzó su forma definitiva y la memoria fue redactada y revisada. Las fases primera y segunda se solapan temporalmente en varios meses, reflejo del carácter iterativo del desarrollo y de la necesidad de avanzar en paralelo sobre la infraestructura y el bloque de modelado.

La arquitectura resultante y las decisiones técnicas adoptadas a lo largo de este proceso se describen en el capítulo 4. Los detalles de implementación de cada componente se recogen en el capítulo 5.

4. Arquitectura del sistema

Este capítulo describe la organización estructural del simulador: los componentes que lo forman, cómo se relacionan entre sí y las decisiones de diseño que condicionan esa organización. El detalle de implementación de cada componente se desarrolla en el Capítulo 5.

El sistema está compuesto por cinco bloques funcionales: una interfaz web, un backend de orquestación, tres instancias del motor de enrutado viario OSRM, un planificador de transporte público basado en OpenTripPlanner y un módulo de inferencia de elección modal. Todos ellos se despliegan como contenedores Docker orquestados mediante Docker Compose.

4.1. Visión general

La Figura 4.1 muestra la arquitectura de alto nivel del sistema. El principio de diseño central es que el frontend nunca se comunica directamente con ningún servicio externo: toda petición pasa por el backend FastAPI, que actúa como única puerta de entrada y punto de orquestación. Esta decisión simplifica el control de versiones de la API, centraliza el manejo de errores y evita exponer los puertos internos de OSRM y OTP al navegador.

El frontend expone la interfaz cartográfica al usuario y delega en el backend el cálculo de rutas, la consulta de horarios y la inferencia modal. El backend integra cuatro routers diferenciados: `/api/osrm` para rutas viarias por perfil de transporte, `/api/otp` para itinerarios de transporte público, `/api/gtfs` para información estática de la red de autobús, y `/api/lpmc` para la inferencia de elección modal. Cada router encapsula la lógica de comunicación con su servicio subyacente y expone al frontend una interfaz homogénea, independiente de los detalles de cada protocolo.

Arquitectura del Simulador de Movilidad Urbana

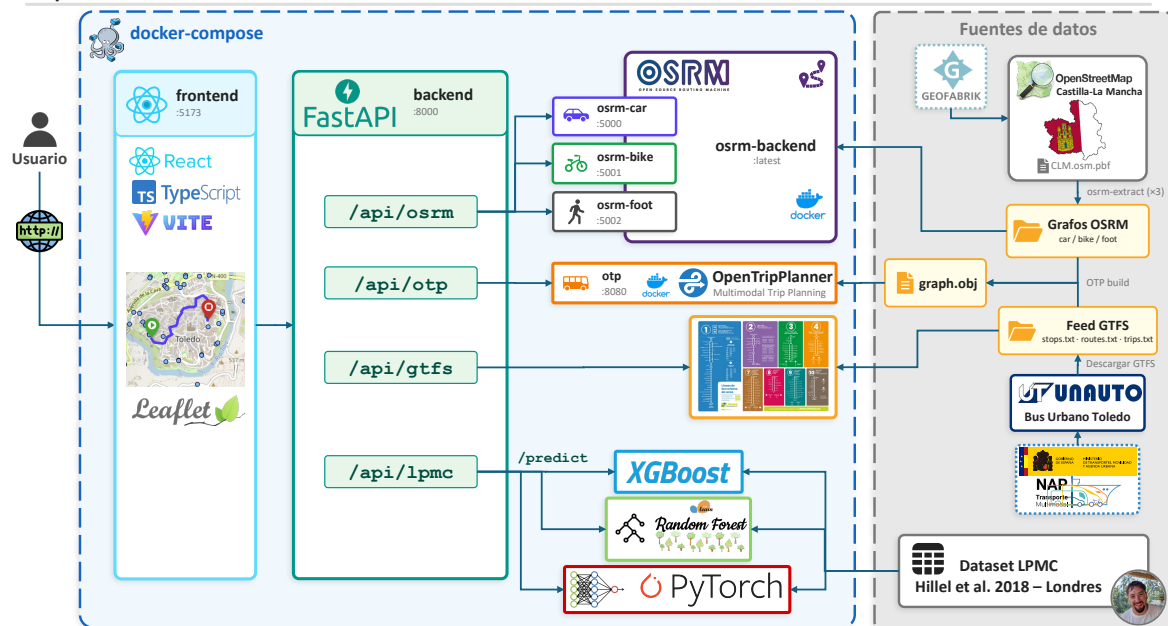


Figura 4.1: Arquitectura general del simulador de movilidad urbana.

4.2. Infraestructura, orquestación y portabilidad

La infraestructura del sistema se define íntegramente en un fichero `docker-compose.yml` en la raíz del repositorio. El despliegue completo se obtiene con un único comando (`docker compose up --build`), que elimina dependencias de entorno en la máquina anfitriona y hace reproducible el sistema entre equipos de desarrollo y de producción. Esta sección reúne todo lo relativo a la infraestructura (versionado de artefactos, despliegue automatizado y portabilidad) para no dispersarlo por el resto de la memoria; el Capítulo 5 da por explicado aquí este funcionamiento.

4.2.1

Versionado de artefactos pesados con Git LFS

Varios servicios requieren artefactos pesados generados a partir de los datos de origen: los grafos viarios de OSRM, el grafo multimodal de OpenTripPlanner y los modelos de aprendizaje automático entrenados. Un repositorio Git convencional está pensado para ficheros de texto y penaliza los binarios grandes, ya que guarda una copia completa de cada versión en el historial. Para evitarlo, estos artefactos se versionan con *Git LFS* (*Large File Storage*), una extensión de Git que almacena los ficheros de gran tamaño en un servidor aparte y deja en el historial únicamente un puntero de texto ligero que los referencia. Así, un clon del

Limite
→ Git
Normal

Cómo instalar LFS
Fuente, comando

repositorio recupera los artefactos ya contruidos sin necesidad de regenerarlos y sin inflar el historial. La Tabla 4.1 recoge los artefactos versionados con LFS, su tamaño y su papel en el sistema.

Tabla 4.1: Artefactos versionados mediante Git LFS.

Artefacto	Tamaño	Papel en el sistema
clm.osm.pbf	~97 MB	Extracto OSM de Castilla-La Mancha; red viaria para OSRM y OTP.
graph.obj	~117 MB	Grafo multimodal de OpenTripPlanner.
GTFS_Urbano_Toledo_2026.zip	~14 MB	Feed GTFS del autobús urbano de Toledo.
xgb_lpmc.joblib	~16 MB	Modelo XGBoost entrenado.
rf_lpmc.joblib	~398 MB	Modelo Random Forest entrenado.
dnn_lpmc.pt	~0,2 MB	Pesos de la red neuronal profunda.
Total	~642 MB	

El artefacto más pesado es, con diferencia, el modelo Random Forest (~398 MB), lo que llevó a valorar si excluirlo del repositorio y exigir su reentrenamiento tras el clonado. Se optó por incluirlo tras un análisis de coste sencillo. El plan gratuito de GitHub ofrece 10 GiB de almacenamiento y 10 GiB mensuales de transferencia para Git LFS [GitHub, 2024]. El repositorio ocupa unos 0,64 GiB de artefactos LFS, holgadamente por debajo del límite de almacenamiento. En cuanto a la transferencia, cada descarga completa consume esos 0,64 GiB; para el número de evaluadores previsto (del orden de ocho), el consumo mensual rondaría los 5 GiB, aún dentro del margen gratuito. Incluir el modelo, por tanto, no supone coste alguno y evita que quien clone el repositorio tenga que reentrenarlo. Si en el futuro el número de descargas creciera hasta comprometer la cuota, la alternativa natural sería distribuir los modelos como *release assets*, que no computan en la cuota de LFS.

4.2.2

Despliegue automatizado

El objetivo de diseño es que el sistema arranque de principio a fin con un único comando, sin pasos manuales. Para ello, además de los servicios de explotación, la orquestación define tres servicios de inicialización que se ejecutan una sola vez antes de los principales. `gtfs-init` descomprime el feed GTFS, `osrm-setup` compila los grafos viarios de OSRM por perfil y `otp-build` construye el grafo de OpenTripPlanner. Los tres son idempotentes:

Puse un límite de 5 EUR

comprueban si su resultado ya existe y, en tal caso, terminan de inmediato sin rehacer el trabajo.

Esta automatización actúa además como mecanismo de reserva (*fallback*) del versionado con LFS. En un clon íntegro, los artefactos ya están presentes gracias a LFS y los servicios de inicialización no hacen nada. Pero si se clona sin LFS, o si se actualiza alguno de los datos de origen (por ejemplo, un feed GTFS más reciente), esos mismos servicios regeneran solo lo que falte. De este modo el sistema es reproducible tanto desde los artefactos versionados como desde cero, siempre con el mismo comando. Los pasos manuales equivalentes, útiles para regenerar un artefacto de forma puntual, se recogen en el Anexo A.1.

4.2.3

Servicios y contenedores

La Tabla 4.2 resume los servicios definidos, sus puertos expuestos y su función en el sistema. La Figura 4.2 ilustra la topología de servicios, sus dependencias y los volúmenes compartidos.

Tabla 4.2: Servicios Docker Compose del simulador.

Servicio	Puerto	Función
frontend	5173	Interfaz web React servida por Vite.
backend	8000	API FastAPI, orquestación y lógica de negocio.
osrm-car	5001	Motor OSRM con perfil de vehículo a motor.
osrm-bike	5002	Motor OSRM con perfil de bicicleta.
osrm-foot	5003	Motor OSRM con perfil peatonal.
otp	8080	OpenTripPlanner 2.x con grafo multimodal de Toledo.
gtfs-init	—	Descomprime el feed GTFS del archivo ZIP.
osrm-setup	—	Compila los grafos viarios OSRM por perfil si no están presentes.
otp-build	—	Construye el grafo multimodal de OTP si no está presente.

La razón por la que OSRM se instancia tres veces en lugar de usar un único servidor multiperfil es que la imagen oficial de OSRM no admite perfiles simultáneos: cada proceso solo puede servir el grafo con el que fue preprocesado. Esta restricción motivó el cambio

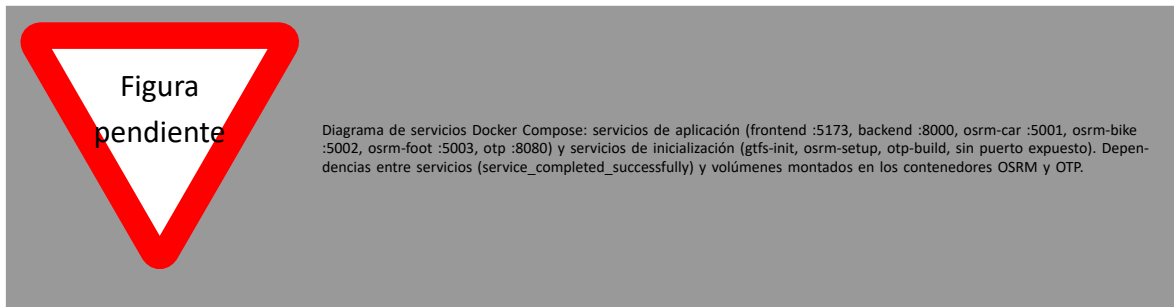


Figura 4.2: Servicios Docker Compose del simulador: dependencias y puertos expuestos.

desde la solución inicial basada en el servidor de demostración público, que únicamente ofrecía el perfil de conducción, a la infraestructura local multiperfil descrita aquí.

4.2.4 Portabilidad

El sistema está pensado para ejecutarse sin cambios en cualquier máquina, con independencia del sistema operativo. Dos decisiones lo hacen posible: la resolución de rutas y la elección de puertos.

En cuanto a las rutas, ningún módulo codifica rutas absolutas del disco, que dependerían del equipo concreto. Cada módulo deriva las rutas que necesita a partir de su propia ubicación mediante la biblioteca estándar `pathlib`, que abstrae el separador de directorios de cada sistema (la barra invertida de Windows frente a la barra de Linux). Por ejemplo, el módulo de inferencia asciende desde su propio fichero hasta la raíz del repositorio y, desde ahí, compone la ruta al directorio de modelos; el mismo código funciona en el entorno de desarrollo (Windows) y en el contenedor de ejecución (Linux), sin puntos de montaje fijos.

En cuanto a los puertos, cada contenedor OSRM escucha internamente en el 5000, y el backend se comunica con ellos por nombre de servicio dentro de la red interna de Docker. Hacia el anfitrión, en cambio, cada instancia se publica en un puerto distinto (5001, 5002 y 5003 para conducción, bicicleta y a pie, respectivamente) para poder verificarlas por separado. Se evita publicar en el 5000 del anfitrión porque es un puerto reutilizado por servicios habituales, entre ellos el receptor AirPlay de macOS, lo que provocaría conflictos en algunos equipos. Esta publicación de puertos es solo una comodidad de depuración: en funcionamiento normal, el backend es el único que habla con OSRM y OTP, siempre por la red interna.

4.3. Backend: API y routers

El backend está implementado con FastAPI sobre Python y se organiza en cuatro routers, cada uno responsable de un dominio funcional. La separación en routers permite desarrollar, probar y documentar cada integración de forma independiente. La Tabla 4.3 recoge los endpoints principales expuestos por cada router.

Tabla 4.3: Endpoints principales de la API REST del backend.

Método	Ruta	Descripción
POST	/api/osrm/routes	Rutas viarias por perfil de transporte (OSRM).
POST	/api/otp/routes	Itinerarios de transporte público con desglose por tramos (OTP).
GET	/api/gtfs/stops	Listado de paradas con filtro por caja delimitadora.
GET	/api/gtfs/routes	Listado de líneas de la red de transporte.
GET	/api/gtfs/routes/{id}	Detalle de línea: paradas ordenadas y trazado.
GET	/api/gtfs/routes/{id}/schedule	Horarios de una línea por fecha.
POST	/api/lpmc/predict	Inferencia de elección modal con el modelo activo.
POST	/api/lpmc/compare	Inferencia con los tres modelos sobre el mismo escenario.
POST	/api/lpmc/debug-features	Vector de características antes y después del escalado.
GET	/api/lpmc/models	Modelos disponibles y modelo predefinido.
GET	/health	Comprobación de disponibilidad.

El router `/api/osrm` recibe un par origen-destino con la lista de perfiles solicitados, consulta en paralelo las instancias OSRM correspondientes mediante `asyncio.gather` y devuelve al frontend las rutas con distancia, duración y geometría decodificada para su representación cartográfica.

El router `/api/otp` consulta OpenTripPlanner para obtener itinerarios de transporte público y gestiona la paginación entre las alternativas devueltas, hasta cinco por consulta. La respuesta incluye el desglose por tramos, con tipo de modo de transporte, geometría, paradas de inicio y fin, y agencia operadora cuando el tramo es en autobús. Los itinerarios se

ordenan por duración; se selecciona automáticamente el primero que incluya al menos un tramo en autobús y, si ninguno lo incluye, el de menor duración total.

El router `/api/gtfs` expone la información estática de la red de autobús a partir del feed GTFS cargado en memoria: listado de paradas con sus líneas asociadas, listado de líneas con sus sentidos, detalle de una línea con sus paradas ordenadas y trazado, y horarios por fecha. Estos datos son independientes de OpenTripPlanner, de modo que la red puede consultarse aunque el planificador no esté disponible.

El router `/api/lpmc` recibe las coordenadas de origen y destino junto con el perfil sociodemográfico del viajero, construye el vector de características consultando OSRM y OTP internamente, aplica el escalado y ejecuta la inferencia con el modelo activo. El endpoint `/compare` ejecuta los tres modelos de forma secuencial sobre el mismo escenario y devuelve sus probabilidades para cada modo de transporte; la ejecución secuencial evita comprometer el rendimiento de la CPU compartida entre contenedores.

4.4. Frontend: interfaz cartográfica

El frontend está desarrollado con React 18, Vite y TypeScript. React 18 es la biblioteca de construcción de interfaces de usuario; Vite actúa como compilador y servidor de desarrollo con recarga automática del módulo; TypeScript añade comprobación estática de tipos sobre JavaScript. La representación cartográfica se realiza con React-Leaflet, que integra la biblioteca de mapas Leaflet en el modelo de componentes de React. Los iconos de la interfaz proceden de Lucide React, una biblioteca de iconos SVG de código abierto. Las peticiones asíncronas al backend se gestionan con TanStack Query, que aporta caché de respuestas, gestión de estados de carga y reintento automático.

La aplicación se estructura en dos componentes principales. App concentra el estado global y la lógica de negocio del cliente: gestiona los puntos de origen y destino, los modos de transporte seleccionados, los resultados de rutas, la línea activa en la red GTFS y el perfil del viajero para la inferencia. MapView recibe el estado relevante mediante *props* —datos que el componente padre pasa al componente hijo de forma unidireccional— y se encarga de renderizar los marcadores, los trazados de ruta y las paradas de autobús sobre el mapa.

La interfaz cartográfica está complementada por una barra lateral de navegación (*rail*) de ancho fijo que da acceso a los paneles funcionales del simulador: Inicio, Rutas, Red GTFS, Predicción IA, Capas y Ajustes. La capa de fondo es seleccionable entre seis opciones; el detalle de cada una se describe en la Sección 5.4.7.

4.5. Pipeline de inferencia modal

La inferencia de elección modal requiere coordinar cuatro peticiones asíncronas para construir el vector de entrada al modelo. La Figura 4.3 ilustra el flujo completo.

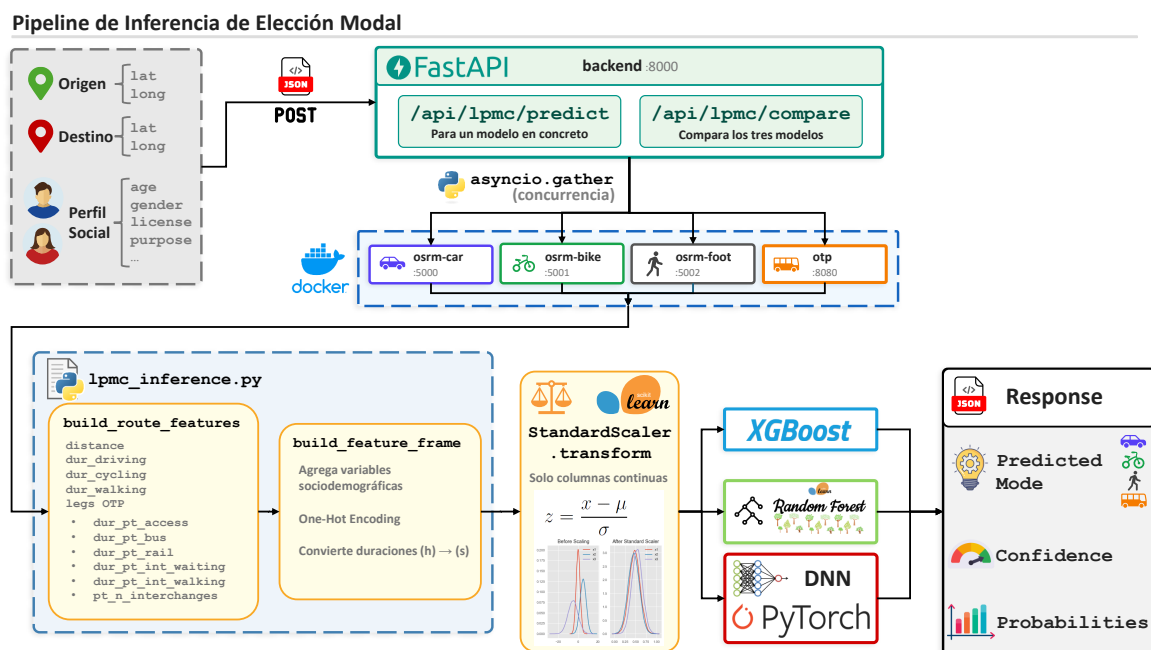


Figura 4.3: Pipeline de inferencia de elección modal.

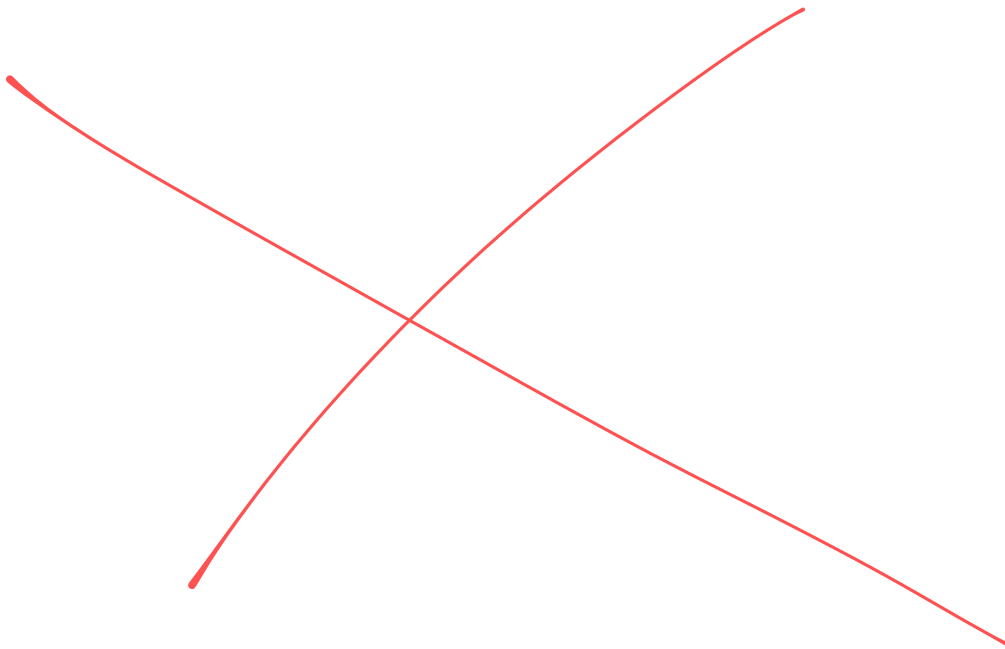
Al recibir una petición **POST /api/lpmc/predict**, el backend lanza en paralelo cuatro consultas asíncronas mediante **asyncio.gather**: una a cada una de las tres instancias OSRM (conducción, bicicleta y a pie) y una a OpenTripPlanner. Con los resultados se construye el vector de características de ruta y se añaden las variables sociodemográficas recibidas en la petición, componiendo el vector completo de entrada al modelo.

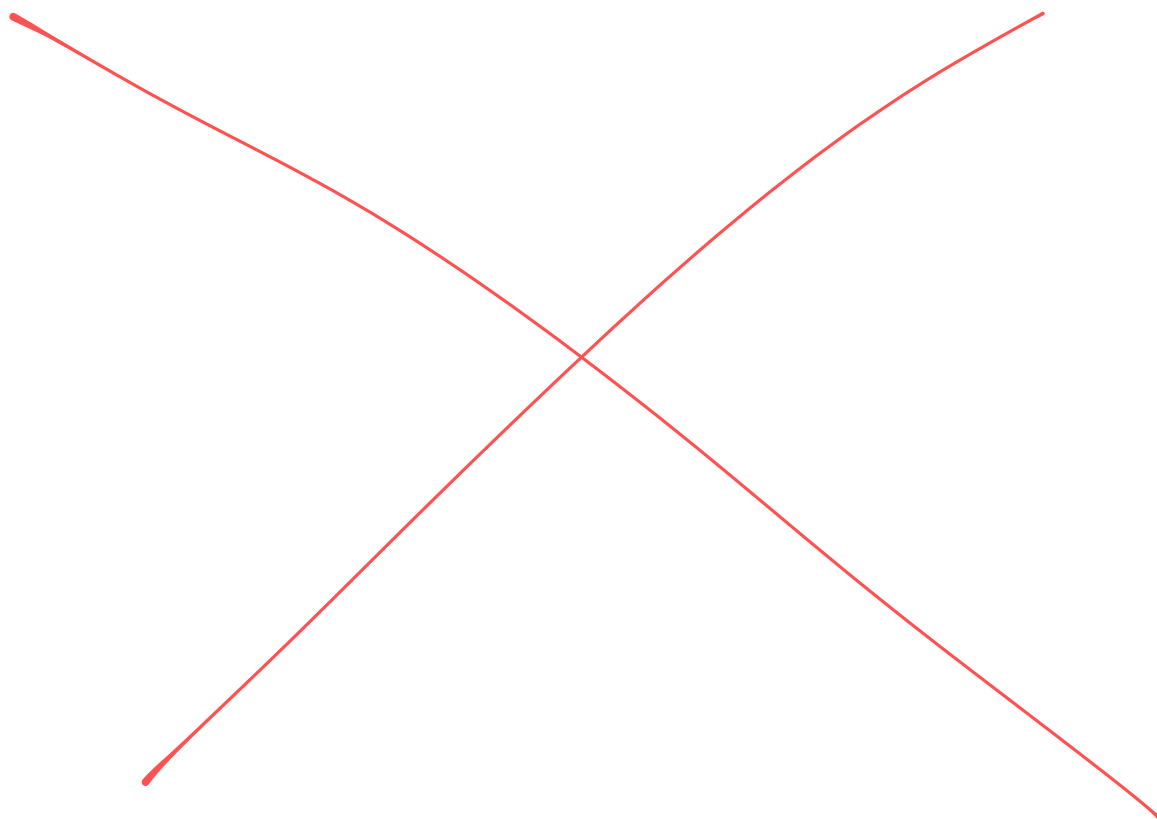
Las variables de entrada proceden de tres fuentes: los tiempos y distancias por perfil de transporte calculados por OSRM, el desglose temporal del itinerario de transporte público proporcionado por OTP, y los datos sociodemográficos y contextuales que el usuario introduce en el panel Predicción IA. El Capítulo 5 detalla la construcción del vector de características y las transformaciones aplicadas.

Las variables continuas se normalizan con un escalado estándar aplicado a las columnas de ruta y contextuales, preservando las variables binarias y las columnas codificadas sin transformar. El modelo devuelve un vector de cuatro probabilidades, una por modo de

transporte: a pie (*walk*), bicicleta (*cycle*), transporte público (*pt*) y conducción (*drive*). El modo predicho es el de mayor probabilidad.

El modelo activo se selecciona mediante el parámetro opcional `model_variant` de la petición; en su ausencia, se recurre a la variable de entorno `LPMC_MODEL_VARIANT` (xgb por defecto). Este mecanismo permite cambiar el modelo de inferencia y facilita la incorporación de modelos entrenados externamente, sin modificar el código del backend.





5. Implementación y resultados

Este capítulo detalla el proceso de construcción del simulador: las decisiones técnicas concretas tomadas en cada componente, los problemas que surgieron durante el desarrollo y la solución adoptada en cada caso, y los resultados del entrenamiento y evaluación de los modelos de elección modal.

El sistema implementado está disponible como código abierto en <https://github.com/ivanuclm/movilidad-urbana>, y permite su reproducción, extensión y reutilización en futuros trabajos de investigación o divulgación sobre movilidad urbana. El procedimiento de despliegue para reproducir el entorno local se detalla en el Anexo A.1.

5.1. Infraestructura de enrutado viario: OSRM

5.1.1. Origen y características del extracto OSM

Open Source Routing Machine (OSRM) trabaja sobre un extracto de OpenStreetMap (OSM), un fichero binario con extensión `.osm.pbf` que contiene la geometría de calles, carreteras y caminos de un territorio junto con sus atributos de velocidad, sentido y accesibilidad por modo de transporte. Geofabrik [Geofabrik GmbH, 2026] distribuye estos extractos por jerarquía geográfica, desde continentes hasta comunidades autónomas.

Se utilizó el extracto de Castilla-La Mancha al ser la región en la que se enmarca el proyecto, con Toledo como caso de estudio, tal como se detalla en la Sección 5.2. OpenTripPlanner reutiliza el mismo extracto para la red viaria peatonal, de modo que un único fichero sirve a dos servicios del sistema. La Tabla 5.1 resume los extractos de OSM evaluados durante el desarrollo.

Tabla 5.1: Comparativa de extractos OSM disponibles en Geofabrik.

Extracto	Tamaño (.osm.pbf)	Uso en el proyecto
Castilla-La Mancha	~98 MB	Producción (OSRM y OTP)
Comunidad de Madrid	~79 MB	Pruebas con feed EMT Madrid
España	~1,3 GB	Evaluated, descartado por tamaño
Europa	~32,0 GB	Descartado por tamaño

5.1.2. Evolución del despliegue

El despliegue de OSRM pasó por tres iteraciones antes de alcanzar la configuración definitiva.

Fase 1: servidor de demostración oficial. La primera implementación utilizó el servidor de demostración público del proyecto OSRM (router.project-osrm.org) [Project OSRM, 2026a]. Este servidor ofrece acceso gratuito a la API de enrutado sin necesidad de infraestructura local y está pensado para pruebas rápidas de integración. El problema es que solo procesa el perfil de conducción independientemente del indicado en la URL [danpat, 2017]; cambiar el perfil en la petición no tiene efecto, de modo que la comparación multiperfil que el simulador requiere resultaba inviable.

Fase 2: instancias públicas FOSSGIS. Las instancias públicas mantenidas por FOSSGIS [FOSSGIS e.V., 2026] sí devuelven rutas diferenciadas por perfil mediante endpoints independientes (`/routed-car`, `/routed-bike`, `/routed-foot`), lo que permitió confirmar que el esquema de tres perfiles independientes funcionaba y ajustar la integración del backend. El inconveniente es que aplican limitaciones de uso (*rate limiting*), presentan latencia variable y no permiten fijar la versión de los datos, comprometiendo así la reproducibilidad.

Fase 3: instancia local con Docker. La solución definitiva fue desplegar OSRM localmente mediante la imagen oficial `osrm/osrm-backend` [Project OSRM, 2026b], con un contenedor independiente por modo de transporte (conducción, bicicleta y a pie). OSRM solo admite un perfil por proceso: cada instancia se preprocesa con un fichero de reglas distinto y sirve únicamente ese modo, de ahí la necesidad de tres contenedores independientes. El transporte público en autobús es responsabilidad de OpenTripPlanner. Esta arquitectura elimina la dependencia de servicios externos en tiempo de ejecución, asegura reproducibilidad y suprime los límites de cuota. El preprocesado de los grafos, que originalmente se ejecutaba a mano, se automatizó como un paso de inicialización de la orquestación Docker Compose (Sección 4.2), de modo que el despliegue completo no requiere ningún comando manual

adicional.

5.1.3. Pipeline de preprocesado por perfil

Para que OSRM pueda calcular rutas, el extracto `.osm.pbf` debe transformarse en una estructura de datos interna optimizada para ejecutar consultas y obtener el camino mínimo entre dos puntos del mapa. Este preprocesado se ejecuta una sola vez por perfil y solo hay que repetirlo si se actualiza el extracto de OSM.

Cada perfil define las reglas de movilidad del modo correspondiente, esto es, qué tipos de vía son accesibles, a qué velocidad y con qué restricciones de giro. Los perfiles se escriben en Lua, el lenguaje de script utilizado por OSRM para describir las reglas de cada modo de transporte. Los tres ficheros de perfil estándar (`car.lua`, `bicycle.lua` y `foot.lua`) están incluidos en la imagen oficial de Docker `osrm/osrm-backend` en `/opt/`, por lo que no es necesario descargarlos ni configurarlos externamente. Todas las etapas del preprocesado se ejecutan con esa misma imagen, asegurando la reproducibilidad en cualquier entorno de ejecución.

1. **Extracción** (`osrm-extract`): lee el `.osm.pbf` y construye el grafo viario interno aplicando las reglas del perfil Lua indicado: qué vías son transitables, a qué velocidad y con qué restricciones. El resultado es el fichero `.osrm` junto con varios ficheros auxiliares de índice.
2. **Particionado** (`osrm-partition`): divide el grafo en celdas jerárquicas para el algoritmo Multi-Level Dijkstra (MLD) [Geisberger et al., 2008]. Esta partición es la que permite calcular rutas de larga distancia en milisegundos, al limitar la búsqueda a fronteras entre celdas en lugar de explorar el grafo completo.
3. **Personalización** (`osrm-customize`): asigna los pesos finales de cada arista sobre la estructura de celdas anterior, produciendo el grafo listo para servir peticiones de ruta.

Las tres etapas generan los ficheros `clm.osrm` y sus auxiliares, almacenados en directorios separados por perfil (`osrm-clm/car/`, `osrm-clm/bike/`, `osrm-clm/foot/`). Por su tamaño, estos artefactos se versionan mediante *Git LFS* (véase la Sección 4.2), de modo que un clon del repositorio dispone de los grafos sin necesidad de regenerarlos.

El preprocesado completo de los tres perfiles sobre el extracto de Castilla-La Mancha tarda aproximadamente 97 segundos en la máquina de desarrollo. Los artefactos generados ocupan en total unos 2,1 GB: el perfil de conducción produce 291 MB porque solo incorpora la red viaria accesible al tráfico rodado, mientras que los perfiles de bicicleta y peatonal

Revisar tiempos

alcanzan los 911 MB cada uno al incluir además senderos, caminos rurales y zonas de acceso restringido al tráfico.

Pipeline de preprocesado OSRM por perfil de transporte

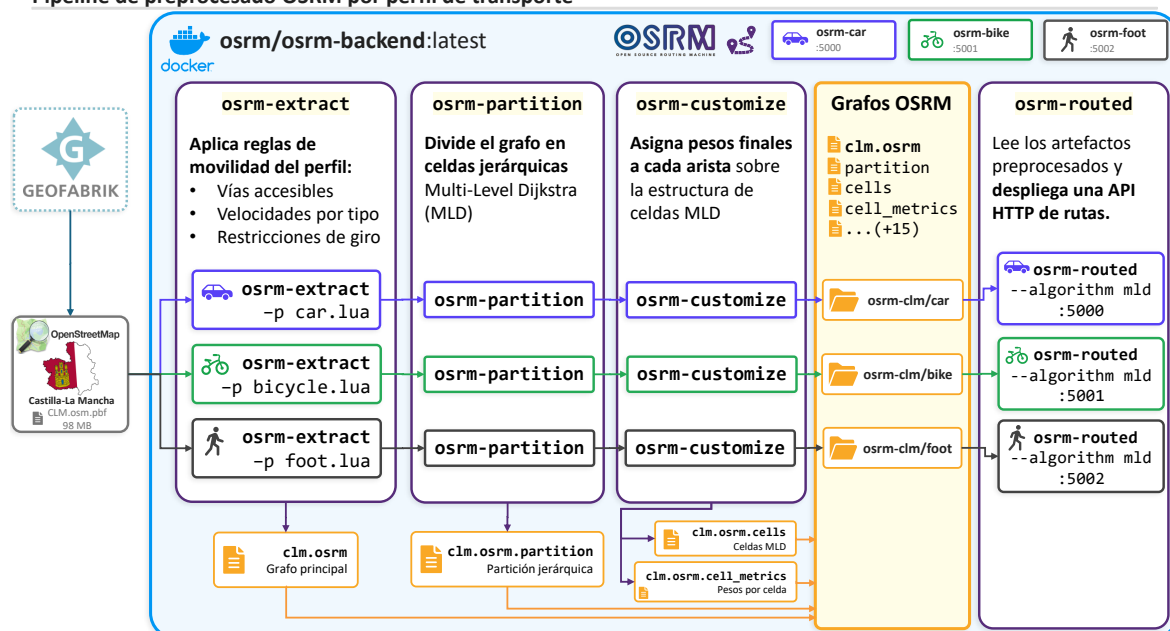


Figura 5.1: Pipeline de preprocesado OSRM por perfil de transporte.

5.1.4. Despliegue y verificación

Cada uno de los tres contenedores OSRM ejecuta el servidor de enrutado sobre su grafo preprocesado con el algoritmo MLD (Figura 5.1):

```
osrm-routed --algorithm mld /data/clm.osrm
```

Los tres contenedores comparten imagen y comando de arranque. Cada uno monta el directorio de artefactos de su perfil (`osrm-clm/car`, `osrm-clm/bike`, `osrm-clm/foot`). Dentro de la red interna de Docker Compose, el proceso `osrm-routed` escucha siempre en el puerto 5000 en todos los contenedores, y el backend los referencia por nombre de servicio (`osrm-car`, `osrm-bike`, `osrm-foot`) en ese mismo puerto. Para verificar las instancias directamente desde el terminal del anfitrión, cada contenedor expone un puerto distinto (5001, 5002 y 5003, respectivamente para los perfiles de conducción, bicicleta y a pie); la elección de esos puertos y el motivo de evitar el 5000 se explican en la Sección 4.2. Esta publicación de puertos se usa solo para depuración, ya que el backend se comunica con OSRM

por la red interna.

Una respuesta válida devuelve "code": "Ok" junto con la distancia en metros, la duración en segundos y la geometría en formato de polilínea codificada [Google LLC,]: una cadena de texto que comprime la secuencia de coordenadas de la ruta mediante diferencias relativas y codificación Base64, que el backend decodifica antes de enviar las coordenadas al frontend. Si el grafo está incompleto o el par origen-destino cae parcialmente fuera del extracto de Castilla-La Mancha, OSRM devuelve "code": "NoRoute" o una ruta incompleta, detectable al comparar visualmente los tres trazados en la interfaz.

```
# osrm-car (conduccion, localhost:5001)
curl "http://localhost:5001/route/v1/driving
    /-4.0356,39.8750;-4.0023,39.8602?overview=full"
# osrm-bike (bicicleta, localhost:5002)
curl "http://localhost:5002/route/v1/cycling
    /-4.0356,39.8750;-4.0023,39.8602?overview=full"
# osrm-foot (peatonal, localhost:5003)
curl "http://localhost:5003/route/v1/foot
    /-4.0356,39.8750;-4.0023,39.8602?overview=full"
```

Código 5.1: Verificación de las tres instancias OSRM desde el terminal del anfitrión. El parámetro `overview=full` fuerza la devolución de la geometría completa de la ruta; sin él, OSRM devuelve una versión simplificada con menos puntos.

La Figura 5.2 muestra las tres rutas calculadas simultáneamente para el mismo par origen-destino: en azul, la ruta de conducción sigue las calles principales; en verde, la ruta de bicicleta prioriza carriles bici y calles tranquilas; en gris con trazado discontinuo, la ruta peatonal accede a zonas restringidas al tráfico rodado. Las diferencias entre los tres trazados reflejan las distintas restricciones de cada perfil sobre la red viaria de Toledo.

↑ **MOVER a FRONTEND**

↓ Mover a FRONTEND

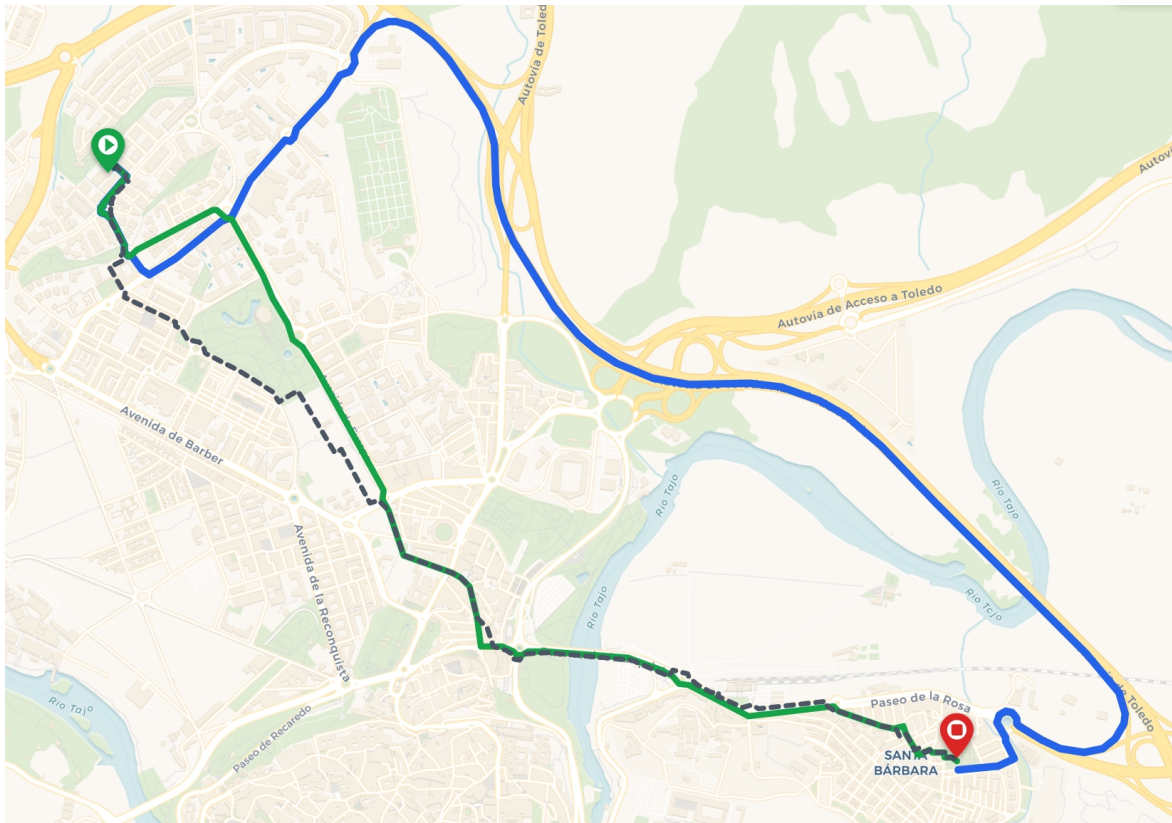


Figura 5.2: Rutas OSRM para los tres perfiles de transporte sobre Toledo.

5.2. Transporte público: OpenTripPlanner y feed GTFS

Esta sección cubre todo el transporte público del simulador: la elección del feed GTFS, la construcción del grafo de OpenTripPlanner (OTP) a partir de ese feed, la estructura de los ficheros GTFS y las particularidades del feed de Toledo, el periodo de validez de los datos y la integración de los itinerarios en el backend.

5.2.1. Fuentes de datos GTFS y proceso de selección

OpenTripPlanner requiere dos fuentes de datos: un extracto viario OSM para modelar los tramos de acceso a pie a las paradas, y un feed GTFS con el servicio programado del operador de transporte público. La fuente utilizada para los feeds fue el Punto de Acceso Nacional de Transporte Multimodal (NAP), gestionado por el Ministerio de Transportes y Movilidad Sostenible, que centraliza feeds GTFS de operadores de toda España [NAP Transporte Multimodal, 2026a]. El extracto OSM de Castilla-La Mancha ya disponible

para OSRM (Sección 5.1.1) se reutilizó directamente, sin necesidad de preparar ningún fichero adicional.

La disponibilidad de feeds de transporte urbano en Castilla-La Mancha resultó muy limitada. En Albacete solo figuraban servicios interurbanos como Alsa, sin datos de red urbana en el NAP. Guadalajara contaba con feeds del Consorcio de Transportes de Madrid, pero sin red municipal propia. Cuenca tampoco disponía de feed urbano. La única opción urbana era Ciudad Real, cuyo feed incluía también Seseña, con paradas repartidas entre dos localidades separadas por más de 170 km; además, en ese momento no incluía geometría de rutas, requisito imprescindible para la representación cartográfica.

Ante la ausencia de un feed válido en Castilla-La Mancha, se evaluaron feeds de otras ciudades. La EMT de Madrid [EMT Madrid, 2026], con 4.914 paradas, 236 rutas y 87.137 viajes programados, sirvió de banco de pruebas para validar la integración de OTP con datos reales y desarrollar los endpoints de la capa estática del backend. El volumen del feed reveló un problema de paginación: la API exige un límite explícito y devolvía solo las primeras 500 paradas por defecto, dejando la red visualmente incompleta. El límite se ajustó a 5.000, suficiente para Madrid y los feeds posteriores.

El feed GTFS elegido finalmente fue el de los autobuses urbanos de Toledo [Unauto, 2026]. Con 268 paradas, 56 rutas y 87.751 viajes que cubren aproximadamente tres meses de servicio programado, tiene un tamaño ajustado al alcance del simulador, abarca el casco urbano, el polígono industrial y las urbanizaciones periféricas, e incluye geometría de rutas para su representación cartográfica. Como el feed no incluye tarifas, el coste del billete se introduce como parámetro en el panel de la interfaz, junto al resto de variables del perfil de viajero, para su uso en la inferencia modal.

5.2.2. Construcción del grafo multimodal

El grafo se construye con un único comando Docker ejecutado sobre el directorio `otp-toledo/`, que debe contener el extracto OSM (`clm.osm.pbf`) y el feed GTFS (`GTFS_Urbano_Toledo_2026.zip`):

```
docker run --rm -v "otp-toledo:/var/opentripplanner"
  opentripplanner/opentripplanner:2.5.0 --build --save
```

OTP lee ambos ficheros y serializa el resultado en `graph.obj`. La red viaria OSM modela los tramos de acceso a pie a las paradas, y el feed GTFS aporta el servicio programado con horarios, paradas y geometría de las líneas. El proceso tarda unos minutos y solo es necesario repetirlo cuando se actualiza el feed; el extracto OSM puede reutilizarse entre versiones. El comando anterior corresponde a la generación manual del grafo.

tiempos

El `graph.obj` resultante (unos 117 MB) se versiona con *Git LFS* y, si falta en el clon, lo regenera automáticamente el servicio de inicialización `otp-build`; ambos mecanismos se describen en la Sección 4.2. En funcionamiento, OTP se lanza en modo de servicio (`--load --serve`) y queda accesible en el puerto 8080, exponiendo la API REST de planificación de rutas [OpenTripPlanner Team, 2026b].

5.2.3. Estructura del feed GTFS y particularidades del feed de Toledo

El formato GTFS (*General Transit Feed Specification*) [MobilityData, 2026a] estructura la información de transporte público como una colección de ficheros de texto plano con extensión `.txt` y formato de valores separados por comas, empaquetados en un único archivo ZIP. En el arranque del servicio, el backend descomprime y carga en memoria con *pandas* los seis ficheros necesarios: `stops.txt` (paradas con coordenadas y nombre), `routes.txt` (líneas de la red), `trips.txt` (servicios programados por línea), `stop_times.txt` (horarios de paso por cada parada), `shapes.txt` (geometría de los trazados) y `calendar_dates.txt` (calendario de excepciones de servicio) [MobilityData, 2026c]. Los *DataFrames* resultantes permanecen en memoria durante toda la vida del proceso, evitando releer el feed en cada petición y eliminando la necesidad de un sistema gestor de base de datos para información que no cambia en tiempo de ejecución.

Responder a preguntas como «¿qué líneas pasan por esta parada?» o «¿cuál es el horario de una ruta para una fecha concreta?» requiere cruzar varios de estos ficheros, tal como ilustra la Figura 5.3. Para obtener las líneas asociadas a una parada se encadenan tres cruces: `stops.txt` → `stop_times.txt` → `trips.txt` → `routes.txt`, unidos por `stop_id`, `trip_id` y `route_id`, respectivamente. Para los horarios por fecha se añade el filtro de `calendar_dates.txt`, que indica los días en que cada servicio está activo.

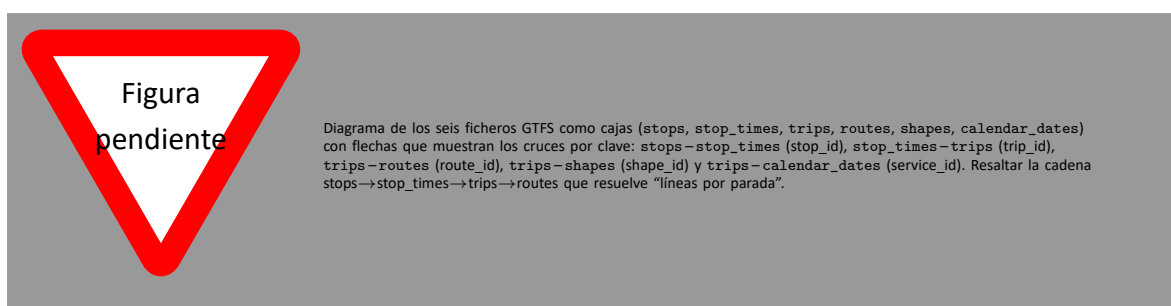


Figura 5.3: Cruces entre los ficheros del feed GTFS para resolver las consultas de la capa estática.

El feed de Toledo presenta una particularidad en la estructura de sus líneas. La empresa operadora, identificada en los feeds del NAP como UNAUTO S.L. (Grupo Ruiz), registra cada sentido de recorrido como un `route_id` independiente con el mismo `route_short_name`: el feed contiene 56 entradas en `routes.txt` que corresponden a solo 25 líneas distintas. Por ejemplo, la línea L5 tiene asignados los identificadores 50011 y 50012, uno por cada sentido de circulación. Tanto el backend como el frontend agrupan los `route_id` con el mismo `route_short_name` en un diccionario de hermanos (`siblingRouteIds`), de modo que el catálogo de líneas muestra una sola entrada por línea. Al seleccionar una parada concreta desde el mapa, que conoce el `route_id` exacto del servicio que pasa por ella, se activa directamente el sentido correcto sin lógica adicional. Esta agrupación por sentido se reutiliza en el router GTFS de la capa estática (Sección 5.3.4) y en el panel Red GTFS del frontend (Sección 5.4.5).

5.2.4. Periodo de validez del feed y selección de fecha

Los feeds GTFS del NAP tienen un periodo de validez limitado; el del operador de Toledo (`GTFS_Urbano_Toledo_2026.zip`) cubre del 22 de febrero al 22 de mayo de 2026. Si la fecha de consulta cae fuera de ese rango, OTP no devuelve itinerarios con tramos de transporte público y el simulador trata la situación como si no hubiera servicio disponible, devolviendo únicamente un trayecto a pie equivalente al que proporciona OSRM con el perfil peatonal.

~~El mismo feed alimenta los dos subsistemas que lo consumen: el grafo de OTP (Sección 5.2.2) y la capa estática de horarios que expone el router GTFS (Sección 5.3.4), de modo que ambos comparten exactamente la misma ventana de validez. La fecha y la hora de consulta se exponen mediante un selector global en la interfaz, con un valor por defecto dentro del rango del feed (2026-05-21, 12:00) y acotado al intervalo válido. Este selector es transversal a toda la simulación: gobierna a la vez los itinerarios multimodales, los horarios de la red y el día y la hora del perfil de inferencia, evitando configuraciones inconsistentes entre las distintas vistas. La ubicación y el comportamiento de este selector en la interfaz se describen en la Sección 5.4.2. El backend recibe la fecha y la hora en la petición; cuando no se indican, aplica el valor por defecto.~~

Por ello,

Se fijó la hora por defecto a las 12:00 por situarse en la franja de mayor frecuencia de servicio y evitar el horario nocturno, donde la oferta es reducida o inexistente.

5.3
Router OTP

5.2.5. Integración de itinerarios en el backend

El router `/api/otp` del backend consulta OTP a través del endpoint REST de planificación `/otp/routers/default/plan`, que es el punto de acceso estándar de OTP 2.x para solicitar itinerarios entre dos coordenadas. OTP devuelve hasta cinco itinerarios alternativos. El criterio de selección automática es el siguiente: se elige el primero que incluya al menos un tramo de transporte público (modo distinto de WALK); si ninguno cumple esta condición, se devuelve el de menor duración total. El frontend permite paginar entre las alternativas mediante los botones “Anterior” y “Siguiente”, que transmiten el índice del itinerario deseado al backend, de modo que el vector de características para la inferencia se calcula siempre sobre el mismo itinerario que el usuario visualiza.

Para cada tramo del itinerario seleccionado se devuelve: modo de transporte, distancia, duración, geometría decodificada desde el formato de polilínea codificada, parada de inicio y fin, nombre de línea, agencia y horarios de salida y llegada en formato HH:MM. Para los tramos de transporte público se añade a la consulta el parámetro `showIntermediateStops`, que incorpora a la respuesta de OTP las paradas intermedias de cada tramo; con ellas el backend compone la secuencia ordenada completa de paradas del tramo, desde el embarque hasta el desembarque, anotando para cada una su nombre, coordenadas y hora de paso. Esta secuencia alimenta el diagrama de paradas del itinerario descrito en la Sección 5.4.4. La geometría de cada tramo se combina para componer la traza completa del itinerario, que el frontend renderiza diferenciando los segmentos por modo de transporte.

OTP devuelve los instantes de paso en tiempo universal (epoch UTC) acompañados del desfase horario de la agencia (`agencyTimeZoneOffset`). El backend suma ese desfase antes de formatear las horas, de modo que los itinerarios se muestran en hora local de Toledo (Europe/Madrid), con el horario de verano resuelto automáticamente según la fecha consultada.

Cuando OTP devuelve un itinerario compuesto exclusivamente por un tramo a pie, las características de transporte público del vector de entrada tomarían valores próximos a cero, que en el dataset caracterizan precisamente un transporte público rápido y directo. La solución adoptada, descrita en la Sección 5.5.5, consiste en sobrescribir esas características con valores de penalización extremos que llevan al modelo a descartar el transporte público en ausencia de servicio real.

La Figura 5.6 en la Sección 5.4.4 ilustra un itinerario multimodal típico en la interfaz del simulador.

5.3. Implementación del backend FastAPI

5.3.1. Estructura modular

El backend está implementado con FastAPI sobre Python y organizado en módulos por responsabilidad. Un directorio `api/` contiene los cuatro routers (`routes_osrm.py`, `routes_otp.py`, `routes_gtfs.py`, `routes_lpmc.py`); un directorio `services/` contiene la lógica de negocio desacoplada de la capa HTTP (`osrm_client.py`, `lpmc_inference.py`); y el módulo raíz `main.py` monta los routers y habilita CORS (*Cross-Origin Resource Sharing*), el mecanismo que autoriza al navegador a que la interfaz web, servida desde un puerto distinto, consuma la API del backend. Los endpoints expuestos por cada router se recogen en la Tabla 4.3 del capítulo 4.

La separación en módulos permite desarrollar, probar y documentar cada router por separado. Además, FastAPI genera de forma automática documentación interactiva siguiendo el estándar OpenAPI en la ruta `/docs`. Durante el desarrollo, esta documentación permitió lanzar peticiones a cada endpoint y comprobar sus respuestas directamente desde el navegador, sin escribir código adicional de prueba.

ni buscas el frontend

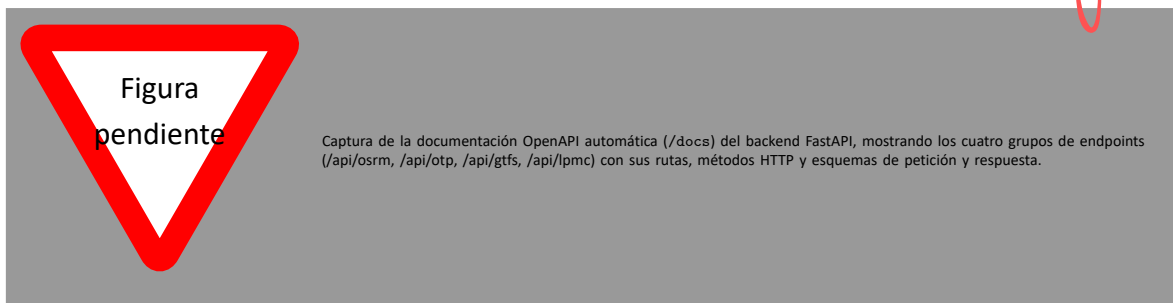


Figura 5.4: Documentación OpenAPI generada automáticamente por FastAPI.

5.3.2. Router OSRM: consultas multiperfil

El router `/api/osrm` expone el endpoint `POST /api/osrm/routes`, que acepta un par origen-destino y la lista de perfiles de transporte solicitados. Las consultas a los tres contenedores OSRM se lanzan en paralelo mediante la primitiva `asyncio.gather`, de modo que el tiempo total de respuesta equivale al de la consulta más lenta y no a la suma de las tres.

Para cada perfil, OSRM devuelve la distancia en metros, la duración en segundos y la geometría en formato de polilínea codificada. El backend decodifica esa geometría en una lista de coordenadas [lat, lon] para que pueda enviarse al frontend y representarse con Leaflet. Por último, las duraciones se convierten de segundos a horas antes de construir el vector de características, ya que el dataset LPMC almacena todas las duraciones en esa unidad.

5.3.3. Router OTP: itinerarios multimodales

El router `/api/otp` expone el endpoint `POST /api/otp/routes`, que consulta el planificador de OTP, gestiona la selección de itinerario (automática o según el que el usuario tenga seleccionado en el frontend) y transforma la respuesta de OTP a un formato propio y homogéneo de tramos, más sencillo de consumir por el frontend. El procesamiento completo de los itinerarios (desglose por tramos, paradas intermedias, horarios y ajuste de zona horaria) ya se ha descrito en la Sección 5.2.5, junto al resto de la integración de OpenTrip-Planner.

5.3.4. Router GTFS: capa estática

El router `/api/gtfs` expone la información estática de la red de autobús directamente a partir de los ficheros del feed GTFS cargados en memoria (Sección 5.2.3), sin depender de OpenTripPlanner. Esto permite consultar y visualizar la red aunque el planificador no esté disponible. Expone cuatro endpoints:

- **Listado de paradas** (GET `/api/gtfs/stops`): devuelve todas las paradas del feed con identificador, nombre, coordenadas y líneas asociadas. Acepta un parámetro `limit` (5.000 por defecto, suficiente para el feed de Toledo con 268 paradas) y un filtro de caja delimitadora opcional para restringir el resultado al área visible en el mapa.
- **Listado de líneas** (GET `/api/gtfs/routes`): catálogo completo de líneas con identificador, nombre corto, nombre largo y atributos de color cuando están disponibles en el feed.
- **Detalle de línea** (GET `/api/gtfs/routes/{route_id}`): dado el identificador de una línea, devuelve la secuencia ordenada de paradas y la geometría del trazado asociado para su representación cartográfica.
- **Horarios por fecha** (GET `/api/gtfs/routes/{route_id}/schedule`): dado un identificador de ruta y una fecha mediante el parámetro `?date=YYYY-MM-DD`, devuel-

ve los servicios programados agrupados por sentido, con primera y última salida y listado completo de horarios. Al igual que con OTP (Sección 5.2.4), la interfaz usa una fecha dentro del rango de validez del feed; cualquier fecha posterior devuelve cero servicios.

La lógica de agrupación de sentidos descrita en la Sección 5.2.3 (los 56 `route_id` del feed de Toledo agrupados en 25 líneas por `route_short_name`) se aplica tanto en este router como en el frontend, de modo que el comportamiento es coherente en todo el sistema.

5.3.5. Router LPMC: inferencia modal y descubrimiento de modelos

El router `/api/lpmc` centraliza la lógica de inferencia de elección modal y el acceso a los modelos entrenados. A diferencia de los routers de enrutado, que delegan el cálculo en servicios externos (OSRM y OTP), este router ejecuta la predicción de forma local sobre los artefactos serializados en `lpmc/models/`. Expone cuatro endpoints:

- **Predicción** (POST `/api/lpmc/predict`). Recibe el par origen-destino, el itinerario de OTP que el usuario tiene seleccionado, las variables sociodemográficas del viajero y, de forma opcional, el nombre del modelo a usar (`model_variant`). Cuando no se indica el nombre del modelo, se recurre a la variable de entorno `LPMC_MODEL_VARIANT`. La respuesta contiene la probabilidad de cada uno de los cuatro modos de transporte (a pie, bicicleta, transporte público y coche), el modo más probable para ese viajero junto con su probabilidad, y el vector de características de ruta con el que se ha inferido. Incluye además algunos metadatos útiles para depuración, entre ellos el indicador `pt_available`, que vale verdadero cuando el itinerario contiene al menos un tramo de transporte público y falso cuando no hay servicio real (caso descrito en la Sección 5.5.5).
- **Comparación de modelos** (POST `/api/lpmc/compare`). Ejecuta la predicción con los tres modelos sobre el mismo escenario y devuelve las probabilidades de cada uno, de modo que pueden contrastarse en la interfaz. Los tres modelos se evalúan uno tras otro reutilizando el mismo vector de características, que solo se construye una vez.
- **Depuración de características** (POST `/api/lpmc/debug-features`). Devuelve el vector de características completo, con sus valores antes y después de aplicar el escalado estándar. Permite comprobar que las variables de ruta calculadas a partir de OSRM y OTP caen en el rango esperado por el dataset, y fue la herramienta principal para diagnosticar los problemas de unidades que se describen en la Sección 5.5.4.

-
- **Modelos disponibles** (GET /api/lpmc/models). Escanea el directorio `lpmc/models/` en busca de pares de ficheros `{nombre}_lpmc.joblib` y `{nombre}_lpmc_scaler.joblib`, y devuelve la lista de modelos encontrados junto con el predeterminado (LPMC_MODEL_VARIANT). Gracias a este barrido, basta con dejar un modelo entrenado fuera del sistema en ese directorio para que la interfaz lo detecte en la siguiente petición, sin reiniciar ningún contenedor.

El nombre del modelo que llega desde el exterior se valida con la expresión regular `^[a-z0-9_]{1,32}$`, que solo admite letras minúsculas, dígitos y guiones bajos. La validación es necesaria por seguridad: ese nombre se concatena para formar la ruta del artefacto a cargar (`{nombre}_lpmc.joblib`), y sin filtrarlo un valor manipulado como `../..` podría leer ficheros fuera del directorio de modelos. Por último, los modelos se cargan de forma diferida y quedan en una caché en memoria: la primera predicción con un modelo asume el coste de leerlo y deserializarlo desde disco, y las siguientes lo reutilizan sin volver a cargarlo. La resolución de rutas de estos artefactos, portable entre sistemas operativos, se describe en la Sección 4.2.

5.4. Implementación del frontend

5.4.1. Tecnologías y estructura

El frontend utiliza las siguientes tecnologías principales:

- **React 18** [Meta Open Source, 2024]: biblioteca de construcción de interfaces de usuario basada en componentes reutilizables y actualizaciones reactivas del árbol de elementos DOM.
- **Vite** [Vite Team, 2024]: herramienta de compilación que sirve los módulos de forma nativa durante el desarrollo, con actualización del módulo modificado sin recargar la página, y genera el paquete de producción optimizado.
- **TypeScript** [Microsoft, 2024]: lenguaje que extiende JavaScript añadiendo tipos comprobados de forma estática. Resulta especialmente útil al modelar las respuestas del backend, como los itinerarios OTP con desglose de tramos, los feeds GTFS con múltiples entidades relacionadas o los vectores de probabilidades de los modelos.
- **Leaflet** [Leaflet Contributors, 2024], integrado mediante **React-Leaflet** [Le Cam and React-Leaflet Contributors, 2024]: biblioteca de mapas interactivos

de código abierto. React-Leaflet expone cada primitiva cartográfica (marcadores, polilíneas, capas de teselas) como un componente React declarativo.

- **Lucide React** [Lucide Contributors, 2024]: colección de iconos SVG de código abierto, utilizada en los botones de la barra lateral y los controles del mapa. Al ser vectoriales, escalan sin pérdida de calidad.
- **TanStack Query** [Linsley and TanStack Contributors, 2024]: librería de gestión del estado asíncrono para React que aporta caché de respuestas, seguimiento del ciclo de vida de cada petición (carga, error, éxito) y reintento automático.

La aplicación se estructura en dos componentes principales: `App` concentra el estado global y la lógica del cliente, y `MapView` recibe el estado necesario como *props* (datos que el componente padre pasa al hijo de forma unidireccional) y es responsable exclusivamente del renderizado cartográfico. Esta separación se describe en la Sección 4.4.

5.4.2. Interfaz cartográfica y controles del mapa

La interfaz sigue el esquema de las aplicaciones cartográficas modernas: el mapa ocupa toda la ventana del navegador como fondo, mientras los controles y los paneles de navegación se superponen sobre él. Leaflet renderiza el mapa a partir de teselas de los proveedores seleccionados, sin descargar ningún mapa local; el extracto OSM que usa OSRM es solo para el cálculo de rutas en el servidor. Las rutas calculadas por OSRM cubren únicamente el territorio del extracto de Castilla-La Mancha, y las paradas de autobús disponibles pertenecen exclusivamente al feed GTFS de Toledo.

Los marcadores de origen y destino se implementan con iconos SVG propios embebidos en CSS, distintos de los marcadores predeterminados de Leaflet, para distinguir visualmente el punto de inicio del de llegada. Las paradas de autobús se representan como marcadores circulares (`CircleMarker`) en una capa de renderizado propia de Leaflet (*pane stopsPane*, z-index 450 frente a los 400 de las polilíneas). Este orden de apilamiento asegura que las paradas sean siempre accesibles a la interacción del usuario aunque un trazado las atraviese.

Los controles de navegación del mapa se implementaron como componentes React, en lugar de los controles predeterminados de Leaflet, para seguir el mismo estilo visual que el resto de la interfaz. Se organizan en dos grupos fijos sobre el mapa:

- **Esquina inferior derecha:** botones de acercar y alejar la vista, más un botón de recentrar que devuelve el mapa al centro de Toledo con el zoom inicial.

-
- **Esquina superior derecha:** tres botones de limpieza que eliminan, respectivamente, las rutas de navegación viaria (OSRM y OTP), la línea de bus activa en el panel Red GTFS, o ambas cosas junto con los puntos de origen y destino. Los botones se deshabilitan automáticamente cuando no hay datos que eliminar.

El zoom puede ajustarse en pasos de 0,25 en lugar de enteros, lo que aporta mayor precisión al encuadrar áreas urbanas concretas.

La barra lateral de navegación (*rail*) de 64 px de ancho, siempre visible a la izquierda de la pantalla, da acceso a los seis paneles funcionales del simulador. Al pulsar un botón de la *rail*, el panel correspondiente se despliega a su derecha; pulsar de nuevo el mismo botón lo cierra. El panel activo al arrancar la aplicación es **Inicio**. Los demás paneles son **Rutas**, **Red GTFS**, **Predicción IA**, **Capas** y **Ajustes**, descritos en las secciones siguientes.

ancho 64 px se ancho

La maqueta es adaptable (*responsive*): el mapa ocupa siempre el espacio disponible y los paneles y controles se dimensionan con unidades relativas, de modo que la interfaz se ajusta a distintos tamaños de ventana y resoluciones, desde un monitor de escritorio hasta la pantalla de una tableta.

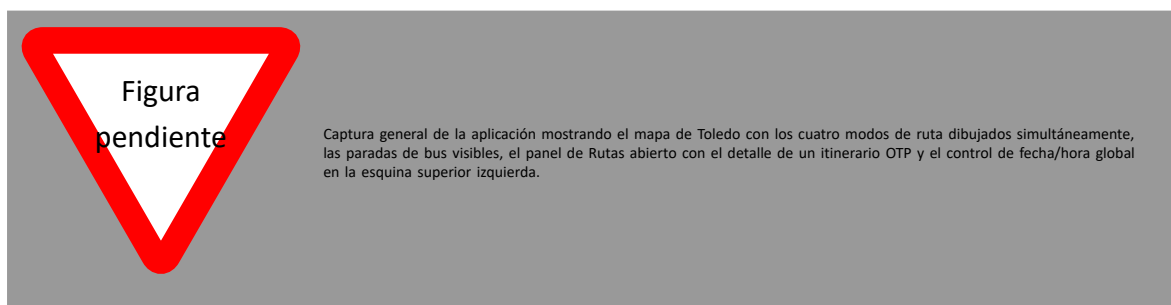


Figura 5.5: Vista general de la interfaz del simulador de movilidad urbana.

5.4.3. Panel Inicio

El panel Inicio es la vista por defecto al cargar la aplicación. Presenta una descripción del proyecto y su contexto académico, una lista de las tecnologías utilizadas con sus versiones, y una tabla de créditos con autor, tutor, centro, dataset, feed GTFS y cartografía base. El objetivo es orientar al usuario que accede al simulador por primera vez sin que necesite consultar documentación externa. Cerrar el panel o cambiar a otro no afecta al estado del mapa ni a los cálculos en curso.

5.4.4. Panel Rutas

El panel Rutas agrupa todas las funciones de navegación viaria y multimodal. Los puntos de origen y destino se establecen mediante el menú contextual (clic derecho sobre el mapa): la primera entrada muestra las coordenadas del punto pulsado con cinco decimales y las copia al portapapeles, con un cambio temporal del texto a «¡Copiado!» como confirmación; las dos entradas siguientes asignan el punto como origen o como destino. El mismo par de coordenadas puede editarse también directamente en los campos de texto del panel, pegando un valor «latitud, longitud» procedente de cualquier fuente; el campo valida el rango geográfico al confirmar con *Intro* o al perder el foco, y restaura el valor previo si el formato no es válido.

Los botones de selección de modo de transporte no son mutuamente excluyentes: un clic normal selecciona en exclusiva el modo pulsado, mientras que un clic con *Shift* añade o retira ese modo sin modificar el resto. Cuando hay varios modos activos, MapView dibuja una polilínea independiente por modo de transporte: azul para conducción, verde para bicicleta y gris discontinuo para el desplazamiento a pie. El itinerario de transporte público combina dos estilos, ya que no es homogéneo: los tramos en autobús se dibujan en naranja (el color asignado a este modo) y los tramos a pie de acceso a la parada o de transbordo, en un naranja más oscuro y discontinuo, para distinguirlos del gris del modo a pie cuando se muestran varios modos a la vez. Sobre el mapa, las paradas del trayecto se colorean con el color real de su línea, para localizar los transbordos de un vistazo. Las tres rutas OSRM se calculan siempre en paralelo, por lo que cambiar los modos activos no requiere ninguna nueva petición al servidor.

Al pulsar «Calcular rutas» se lanza la petición a `POST /api/osrm/routes` y, si el modo de transporte público está activo, también a `POST /api/otp/routes`. Una vez realizado ese primer cálculo, cualquier modificación posterior de los extremos O/D relanza ambas peticiones de forma automática. Este auto-recálculo se desactiva al limpiar las rutas y permanece inactivo hasta el siguiente cálculo manual, evitando peticiones antes de que el usuario haya definido el escenario.

Los resultados OSRM se presentan en una tabla con una fila por modo activo, mostrando distancia y duración estimada. El itinerario OTP se muestra en una tarjeta independiente con controles de paginación para navegar entre las alternativas devueltas. El encabezado de la tarjeta resume la hora de salida, la hora de llegada y la duración total; a continuación se despliega el detalle tramo a tramo. Los tramos a pie se resumen en una línea con distancia y duración. Cada tramo en autobús se encabeza con la etiqueta coloreada de la línea y las horas de salida y llegada, y bajo él se dibuja un diagrama vertical con todas las paradas:

una barra del color de la línea, extremos como puntos rellenos, paradas intermedias como aros huecos y la hora de paso junto a cada parada. Al pulsar una parada del diagrama el mapa se desplaza hasta ella y se abre su ventana emergente. Cuando el itinerario incluye transbordos directos (cambio de línea en la misma parada sin tramo a pie entre ellos), el diagrama lo indica con una etiqueta de transbordo entre los dos bloques.

La etiqueta coloreada de cada línea en el panel debe coincidir con el color del catálogo GTFS. Sin embargo, OTP identifica las líneas con un identificador prefijado por el código del feed (por ejemplo, 1:50011), que no coincide directamente con el `route_id` del feed estático. La función auxiliar `resolveGtfsRoute()` resuelve la correspondencia en tres intentos sucesivos: por identificador exacto, por identificador sin el prefijo del feed y por nombre corto. Este proceso es necesario también para aplicar el contorno de contraste que se describe en la Sección 5.4.5.

La Figura 5.6 muestra un itinerario multimodal con el desglose de tramos desplegado en la interfaz.

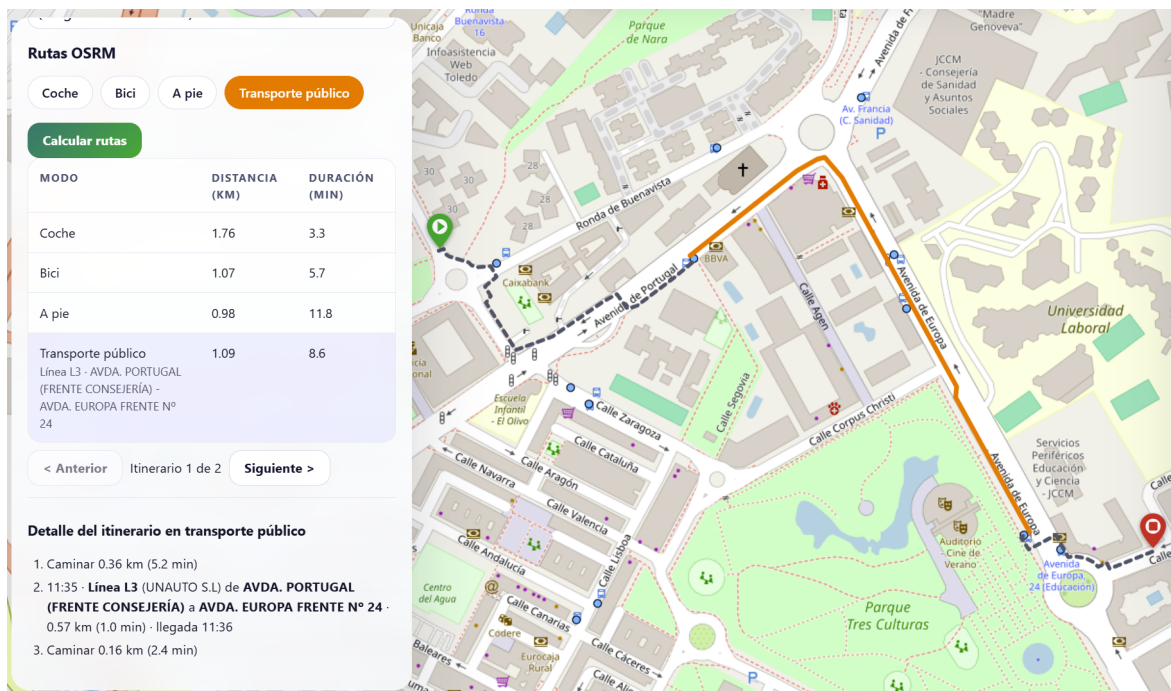


Figura 5.6: Itinerario multimodal OTP con desglose de tramos en la interfaz.

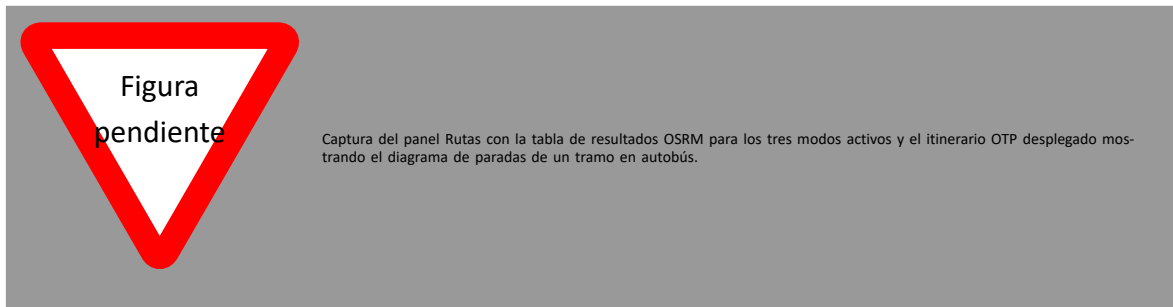


Figura 5.7: Panel Rutas con resultados OSRM e itinerario OTP.

5.4.5. Panel Red GTFS

El panel Red GTFS permite explorar la red de autobús urbano de Toledo sin necesidad de calcular un itinerario. Muestra las líneas disponibles, el trazado de la línea seleccionada sobre el mapa, la secuencia de paradas y los horarios del día activo.

El feed de Toledo contiene 56 sentidos (identificadores `route_id` distintos) agrupados en 25 líneas por nombre corto (`route_short_name`). El panel organiza los contenidos en un acordeón, un listado plegable en el que cada elemento puede expandirse o contraerse de forma independiente para mostrar u ocultar su contenido. Cada entrada del nivel superior representa una línea; las líneas con más de un sentido muestran un icono de despliegue y, al expandirlas, listan los sentidos disponibles con el sentido activo resaltado. El color de cada línea sigue una paleta fija de 26 colores definida en el cliente: los nombres cortos únicos se ordenan alfabéticamente y el i -ésimo recibe `LINE_COLORS[i mod 26]`, con lo que las 25 líneas del feed reciben colores únicos. Una vez determinado el color, la función `isLightColor()` calcula la luminancia perceptual con la fórmula $0,299R + 0,587G + 0,114B$; si el resultado supera 0,9 (líneas de color claro, como la L14 en blanco), se añade un contorno negro a las etiquetas coloreadas, a los marcadores de parada y al trazado del mapa para mantener la legibilidad sobre fondo blanco.

Al seleccionar una línea, el mapa dibuja su trazado como polilínea del color correspondiente y sus paradas como marcadores circulares rellenos con ese mismo color y un contorno de contraste. Los marcadores se renderizan en la capa `stopsPane` (z-index 450), por encima de las polilíneas (z-index 400), de modo que son siempre accesibles a la interacción aunque el trazado los cruce. Al pulsar una parada del listado lateral, la vista se desplaza hasta ella (`flyTo` a zoom 17) y, transcurridos 900 ms, se abre la ventana emergente de la parada. Las ventanas emergentes muestran nombre, código y las etiquetas coloreadas de las líneas que pasan por ella; al pulsar una etiqueta se selecciona ese sentido directamente en el panel.

Los horarios se presentan agrupados por hora: una columna con la hora y, a su derecha,

los minutos de cada salida estilizados según su posición respecto a la hora activa del control global de fecha. Los minutos ya pasados aparecen en gris; los futuros, en azul; el próximo (menor tiempo restante), en negrita. Si no quedan salidas futuras ese día se muestra un aviso; si la línea no circula en la fecha seleccionada, el panel informa de ello sin tabla de horarios.

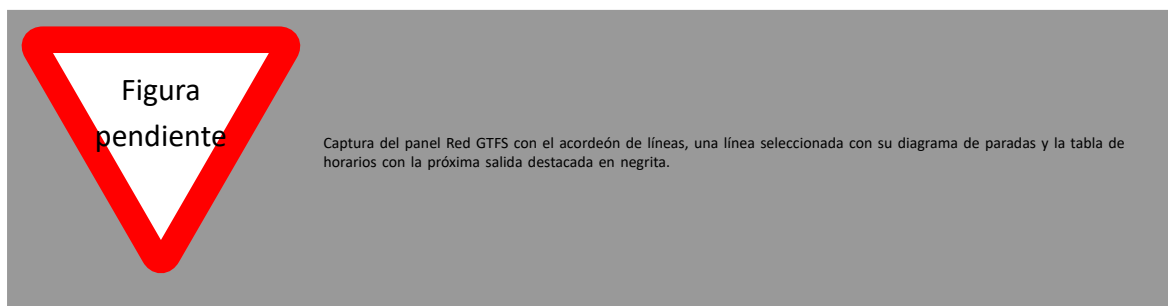


Figura 5.8: Panel Red GTFS con trazado, paradas y horarios de una línea.

5.4.6. Panel Predicción IA

El panel Predicción IA, que muestra la Figura 5.9, permite obtener la probabilidad de cada modo de transporte para un viajero concreto en el escenario definido sobre el mapa. Su disposición sigue el flujo de los datos que recibe el modelo. En primer lugar, en modo de solo lectura, aparecen las coordenadas de origen y destino que se pasan a OSRM y OTP, de modo que el usuario verifica de un vistazo el escenario. Justo debajo, también en solo lectura, se muestran los tiempos y distancias que esos servicios calculan para cada modo de transporte (las variables de ruta que alimentan al modelo), de manera que el usuario ve exactamente con qué cifras se va a inferir. A continuación se indica la información del viaje (día de la semana y hora de salida), derivada del control global de fecha y hora; al no ser editable aquí, se evita cualquier inconsistencia entre el itinerario calculado y el perfil enviado al modelo.

Para facilitar la exploración del simulador, el panel ofrece tres perfiles sociodemográficos predefinidos que autocompletan todos los campos del formulario y ajustan además la fecha y la hora globales al escenario que representan:

- **Commuter** (del inglés *commuter*, persona que se desplaza a diario entre su domicilio y el trabajo): viaje al trabajo (HBW, *home-based work*) un martes a las 8:15. Varón de 36 años con carnet de conducir y un vehículo en el hogar.

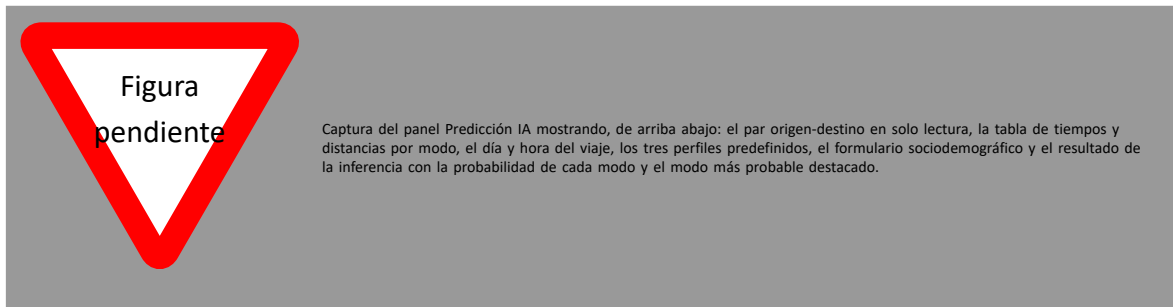


Figura 5.9: Panel Predicción IA con el formulario y el resultado de la inferencia.

- **Estudiante:** viaje por motivos de educación (HBE, *home-based education*) un miércoles a las 7:45. Mujer de 21 años sin carnet de conducir ni vehículo en el hogar, con tarifa de transporte público reducida.
- **Familiar:** viaje de ocio (HBO, *home-based other*) un sábado a las 11:30. Mujer de 44 años con carnet de conducir y dos vehículos en el hogar.

Al cargar la aplicación se aplica el perfil *Commuter* por defecto, de modo que el panel arranca con un escenario completo y coherente. El indicador de perfil activo desaparece en cuanto el usuario modifica algún campo del formulario, eliminando la ambigüedad sobre qué valores se están usando. El resto de parámetros sociodemográficos (edad, género, carnet, vehículos en el hogar, motivo del viaje, tipo de combustible del vehículo, costes) se introducen en el formulario y son editables en cualquier momento. La inferencia se lanza con el botón «Inferir modo», que muestra junto a su rótulo el nombre del modelo activo. El resultado es un conjunto de probabilidades para los cuatro modos de transporte (a pie, bicicleta, transporte público y conducción), con el modo de mayor probabilidad destacado visualmente.

Tras la inferencia, bajo el botón aparecen las probabilidades de los cuatro modos en forma de barras y, solo entonces, se habilitan dos acciones secundarias para profundizar en el resultado. La primera, «Comparar modelos», llama a `POST /api/lpmc/compare` y presenta una tabla con las probabilidades de los tres modelos (XGBoost, Random Forest y red neuronal) para el mismo escenario, lo que permite ver hasta qué punto coinciden (Figura 5.10). La segunda, «Ver variables», llama a `POST /api/lpmc/debug-features` y abre una ventana modal con el vector de características completo, en sus valores originales y tras el escalado, útil para inspeccionar con qué datos exactos se ha inferido (Figura 5.11).

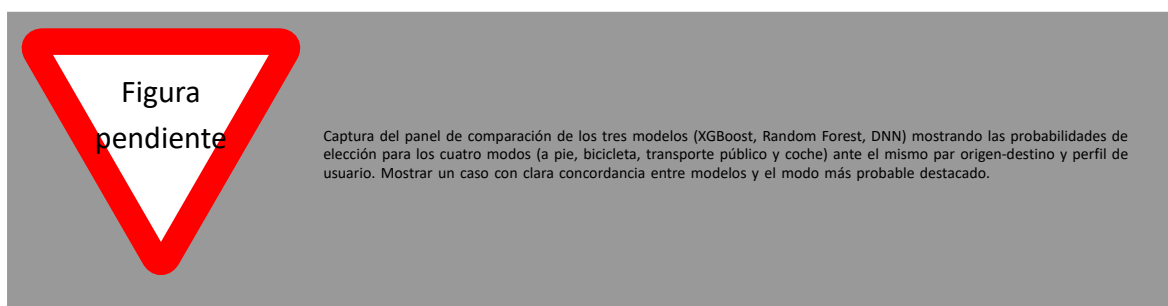


Figura 5.10: Panel de comparación de probabilidades de los tres modelos de elección modal.

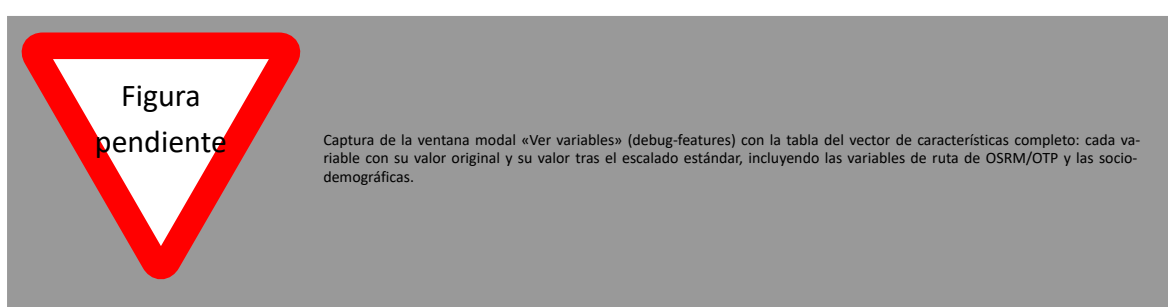


Figura 5.11: Ventana de depuración del vector de características que recibe el modelo.

5.4.7. Panel Capas

El panel Capas permite seleccionar la capa de fondo del mapa entre seis opciones, cada una orientada a un tipo de análisis o preferencia visual:

- **Color** (CartoDB Voyager): calles y nombres de lugar en tonos suaves. Seleccionada por defecto al iniciar la aplicación por ofrecer contexto urbano sin competir visualmente con las rutas superpuestas.
- **Blanco y negro** (CartoDB Positron): paleta completamente desaturada que minimiza el ruido visual y maximiza la legibilidad de los trazados.
- **OpenStreetMap**: mapa estándar de la comunidad OSM, con mayor detalle de nombres e infraestructuras.
- **Topográfico** (OpenTopoMap): curvas de nivel y sombreado de relieve, especialmente relevante para trayectos en bicicleta o a pie por zonas con desnivel.
- **Satélite** (Esri World Imagery): ortofoto aérea global con resolución variable según la región.

- **PNOA** (Plan Nacional de Ortofotografía Aérea, IGN España): ortofoto servida mediante el protocolo WMTS por el Instituto Geográfico Nacional; cubre el territorio español con resolución de 25 cm en zonas urbanas y no requiere token de autenticación.

~~La selección de capa persiste mientras el usuario navega entre los demás paneles. Un componente auxiliar (BasemapZoomSnapper) detecta el cambio de proveedor y redondea el nivel de zoom al entero más próximo si es fraccionario, evitando errores de carga con los servidores de teselas que no sirven niveles no enteros.~~

Si se hace cambio de capa...

5.4.8. Panel Ajustes

~~El panel Ajustes agrupa tres secciones de configuración global. La primera sincroniza la visibilidad de las paradas de autobús en el mapa con el interruptor del panel Red GTFS, permitiendo activarlas o desactivarlas desde cualquiera de los dos puntos de la interfaz sin inconsistencias. La segunda aloja el selector de modelo activo de inferencia: al abrirse el panel por primera vez, la interfaz consulta GET /api/lpmc/models para obtener la lista de modelos disponibles e inicializa el selector con el valor de default_variant que devuelve la API, que a su vez refleja el contenido de la variable de entorno LPMC_MODEL_VARIANT. Cambiar la selección no dispara ninguna petición inmediata; el identificador del modelo elegido se adjunta como campo del cuerpo de la siguiente petición a POST /api/lpmc/predict. La tercera sección contiene instrucciones colapsables para añadir modelos propios: convenio de nombres ({nombre}_lpmc.joblib y {nombre}_lpmc_scaler.joblib en lpmc/models/), formatos admitidos (cualquier objeto sklearn con interfaz predict_proba, o una red PyTorch cargable mediante TorchModalWrapper) y cómo fijar el modelo predeterminado. Cualquier par de ficheros que siga el convenio es detectado automáticamente por GET /api/lpmc/models sin modificar el código ni reiniciar los contenedores.~~

La Figura 5.12 muestra el panel con los tres modelos incluidos en el repositorio disponibles en el desplegable.

5.4.9. Gestión del estado con TanStack Query

~~Las peticiones al backend se gestionan con TanStack Query (@tanstack/react-query). Los datos GTFS estáticos (paradas, lista de rutas) se cargan con useQuery al arrancar la aplicación y se mantienen en caché durante la sesión. Las peticiones de rutas OSRM, itinerarios OTP e inferencia LPMC se declaran como mutaciones (useMutation), que se disparan manualmente al pulsar los controles de cálculo en la interfaz.~~

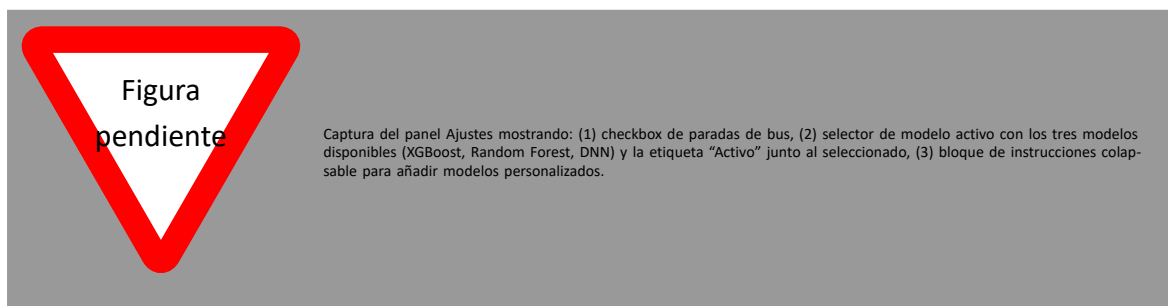


Figura 5.12: Panel de ajustes con selector de modelo activo y documentación de extensión.

TanStack Query gestiona de forma automática los estados de carga y error, la caché y el reintento con retroceso exponencial (cada reintento espera más que el anterior). Las tres mutaciones de inferencia LPMC (`/predict`, `/compare`, `/debug-features`) se declaran por separado, de modo que la predicción con el modelo activo y la comparación de los tres modelos se lanzan de forma independiente, sin afectar al estado del resto de la interfaz.

5.5. Modelos de elección modal

Esta sección describe el proceso de entrenamiento, validación y despliegue en producción de los tres modelos de elección modal que integra el simulador. La arquitectura del pipeline de inferencia se describe en la sección 4.5; aquí se detalla la implementación concreta: el dataset utilizado, las decisiones de preprocesado, la configuración de los modelos y los resultados obtenidos.

5.5.1. Dataset LPMC y variables de entrada

El dataset London Passenger Mode Choice (LPMC) recoge registros de viajes reales observados en el área metropolitana de Londres entre 2012 y 2015. Procede de la *London Travel Demand Survey* (LTDS), la encuesta de movilidad de Transport for London, sobre la que Hillel et al. construyeron el dataset que aquí se utiliza: a cada viaje observado le añadieron las variables de nivel de servicio, esto es, los tiempos y costes de cada modo de transporte alternativo (a pie, bicicleta, transporte público y coche) para ese mismo par origen-destino, calculados aunque el viajero no eligiera ese modo. Se citan dos publicaciones porque corresponden a aportaciones distintas: el artículo original que presenta el dataset [Hillel et al., 2018] y su descripción técnica detallada [Hillel, 2019]. El dataset es, ade-

más, el utilizado en los experimentos de referencia sobre los que se apoya este trabajo [Martín-Baos, 2023, José Ángel Martín-Baos, 2023]. La Tabla 5.2 resume sus características principales.

Tabla 5.2: Características generales del dataset LPMC.

Característica	Valor
Registros totales	81.086
Hogares únicos	17.616
Modos de viaje	4 (<i>walk, cycle, pt, drive</i>)
Oleadas de encuesta	3 (<i>survey_year</i> 1, 2, 3)
Años de viaje cubiertos	2012–2015
Variables originales	34

El interés metodológico del dataset para este TFM no reside en el caso geográfico concreto (Londres), sino en que sus variables de nivel de servicio son justamente las que OSRM y OTP calculan para cualquier par origen-destino (tiempos por modo, transbordos, costes). Contiene también las variables sociodemográficas que el simulador pide al usuario, tiene un tamaño suficiente para entrenar modelos de aprendizaje automático con validación cruzada robusta, y ya ha sido validado para esta misma tarea en investigaciones previas [José Ángel Martín-Baos, 2023].

Las variables de entrada al modelo se dividen en dos grupos. El primero comprende las variables de ruta, derivadas automáticamente de OSRM y OTP para cada par origen-destino consultado en el simulador. El segundo comprende las variables sociodemográficas y contextuales, introducidas por el usuario a través del panel lateral de la interfaz o autocompletadas a partir de uno de los perfiles predefinidos. El listado completo de variables, con su tipo de dato y su origen, se recoge en la Tabla A.1 del Anexo A.3.

5.5.2. Preprocesado de datos

El preprocesado del dataset transforma los datos en bruto en el conjunto de variables que reciben los modelos. Se aplica en cuatro pasos.

1. **Eliminación de columnas sin valor predictivo.** Se descartan doce columnas. Tres son identificadores (*trip_id*, *person_n*, *trip_n*), que no aportan señal. Tres son variables de fecha de grano fino (*travel_year*, *travel_month*, *travel_date*), cuya información ya recogen *day_of_week* y *start_time_linear*. Una es un fac-

tor de política exógeno que no está disponible en inferencia (*bus_scale*). Las cinco restantes son variables derivadas redundantes, que se obtienen sumando otras ya presentes (*dur_pt_total*, *dur_pt_int_total*, *cost_driving_fuel*, *cost_driving_con_charge*, *driving_traffic_percent*).

2. **Codificación de variables categóricas.** Las dos variables categóricas se transforman a representación *one-hot*, esto es, una columna binaria por categoría. El motivo del viaje (*purpose*) genera cinco columnas (*B*, *HBE*, *HBO*, *HBW*, *NHBO*). El tipo de combustible (*fueltype*) se consolida primero de seis categorías a cuatro (*Petrol*, *Diesel*, *Hybrid*, *Average*), unificando las variantes de furgoneta ligera (*Petrol_LGV*, *Diesel_LGV*) con sus equivalentes de turismo, y después se codifica con *one-hot*. La variable objetivo *travel_mode* se convierte a entero (*walk* = 0, *cycle* = 1, *pt* = 2, *drive* = 3).
3. **División temporal de los datos.** El conjunto se separa por la oleada de encuesta (*survey_year*), no de forma aleatoria. Las oleadas 1 y 2 forman el conjunto de entrenamiento y la oleada 3, la más reciente, el conjunto de prueba. Con una división aleatoria, viajes de un mismo periodo caerían a la vez en entrenamiento y prueba, y el modelo podría aprender tendencias del periodo de prueba en lugar de generalizar hacia el futuro. La separación temporal reproduce la condición real de uso del simulador, que predice sobre escenarios posteriores a los datos con los que se entrenó.
4. **Normalización de variables continuas.** Las dieciséis variables continuas (distancias, duraciones por modo, transbordos, edad, coches del hogar, hora, día y costes) se normalizan con *StandardScaler* de *scikit-learn*, que resta la media y divide por la desviación típica de cada variable para dejarla con media 0 y desviación típica 1. Las variables binarias y las columnas *one-hot* no se escalan. Los modelos de árbol (*XGBoost* y *Random Forest*) son insensibles al escalado, pero se aplica el mismo preprocesado a los tres modelos para que el pipeline de inferencia del backend sea idéntico con cualquiera de ellos.

El escalador se ajusta siempre solo con datos de entrenamiento y nunca con los de evaluación. En la validación cruzada se reajusta sobre el subconjunto de entrenamiento de cada *fold*; para el modelo final, sobre todo el conjunto de entrenamiento. De este modo, la media y la desviación típica que usa el escalador no incorporan información del conjunto de validación, lo que evitaría una fuga de datos que daría una estimación de error demasiado optimista.

5.5.3. Ajuste de hiperparámetros

Los hiperparámetros son los ajustes que gobiernan cómo aprende cada modelo (por ejemplo, el número de árboles o la tasa de aprendizaje) y que no se estiman durante el entrenamiento, sino que hay que fijar de antemano. Para los tres modelos se realizó una búsqueda propia de hiperparámetros con una metodología común: se trabajó sobre una muestra del conjunto de entrenamiento (para acotar el tiempo de cómputo), se evaluó cada configuración con validación cruzada agrupada por hogar y se tomó como criterio la entropía cruzada media en validación, equivalente a maximizar el GMPCA descrito en la Sección 5.5.4. Esta búsqueda nos da una configuración de partida razonable para cada modelo. En el caso particular de XGBoost, además, se dispone de la búsqueda exhaustiva publicada en la investigación de referencia (Martín-Baos, 2023), mucho más costosa y ya validada, que se adopta para el modelo final por encima de la nuestra. Los apartados siguientes detallan cada modelo.

Random Forest

La búsqueda para el Random Forest se hizo con optimización Bayesiana (HyperOpt con el estimador de Parzen estructurado en árbol, TPE), 40 evaluaciones sobre el 25 % del conjunto de entrenamiento. El TPE construye un modelo probabilístico de la relación entre los hiperparámetros y el error, y dirige cada nueva evaluación hacia las regiones del espacio que mejores resultados han dado, de modo que converge antes que una búsqueda en rejilla (que probaría todas las combinaciones) o que una búsqueda aleatoria (que muestrea sin aprender de las pruebas anteriores). El espacio explorado abarcó el número de árboles, la profundidad máxima, los tamaños mínimos de nodo y hoja, y el número de variables candidatas por división. La mejor configuración encontrada (Tabla 5.3) se adoptó como modelo final.

Tabla 5.3: Hiperparámetros del Random Forest hallados por la búsqueda propia (HyperOpt/TPE, 40 evaluaciones).

Parámetro	Valor
n_estimators	600
max_depth	20
min_samples_split	9
min_samples_leaf	5
max_features	sqrt

REVISAR
MODELO
ANTERIOR
PERO NO
RF

Los 600 árboles del bosque se entrenan cada uno sobre una muestra distinta de los datos y de las variables (*bagging*); promediar sus predicciones reduce la varianza del conjunto. La profundidad máxima de 20 niveles, junto con los mínimos de 9 muestras para dividir un nodo y 5 para formar una hoja, impide que los árboles crezcan hasta memorizar casos concretos, lo que limita el sobreajuste. El parámetro `max_features=sqrt` indica que en cada división solo se considera la raíz cuadrada del número total de variables; es el valor que Breiman recomienda para clasificación [Breiman, 2001], ya que decorrelaciona los árboles y mejora la capacidad de generalización del conjunto.

XGBoost

Para XGBoost se ejecutó primero una búsqueda propia equivalente a la del Random Forest (HyperOpt/TPE, 80 evaluaciones sobre el 25 % del conjunto de entrenamiento), que sirvió como base de comparación. No obstante, la investigación de referencia [Martín-Baos, 2023] publica una búsqueda mucho más exhaustiva sobre este mismo dataset, con 1.000 evaluaciones y mayor presupuesto de cómputo, cuyos hiperparámetros (Tabla 5.4) producían mejores resultados que los de nuestra búsqueda reducida. Por ese motivo se adoptaron directamente esos valores para el modelo final, en lugar de los de la búsqueda propia.

Tabla 5.4: Hiperparámetros de XGBoost adoptados de la búsqueda de referencia (HyperOpt/TPE, 1.000 evaluaciones).

Parámetro	Valor
n_estimators	4.809
learning_rate	0,01
max_depth	4
min_child_weight	35
gamma	3,93
max_delta_step	9
subsample	0,767
colsample_bytree	0,934
colsample_bylevel	0,651
reg_alpha (L1)	0,048
reg_lambda (L2)	0,037

La combinación de una tasa de aprendizaje baja (`learning_rate=0,01`) con un número

elevado de árboles (4.809) es característica de un modelo robusto: con pasos pequeños el modelo necesita más iteraciones para converger, pero el resultado tiene menor varianza. La penalización $\gamma=3,93$ y el peso mínimo de hoja $\text{min_child_weight}=35$ actúan como regularización estructural, ya que evitan divisiones que no aporten una ganancia suficiente y reducen el sobreajuste en las clases minoritarias, como la bicicleta.

Red neuronal profunda

Como cada evaluación de la red exige un ciclo completo de entrenamiento, en lugar de la búsqueda Bayesiana se realizó una búsqueda aleatoria acotada: se muestrearon 12 configuraciones del espacio de hiperparámetros (tamaños de las capas, tasa de aprendizaje, *dropout*, regularización y tamaño de lote), cada una evaluada con validación cruzada agrupada por hogar sobre la mitad del conjunto de entrenamiento. La mejor configuración (Tabla 5.5) se adoptó para el modelo final.

Tabla 5.5: Configuración de la red neuronal profunda hallada por la búsqueda propia.

Hiperparámetro	Valor
Neuronas por capa oculta	256, 128, 64
<i>Dropout</i> (capas 1 y 2)	0,4 y 0,3
Optimizador	Adam
Tasa de aprendizaje	0,003
Regularización L2 (<i>weight_decay</i>)	0,0001
Tamaño de lote	512
Función de pérdida	Entropía cruzada (<i>label smoothing</i> 0,1)

La red, representada en la Figura 5.13, transforma el vector de entrada a través de tres capas ocultas de 256, 128 y 64 neuronas y una capa final de 4 salidas, una por modo de transporte. Cada capa oculta encadena cuatro operaciones. Primero, una capa lineal (*Linear*) combina sus entradas mediante una suma ponderada. Después, la normalización por lotes (*BatchNorm*) reescala esas sumas para que tengan media y escala estables dentro de cada lote de entrenamiento, lo que evita que los valores crezcan o se desvanezcan al atravesar las capas y hace que el entrenamiento sea más estable. A continuación, la función de activación ReLU introduce la no linealidad que permite a la red aprender relaciones complejas, dejando pasar los valores positivos y anulando los negativos. Por último, en las dos primeras capas, el *dropout* desactiva al azar una fracción de las neuronas (40 % y 30 %, respectivamente) en

cada paso de entrenamiento, lo que obliga a la red a no depender de neuronas concretas y reduce el sobreajuste.

El orden de estas operaciones importa: la normalización se aplica después de la capa lineal y antes de la activación, sobre las sumas ponderadas y no sobre la salida de la capa anterior, lo que mantiene la entrada a la ReLU en un rango estable a lo largo del entrenamiento. La red se entrena con el optimizador Adam, que adapta el tamaño del paso de actualización para cada parámetro, y la función de pérdida es la entropía cruzada con suavizado de etiquetas (*label smoothing* de 0,1): en lugar de exigir al modelo una probabilidad de 1 para la clase correcta y 0 para el resto, se le pide repartir una pequeña parte de la probabilidad entre todas las clases, lo que evita que se vuelva excesivamente confiado y mejora el ajuste de sus probabilidades. El entrenamiento incorpora además tres mecanismos de control: reducción de la tasa de aprendizaje a la mitad cuando la pérdida de validación deja de mejorar (`ReduceLRonPlateau`), recorte de la magnitud de los gradientes para evitar actualizaciones bruscas (`clip_grad_norm=1,0`) y parada anticipada, que detiene el entrenamiento y recupera el mejor estado si la pérdida de validación no mejora en 10 épocas seguidas. La semilla aleatoria se fijó a 481516 en PyTorch y NumPy para que el entrenamiento sea reproducible.

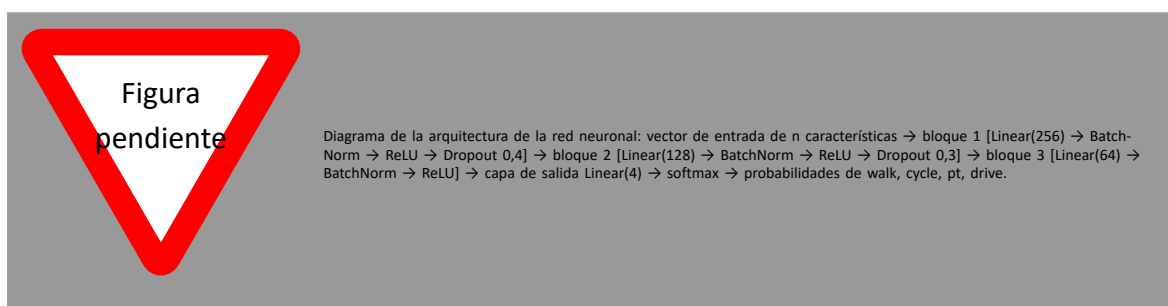


Figura 5.13: Arquitectura de la red neuronal profunda de elección modal.

5.5.4. Entrenamiento y evaluación

Los tres modelos se entrenaron y validaron con validación cruzada de 5 *folds* agrupada por hogar (GroupKFold de scikit-learn, con `household_id` como variable de agrupación). Esta estrategia obliga a que todos los viajes de un mismo hogar queden en el mismo *fold*, nunca repartidos entre entrenamiento y validación. De no ser así se produciría una fuga de datos (*data leakage*), porque los viajes de un mismo hogar comparten las variables del hogar (como el número de coches o la posesión de carnet) y patrones de movilidad muy parecidos:

el modelo podría reconocer al hogar y memorizar su comportamiento en lugar de aprender a generalizar, lo que daría una estimación de error demasiado optimista. Al agrupar por hogar, la validación se aproxima a un escenario *leave-one-household-out*, en el que cada hogar se evalúa con un modelo que no ha visto ninguno de sus viajes, que es justamente la situación real cuando el simulador predice para un viajero nuevo. Las métricas de la Tabla 5.6 son la media de los cinco *folds*.

La evaluación combina dos métricas. La primera es la exactitud (*accuracy*), la proporción de viajes cuyo modo de transporte predicho (el de mayor probabilidad) coincide con el modo real:

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[\hat{y}_i = y_i] \quad (5.1)$$

donde N es el número de viajes evaluados, y_i el modo real del viaje i , $\hat{y}_i$ el modo predicho y $\mathbb{I}[\cdot]$ una función indicadora que vale 1 cuando ambos coinciden y 0 en caso contrario. La exactitud solo atiende al modo más probable e ignora con cuánta seguridad lo predice el modelo. Para medir también la calidad de las probabilidades se parte de la entropía cruzada media H :

$$H = -\frac{1}{N} \sum_{i=1}^N \ln \hat{p}(y_i) \quad (5.2)$$

donde $\hat{p}(y_i)$ es la probabilidad que el modelo asigna al modo realmente elegido en el viaje i . La entropía cruzada penaliza asignar probabilidades bajas a la clase correcta, tanto más cuanto más confiada y equivocada sea la predicción. A partir de ella se define el GMPCA (*Geometric Mean Probability of Correct Alternative*):

$$\text{GMPCA} = \exp(-H) = \exp\left(\frac{1}{N} \sum_{i=1}^N \ln \hat{p}(y_i)\right) \quad (5.3)$$

esto es, la media geométrica de las probabilidades que el modelo asigna al modo correcto. Un clasificador que repartiera probabilidad uniforme entre los cuatro modos obtendría $\text{GMPCA} = 0,25$, y un clasificador perfecto, $\text{GMPCA} = 1,0$. Frente a la exactitud, el GMPCA recompensa que el modelo asigne probabilidades altas al modo correcto y es la métrica más recomendada para la comparación en la literatura de elección modal con aprendizaje automático [José Ángel Martín-Baos, 2023].

Las Tablas 5.6 y 5.7 muestran que los tres modelos alcanzan un rendimiento muy similar, con XGBoost ligeramente por delante en ambas métricas. La diferencia entre la validación cruzada y la prueba se mantiene por debajo de un punto porcentual en todos los modelos, señal de que la división temporal no introduce diferencias apreciables entre las oleadas del

Tabla 5.6: Resultados en entrenamiento con validación cruzada de 5-folds agrupada por hogar (dataset LPMC).

Modelo	Accuracy (%)	GMPCA (%)
Random Forest	74,95	51,51
XGBoost	75,48	52,54
DNN	75,17	51,31

Tabla 5.7: Resultados en el conjunto de prueba independiente (dataset LPMC).

Modelo	Accuracy (%)	GMPCA (%)
Random Forest	74,03	50,60
XGBoost	74,41	51,63
DNN	74,35	50,36

dataset. El GMPCA, en torno al 50–52 %, duplica aproximadamente al de un clasificador aleatorio sobre cuatro clases (25 %), en línea con los resultados de investigaciones previas sobre el mismo dataset [José Ángel Martín-Baos, 2023]. XGBoost obtiene el mayor valor de GMPCA en prueba (51,63 %), una ligera ventaja tanto en exactitud como en la calidad de las probabilidades que asigna.

La elección de XGBoost como modelo principal del simulador no responde solo a esta ventaja numérica, que es estrecha, sino sobre todo a su comportamiento en la práctica ante los ajustes de conocimiento experto que aplica el backend, como se detalla en la Sección 5.5.5.

5.5.5. Ajustes y conocimiento experto

Los tres modelos infieren siempre sobre un número fijo y predefinido de modos de transporte, con independencia de que en un trayecto concreto alguno de esos modos no exista realmente. Para que las predicciones sean razonables en esos casos límite, el backend inyecta dos ajustes de conocimiento experto en las variables de entrada antes de la inferencia. No son correcciones de errores de OSRM o de OTP, que devuelven datos correctos, sino adaptaciones del escenario a lo que el modelo espera; por eso se documentan aquí, junto a los modelos, y no en la capa de enrutado.

El primer ajuste corrige los trayectos sin transporte público real. Cuando OTP devuelve un itinerario compuesto exclusivamente por un tramo a pie, las seis características

CONTROL

de transporte público (tiempo de acceso a la parada, tiempo en autobús, tiempo en ferroviario, tiempo de espera y de desplazamiento en transbordos, y número de transbordos) tomarían valores próximos a cero, que en el dataset caracterizan precisamente a un transporte público rápido y directo. El modelo podría entonces asignar una probabilidad elevada a un modo que, en realidad, no está disponible. Como no es posible eliminar un modo de la inferencia sin cambiar la arquitectura, el backend comprueba si el itinerario contiene algún tramo de transporte público y, si no es así, sobrescribe esas seis características (`dur_pt_access`, `dur_pt_bus`, `dur_pt_rail`, `dur_pt_int_waiting`, `dur_pt_int_walking` y `pt_n_interchanges`) con un valor situado en el extremo superior de su distribución. En concreto, a cada característica se le asigna el valor que, tras el escalado estándar (`StandardScaler`) del modelo, queda cinco desviaciones típicas por encima de la media de entrenamiento: es decir, $\mu + 5\sigma$, tomando la media μ y la desviación típica σ que el escalador aprendió de esa característica. De este modo el modelo percibe un transporte público con tiempos y transbordos extremos, prácticamente inviable, y descarta esa opción sin necesidad de modificar su arquitectura ni el número de variables de entrada.

El segundo ajuste corrige los trayectos muy cortos en coche. En trayectos de menos de 500 m de distancia en línea recta (calculada con la fórmula de Haversine), el backend añade un recargo fijo de 10 minutos al tiempo de conducción, que representa el aparcamiento y el acceso al vehículo. Sin él, el modelo tendería a elegir el coche siempre que su tiempo de conducción fuese menor, ignorando que para distancias tan cortas el sobre coste de aparcar hace más razonable ir a pie.

Ambos ajustes conviene enmarcarlos en una limitación de fondo de los modelos de elección modal, tanto los clásicos de elección discreta como los de aprendizaje automático. Todos maximizan una utilidad y asumen un viajero racional, pero las decisiones reales incorporan un componente de ruido irreducible: no es realista esperar una probabilidad exactamente nula para un modo, de modo que una probabilidad residual del orden del 1 % no debe interpretarse como un error del modelo. Por otra parte, todo modelo de aprendizaje automático pierde fiabilidad al extrapolar fuera del rango de datos con el que se entrenó. El dataset LPMC, por ejemplo, contiene pocos trayectos muy cortos, y de ahí que el segundo ajuste sea necesario. En estas regiones poco cubiertas por los datos, inyectar conocimiento experto en las entradas corrige el comportamiento sin tener que tocar el modelo.

El comportamiento de los tres modelos ante la penalización del transporte público resultó, además, revelador de sus diferencias. Esa penalización lleva las características de transporte público a valores extremos, muy alejados de los vistos en entrenamiento. XGBoost responde como se espera y reduce la probabilidad del transporte público a un valor casi

nulo. El Random Forest, en cambio, está acotado por construcción: su predicción es el promedio de las distribuciones de clase almacenadas en las hojas, por lo que la probabilidad del transporte público desciende pero no llega a anularse, quedándose en un valor residual que el usuario percibe como una opción todavía algo probable. La red neuronal no tiene esa cota: ante entradas tan alejadas de la distribución de entrenamiento puede extrapolar de forma descontrolada y concentrar casi toda la probabilidad en un único modo de transporte. La literatura atribuye a las redes neuronales una capacidad de extrapolación más flexible que la de los modelos de árbol [Goodfellow et al., 2016], pero esa flexibilidad no asegura que la extrapolación sea acertada en un caso concreto. Por todo ello, tanto por su rendimiento como por su respuesta más predecible ante los escenarios límite del simulador, se escogió XGBoost como modelo principal.

6. Conclusiones y trabajo futuro

6.1. Conclusiones técnicas

6.2. Cumplimiento de objetivos

6.3. Trabajo futuro

A. Anexos

A.1. Manual de despliegue y ejecución

Este anexo recoge el procedimiento para desplegar el simulador en un entorno local, tanto de forma automática (recomendada) como mediante los comandos manuales equivalentes que generan los artefactos de enrutado.

A.1.1. Requisitos del entorno

El despliegue solo necesita **Docker** y **Docker Compose** (incluidos en Docker Desktop) y **Git** con la extensión **Git LFS** instalada. Git LFS es imprescindible: los artefactos pesados (grafos OSRM, grafo de OTP, feed GTFS y modelos entrenados) se almacenan mediante esta extensión, de modo que un clon sin LFS descargaría solo punteros de texto en su lugar. El primer arranque descarga las imágenes oficiales de OSRM, OpenTripPlanner, Python y Node, por lo que requiere conexión a internet; las ejecuciones posteriores funcionan sin red.

```
# Clonado del repositorio con los artefactos de Git LFS
git lfs install
git clone https://github.com/ivanuclm/movilidad-urbana.git
cd movilidad-urbana
```

A.1.2. Despliegue automático en un comando

Todo el sistema se levanta con una sola orden ejecutada en la raíz del repositorio:

```
docker compose up --build
```

La orquestación define, además de los servicios de explotación, varios servicios de inicialización idempotentes que se ejecutan una sola vez antes de levantar el sistema y que se omiten si su resultado ya está presente:

- `gtfs-init`: descomprime el feed GTFS (`GTFS_Urbano_Toledo_2026.zip`) en el directorio de datos del backend.
- `osrm-setup`: preprocesa los tres grafos viarios (perfiles de conducción, bicicleta y a pie) si no están ya generados.
- `otp-build`: construye el grafo multimodal de OpenTripPlanner (`graph.obj`) si no existe.

Como los artefactos vienen versionados con Git LFS, en un clon íntegro estos servicios detectan que el trabajo ya está hecho y terminan de inmediato; solo reconstruyen lo que falte (por ejemplo, tras actualizar el feed o en un clon sin LFS). Una vez arrancado, el simulador queda accesible en `http://127.0.0.1:5173`, la documentación interactiva del backend en `http://127.0.0.1:8000/docs` y la comprobación de estado en `http://127.0.0.1:8000/health`.

A.1.3. Generación manual de los artefactos de enrutado

El preprocesado de OSRM se ejecuta con la imagen oficial `osrm/osrm-backend`, que ya incluye los perfiles Lua. Las tres etapas (extracción, particionado y personalización) se repiten para cada perfil sobre el extracto `clm.osm.pbf`, produciendo un directorio de artefactos independiente por modo:

```
# Perfil de conduccion (analogo para bicycle.lua y foot.lua)
docker run --rm -v "${PWD}/osrm-clm/car:/data" osrm/osrm-backend \
  osrm-extract -p /opt/car.lua /data/clm.osm.pbf
docker run --rm -v "${PWD}/osrm-clm/car:/data" osrm/osrm-backend \
  osrm-partition /data/clm.osrm
docker run --rm -v "${PWD}/osrm-clm/car:/data" osrm/osrm-backend \
  osrm-customize /data/clm.osrm
```

El grafo de OpenTripPlanner se construye con un único comando sobre el directorio `otp-toledo/`, que debe contener el extracto OSM y el feed GTFS:

```
docker run --rm -v "${PWD}/otp-toledo:/var/opentripplanner" \
  opentripplanner/opentripplanner:2.5.0 --build --save
```

A.1.4. Verificación

Cada motor OSRM se publica en un puerto distinto del anfitrión (5001, 5002 y 5003 para conducción, bicicleta y a pie, respectivamente; se evita el 5000 por su conflicto con un servicio del sistema en macOS). Una consulta de prueba devuelve "code": "Ok" y la geometría de la ruta:

```
curl "http://localhost:5001/route/v1/driving
    /-4.0356,39.8750;-4.0023,39.8602?overview=full"
```

OpenTripPlanner queda accesible en el puerto 8080 y el backend en el 8000. La documentación OpenAPI de /docs permite ejercitar todos los endpoints desde el navegador.

A.2. Endpoints y contratos de la API

La Tabla 4.3 del capítulo 4 resume los endpoints de la API. Este anexo recoge ejemplos de petición y respuesta de los tres principales. Todas las peticiones POST usan cuerpo JSON y todas las coordenadas son pares (lat, lon) en grados decimales.

A.2.1. Rutas viarias multiperfil (POST /api/osrm/routes)

```
// Petición
{ "origin": {"lat": 39.8785, "lon": -4.0370},
  "destination": {"lat": 39.8601, "lon": -4.0023},
  "profiles": ["driving", "cycling", "foot"] }

// Respuesta (resumida)
{ "origin": {...}, "destination": {...},
  "results": [
    {"profile": "driving", "distance_m": 4120.5, "duration_s":
      612.0,
      "geometry": [{"lat": 39.8785, "lon": -4.0370}, ...]},
    {"profile": "cycling", "distance_m": 3980.1, "duration_s":
      1015.0, "geometry": [...]},
    {"profile": "foot", "distance_m": 3760.7, "duration_s":
      2890.0, "geometry": [...]}
  ] }
```

A.2.2. Itinerario de transporte público (POST /api/otp/routes)

```
// Peticion (date y time son opcionales; por defecto 2026-05-21
12:00)
{ "origin": {"lat": 39.8785, "lon": -4.0370},
  "destination": {"lat": 39.8601, "lon": -4.0023},
  "itinerary_index": 0, "date": "2026-05-21", "time": "08:00" }

// Respuesta (resumida)
{ "result": {
  "distance_m": 5230.0, "duration_s": 1500.0,
  "start_time": "08:22", "end_time": "08:47",
  "itinerary_index": 0, "total_itineraries": 5,
  "segments": [
    {"mode": "WALK", "distance_m": 400.0, "duration_s": 360.0, "
      geometry": [...]},
    {"mode": "BUS", "route_short_name": "L91", "from_stop_name":
      "...",
      "to_stop_name": "...", "departure": "08:28", "arrival": "
        08:33",
      "stops": [{"name": "...", "lat": 39.87, "lon": -4.03, "time"
        : "08:28"}, ...],
      "geometry": [...]},
    {"mode": "WALK", "distance_m": 220.0, "duration_s": 180.0, "
      geometry": [...]}
  ] } }
```

A.2.3. Inferencia de elección modal (POST /api/lpmc/predict)

```
// Peticion (model_variant opcional; por defecto, la variable
LPMC_MODEL_VARIANT)
{ "origin": {"lat": 39.8785, "lon": -4.0370},
  "destination": {"lat": 39.8601, "lon": -4.0023},
  "itinerary_index": 0, "model_variant": "xgb",
  "user_profile": {
    "purpose": "HBW", "fueltype": "Petrol", "day_of_week": 2,
    "start_time_linear": 8.25, "age": 36, "female": 0,
    "driving_license": 1, "car_ownership": 1,
```

```
"cost_transit": 1.5, "cost_driving_total": 3.5 } }
```

```
// Respuesta (resumida)  
{ "predicted_mode": "drive", "confidence": 0.58,  
  "probabilities": {"walk": 0.07, "cycle": 0.03, "pt": 0.32, "drive  
    ": 0.58},  
  "route_features": {"distance": 4120.5, "dur_driving": 0.17, "  
    dur_pt_bus": 0.21, ...},  
  "model_info": {"model_path": "...", "pt_available": true, "  
    short_trip": false} }
```

A.3. Configuración experimental y reproducibilidad

Los hiperparámetros de los tres modelos se detallan en la sección 5.5.3 (Tablas 5.3, 5.4 y 5.5 para Random Forest, XGBoost y la red neuronal, respectivamente) y los resultados de validación cruzada y prueba en la sección 5.5.4 (Tablas 5.6 y 5.7). Este anexo reúne las condiciones que aseguran la reproducibilidad de esos experimentos:

- **Dataset:** London Passenger Mode Choice (LPMC), con división temporal no aleatoria (oleadas 1 y 2 para entrenamiento, oleada 3 para test).
- **Validación:** validación cruzada de 5 *folds* agrupada por hogar (GroupKFold con `household_id` como grupo), de modo que ningún hogar aparece simultáneamente en entrenamiento y validación.
- **Preprocesado:** `StandardScaler` ajustado solo sobre el subconjunto de entrenamiento de cada fold; codificación *one-hot* de las variables categóricas `purpose` y `fueltype`; exclusión de `household_id` de las características.
- **Semilla aleatoria:** 481516, fijada en NumPy y PyTorch.
- **Bibliotecas:** scikit-learn (Random Forest y escalado), XGBoost (*gradient boosting*) y PyTorch (red neuronal profunda).
- **Métricas:** exactitud (*accuracy*) y GMPCA (*Geometric Mean Probability of Correct Alternative*).

La Tabla A.1 detalla las variables de entrada del modelo, agrupadas según procedan de los servicios de enrutado (OSRM y OTP) o del perfil sociodemográfico aportado por el usuario, tal como se introdujo en la Sección 5.5.1.

Tabla A.1: Variables de entrada al modelo de elección modal. Las variables de ruta se derivan automáticamente de OSRM y OTP; las sociodemográficas las aporta el usuario a través de la interfaz.

Variable	Tipo / Rango	Descripción
<i>Variables de ruta (derivadas de OSRM y OTP)</i>		
distance	float, m	Distancia en coche (OSRM driving).
dur_driving	float, h	Tiempo de conducción (OSRM driving).
dur_cycling	float, h	Tiempo en bicicleta (OSRM cycling).
dur_walking	float, h	Tiempo a pie hasta destino (OSRM foot).
dur_pt_access	float, h	Tiempo del primer tramo a pie hasta la parada (OTP).
dur_pt_bus	float, h	Tiempo acumulado en autobús (OTP).
dur_pt_rail	float, h	Tiempo acumulado en ferroviario (OTP).
dur_pt_int_waiting	float, h	Tiempo de espera en transbordos (OTP).
dur_pt_int_walking	float, h	Desplazamiento a pie entre transbordos (OTP).
pt_n_interchanges	int, ≥ 0	Número de transbordos (OTP).
<i>Variables sociodemográficas y contextuales (aportadas por el usuario)</i>		
age	int, [16, 100]	Edad del viajero.
female	int, {0, 1}	Sexo (1 = mujer).
driving_license	int, {0, 1}	Posesión de carnet de conducir.
car_ownership	int, [0, 3]	Número de coches en el hogar.
day_of_week	int, [1, 7]	Día de la semana.
start_time_linear	float, [0, 24]	Hora de inicio del viaje (decimal).
cost_transit	float, ≥ 0	Coste del billete de transporte público.
cost_driving_total	float, ≥ 0	Coste total estimado del viaje en coche.
purpose	cat. (one-hot)	Motivo del viaje: B, HBE, HBO, HBW, NHBO.
fueltype	cat. (one-hot)	Tipo de combustible: Average, Diesel, Hybrid, Petrol.

A.4. Evidencias de sprints

La planificación y el desarrollo siguieron un proceso Scrum adaptado, descrito en el capítulo 3, donde se detallan los sprints con sus objetivos, fechas y resultados. La trazabilidad fina del trabajo (tareas, decisiones técnicas y artefactos) queda registrada en el historial de control de versiones del repositorio público del proyecto (<https://github.com/ivanuclm/movilidad-urbana>), cuyos mensajes de *commit* documentan la evolución incremental del sistema sprint a sprint.

Referencia bibliográfica

- [Breiman, 2001] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1):5–32.
- [Chen and Guestrin, 2016] Chen, T. and Guestrin, C. (2016). Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794.
- [danpat, 2017] danpat (2017). Foot and bike are returning the same route as car (issue #4034). <https://github.com/Project-OSRM/osrm-backend/issues/4034>. Project OSRM, GitHub. Consultado el 28 de mayo de 2026.
- [Delling et al., 2017] Delling, D., Goldberg, A. V., Pajor, T., and Werneck, R. F. (2017). Customizable route planning in road networks. *ACM Journal of Experimental Algorithmics*, 22:1–48.
- [EMT Madrid, 2026] EMT Madrid (2026). Nap transporte multimodal – fichero de detalle 896 (autobús urbano madrid). <https://nap.transportes.gob.es/Files/Detail/896>. Consultado el 6 de mayo de 2026.
- [EMT Valencia, 2026] EMT Valencia (2026). Nap transporte multimodal - fichero de detalle 965 (autobus urbano de valencia). <https://nap.transportes.gob.es/Files/Detail/965>. Consultado el 23 de febrero de 2026.
- [FOSSGIS e.V., 2026] FOSSGIS e.V. (2026). Öffentliche osrm-routinginstanzen. <https://routing.openstreetmap.de>. Consultado el 28 de mayo de 2026.
- [Friedman, 2001] Friedman, J. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29:1189–1232.

-
- [Geisberger et al., 2008] Geisberger, R., Sanders, P., Schultes, D., and Delling, D. (2008). Contraction hierarchies: Faster and simpler hierarchical routing in road networks. In *Proceedings of the 7th International Conference on Experimental Algorithms (WEA)*, pages 319–333.
- [Geofabrik GmbH, 2026] Geofabrik GmbH (2026). Geofabrik download server – spain: Castilla-la mancha. <https://download.geofabrik.de/europe/spain/castilla-la-mancha.html>. Consultado el 6 de mayo de 2026.
- [GitHub, 2024] GitHub (2024). About billing for git large file storage. <https://docs.github.com/en/billing/managing-billing-for-git-large-file-storage/about-billing-for-git-large-file-storage>. Consultado el 30 de junio de 2026.
- [Goodfellow et al., 2016] Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press.
- [Google, 2026a] Google (2026a). Google maps platform billing and pricing. <https://developers.google.com/maps/billing-and-pricing/pricing>. Consultado el 22 de febrero de 2026.
- [Google, 2026b] Google (2026b). Routes api overview. <https://developers.google.com/maps/documentation/routes/overview>. Consultado el 22 de febrero de 2026.
- [Google LLC,] Google LLC. Encoded polyline algorithm format. <https://developers.google.com/maps/documentation/utilities/polylinealgorithm>. Google Maps Platform. Consultado el 28 de mayo de 2026.
- [Haklay, 2010] Haklay, M. (2010). How good is volunteered geographical information? a comparative study of openstreetmap and ordnance survey datasets. *Environment and Planning B: Planning and Design*, 37(4):682–703.
- [HeiGIT gGmbH, 2026] HeiGIT gGmbH (2026). openrouteservice api documentation. <https://giscience.github.io/openrouteservice/api-reference/>. Consultado el 23 de febrero de 2026.
- [Hillel, 2019] Hillel, T. (2019). London passenger mode choice (lpmc) – dataset description. Documento tecnico del dataset (archivo local: papers/CS_LPMC.pdf). Consultado el 22 de febrero de 2026.
-

- [Hillel et al., 2018] Hillel, T., Elshafie, M. Z. E. B., and Jin, Y. (2018). Recreating passenger mode choice-sets for transport simulation: A case study of london, uk. *Proceedings of the Institution of Civil Engineers - Smart Infrastructure and Construction*, 171(1):29–42.
- [Horni et al., 2016] Horni, A., Nagel, K., and Axhausen, K. W., editors (2016). *The Multi-Agent Transport Simulation MATSim*. Ubiquity Press.
- [José Ángel Martín-Baos, 2023] José Ángel Martín-Baos, Ricardo García Rodenas, L. R. B. (2023). A prediction and behavioural analysis of machine learning methods for modelling travel mode choice. *Transportation Research Part C: Emerging Technologies*, 156:104318.
- [Le Cam and React-Leaflet Contributors, 2024] Le Cam, P. and React-Leaflet Contributors (2024). React leaflet — React components for leaflet maps. <https://react-leaflet.js.org>. Consultado el 10 de junio de 2026.
- [Leaflet Contributors, 2024] Leaflet Contributors (2024). Leaflet — an open-source JavaScript library for mobile-friendly interactive maps. <https://leafletjs.com>. Consultado el 10 de junio de 2026.
- [Linsley and TanStack Contributors, 2024] Linsley, T. and TanStack Contributors (2024). Tanstack query — powerful asynchronous state management. <https://tanstack.com/query>. Consultado el 8 de junio de 2026.
- [Lopez et al., 2018] Lopez, P. A., Behrisch, M., Bieker-Walz, L., Erdmann, J., Flötteröd, Y.-P., Hilbrich, R., Lücken, L., Rummel, J., Wagner, P., and Wießner, E. (2018). Microscopic traffic simulation using sumo. In *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*, pages 2575–2582.
- [Lucide Contributors, 2024] Lucide Contributors (2024). Lucide — beautiful & consistent icon toolkit. <https://lucide.dev>. Consultado el 18 de junio de 2026.
- [Mapbox, 2026] Mapbox (2026). Directions api. <https://docs.mapbox.com/api/navigation/directions/>. Consultado el 23 de febrero de 2026.
- [Martín-Baos, 2023] Martín-Baos, J. A. (2023). *Machine Learning Methods Applied to Transport Demand Modelling*. Phd thesis, University of Castilla-La Mancha. Tesis doctoral.
- [McFadden, 1974] McFadden, D. (1974). Conditional logit analysis of qualitative choice behavior. In Zarembka, P., editor, *Frontiers in Econometrics*, pages 105–142. Academic Press.

[Meta Open Source, 2024] Meta Open Source (2024). React — the library for web and native user interfaces. <https://react.dev>. Consultado el 8 de junio de 2026.

[Microsoft, 2024] Microsoft (2024). Typescript — JavaScript with syntax for types. <https://www.typescriptlang.org>. Consultado el 8 de junio de 2026.

[MobilityData, 2026a] MobilityData (2026a). General transit feed specification (GTFS) — overview. <https://gtfs.org/documentation/overview/>. Consultado el 24 de junio de 2026.

[MobilityData, 2026b] MobilityData (2026b). Gtfs schedule best practices. <https://gtfs.org/documentation/schedule/schedule-best-practices/>. Consultado el 23 de febrero de 2026.

[MobilityData, 2026c] MobilityData (2026c). Gtfs schedule reference. <https://gtfs.org/documentation/schedule/reference/>. Consultado el 23 de febrero de 2026.

[MobilityData, 2026d] MobilityData (2026d). Gtfs validator (github repository). <https://github.com/MobilityData/gtfs-validator>. Consultado el 23 de febrero de 2026.

[NAP Transporte Multimodal, 2026a] NAP Transporte Multimodal (2026a). Nap transporte multimodal - home. <https://nap.transportes.gob.es/>. Consultado el 23 de febrero de 2026.

[NAP Transporte Multimodal, 2026b] NAP Transporte Multimodal (2026b). Nap transporte multimodal - quienes somos. <https://nap.transportes.gob.es/quienes-somos>. Consultado el 23 de febrero de 2026.

[openrouteservice, 2026] openrouteservice (2026). Api restrictions and limits. <https://openrouteservice.org/restrictions/>. Consultado el 23 de febrero de 2026.

[OpenTripPlanner Team, 2026a] OpenTripPlanner Team (2026a). Build configuration (opentripplanner documentation). <https://docs.opentripplanner.org/en/latest/BuildConfiguration/>. Consultado el 23 de febrero de 2026.

[OpenTripPlanner Team, 2026b] OpenTripPlanner Team (2026b). Opentripplanner documentation. <https://docs.opentripplanner.org/en/latest/>. Consultado el 23 de febrero de 2026.

- [OpenTripPlanner Team, 2026c] OpenTripPlanner Team (2026c). Opentripplanner (github repository). <https://github.com/opentripplanner/OpenTripPlanner>. Consultado el 23 de febrero de 2026.
- [OpenTripPlanner Team, 2026d] OpenTripPlanner Team (2026d). Routerequest (opentripplanner documentation). <https://docs.opentripplanner.org/en/latest/RouteRequest/>. Consultado el 23 de febrero de 2026.
- [OSM Foundation, 2026] OSM Foundation (2026). About openstreetmap. <https://www.openstreetmap.org/about>. Consultado el 23 de febrero de 2026.
- [Project OSRM, 2026a] Project OSRM (2026a). Osmr api documentation v5.24.0. <https://project-osrm.org/docs/v5.24.0/api/>. Consultado el 23 de febrero de 2026.
- [Project OSRM, 2026b] Project OSRM (2026b). osrm-backend (github repository). <https://github.com/Project-OSRM/osrm-backend>. Consultado el 23 de febrero de 2026.
- [Schwaber and Sutherland, 2020] Schwaber, K. and Sutherland, J. (2020). The scrum guide: The definitive guide to scrum: The rules of the game. <https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf>. Consultado el 17 de marzo de 2026.
- [TomTom, 2026] TomTom (2026). Routing api - calculate route. <https://developer.tomtom.com/routing-api/documentation/tomtom-maps/calculate-route>. Consultado el 23 de febrero de 2026.
- [Train, 2009] Train, K. E. (2009). *Discrete Choice Methods with Simulation*. Cambridge University Press, 2 edition.
- [Unauto, 2026] Unauto (2026). Nap transporte multimodal - fichero de detalle 1377 (autobuses urbanos toledo). <https://nap.transportes.gob.es/Files/Detail/1377>. Consultado el 23 de febrero de 2026.
- [Vite Team, 2024] Vite Team (2024). Vite — next generation frontend tooling. <https://vite.dev>. Consultado el 8 de junio de 2026.